UNIVERSIDAD POLITÉCNICA DE MADRID

Escuela Técnica Superior de Ingenieros Informáticos

Departamento de Lenguajes y Sistemas Informáticos e Ingeniería de Software

A Technology-Agnostic Approach to Auto-Scale Services in Heterogeneous Clouds

Thesis submitted for the degree of Doctor of Philosophy

Supervisors: **Prof. Francisco Javier Soriano Camino**
Ph.D. in Computer Science

Prof. David Lizcano Casas
Ph.D. in Computer Science

Author:
Víctor Rampérez Martín
M.Sc. in Informatics Engineering

2021

Departamento de Lenguajes y Sistemas Informáticos
e Ingeniería de Software

Escuela Técnica Superior de Ingenieros Informáticos

A Technology-Agnostic Approach to Auto-Scale Services in Heterogeneous Clouds

Author: Víctor Rampérez
M.Sc. in Informatics Engineering

Supervisors: Prof. Francisco Javier Soriano Camino
Ph.D. in Computer Science

Prof. David Lizcano Casas
Ph.D. in Computer Science

2021

To my parents Ana and Alfredo,
to my brother Alberto,
and to my girlfriend Isabel.

Acknowledgements

These lines should serve to name all those people who have directly or indirectly supported and encouraged me over the years, without whom this work would not be a reality today. However, I am aware that most of them will not be mentioned. Please serve these lines to express my gratitude and my consideration to them.

I would like to make a special mention to my directors of Thesis, Doctors Javier Soriano Camino and David Lizcano Casas, whom I hold in high esteem and admiration. Without their experience, advice and guidance, this work would not have been possible. Your contribution throughout the process of making this work is of great value.

I wish to show my gratitude to my family for all their support, love and inspiration that have allowed me to get this far; especially to my parents Ana and Alfredo, my brother Alberto and my girlfriend Isabel, to whom I wanted to dedicate this work. Their contribution during all this work is of incalculable value to me.

I would also like to express my deepest gratitude to Dr. Sergio Vavassori, whom I sincerely admire. His contribution goes far beyond this work, both temporarily and qualitatively. Thank you with all my heart for transmitting to me your passion for this profession as well as for scientific values, and above all, for your infinite patience with me. Without you, this doctoral thesis would never even have been conceived.

I wish to thank Professor Arminda Moreno for her great guidance with all of the data analysis and statistical modelling of this work.

I would like to thank all the people of the Politecnico di Milano, and especially Professor Della Valle, for all that I have learned with them and for their good reception, which made me feel at home in the last phase of my doctoral studies.

Finally, I would not like to end these acknowledgements without mentioning my colleagues from the ETS de Ingenieros Informáticos of the UPM. More specifically to the colleagues of Computer Networks and the CETTICO research group: Miguel Ortega, Miguel Jiménez, Sonia, Genoveva, Agustín, Guillermo, Raúl and Jose Luis. I am enormously grateful to them for their friendship and for the environment and work context that have allowed me to develop this work.

Abstract

In recent years we have seen a digital revolution that is creating more digitised societies, in which many aspects of our daily lives depend on Internet services. It is therefore not surprising that computational resources have become essential services in these societies. As a result, cloud computing emerges as a way to offer computing resources on demand and with a pay-per-use model.

One of the key aspects of cloud computing is elasticity, being the ability to acquire and release computing resources on demand. These requirements are often formally expressed in a Service Level Agreement (SLA). The wide variety of these SLAs makes it difficult for consumers to adequately estimate the amount of computational resources needed at any given time.

To solve this problem, auto-scaling systems are responsible for ensuring compliance with SLAs automatically, adding and/or releasing the necessary amount of computational resources needed at any given moment, minimising over-provisioning and under-provisioning times.

Despite the wide variety of auto-scaling systems, there are a number of limitations and shortcomings that prevent the full exploitation of their potential. On the one hand, the proposals of the main cloud computing providers rely on reactive auto-scaling techniques, which lack the necessary anticipation to avoid violating SLAs. Moreover, these systems often do not automatically detect which resource needs to be scaled. Similarly, it is also often left to the consumer to translate their requirements (SLAs) into specific configurations of these systems. Finally, auto-scaling systems that can be used in multi-cloud environments are scarce, mainly due to the lack of a set of monitoring metrics that can be measured across multiple providers.

To address these shortcomings and limitations, this work has designed and implemented FLAS (Forecasted Load Auto-Scaling), an SLA-centric autoscaling system for distributed systems that automatically detects bottleneck computational resources and decides how much and when additional resources should be added or released to meet the SLAs. For this purpose, FLAS combines proactive and reactive auto-scaling techniques.

In addition, a knowledge-based system has been designed that is able to translate, automatically and transparently for the consumer, SLAs into conditions expressed through low-level metrics. Finally, a repository of vendor-neutral monitoring metrics has been built that can be measured in these providers (multi-cloud environments) by means of a set of functions that allow mapping neutral metrics with those exported by each provider.

In order to evaluate this work, a series of experiments have been carried out that have demonstrated its validity. On the one hand, FLAS has been evaluated as an auto-scaler of a distributed publish-subscribe cloud system. The results of this evaluation show that FLAS is capable of ensuring compliance with the requirements of the SLAs more than 99% of the time, taking the most appropriate decisions and scaling actions at all times.

With all this, it can be concluded that it is possible to automatically scale cloud services through the use of auto-scaling systems that allow (i) compliance and adherence to the SLAs specified by the cloud consumers, (ii) automatic detection of the computational resources that are the bottleneck, and (iii) monitoring of the essential metrics for scaling decisions in multi-cloud environments.

Resumen

En los últimos años hemos sido testigos de una revolución digital que está generando sociedades más digitalizadas, en las cuales, gran parte de los aspectos de nuestra vida cotidiana dependen de servicios en Internet. Por ello, no es de extrañar que los recursos computacionales se hayan convertido en servicios esenciales en estas sociedades. Como resultado, la computación en la nube surge como una forma de ofrecer recursos de computación bajo demanda y con un modelo de pago por uso.

Uno de los aspectos claves de la computación en la nube es la elasticidad, siendo esta la capacidad para adquirir y liberar recursos computacionales bajo demanda. Estos requisitos se suelen expresar formalmente en lo que se conoce como Acuerdo de Nivel de Servicio o *Service Level Agreement* (SLA). La gran variedad de estas SLAs, hace que sea complicado estimar de forma adecuada por los consumidores la cantidad de recursos computacionales necesarios en cada momento.

Para solventar ese problema, los sistemas de auto-escalado se encargan de asegurar el cumplimiento de las SLAs de forma automática, añadiendo y/o liberando la cantidad necesaria de recursos computacionales necesaria en cada instante minimizando los tiempos de aprovisionamiento por exceso y por defecto.

A pesar de la gran variedad de sistemas de auto-escalado, existen una serie de limitaciones y carencias que impiden explotar por completo su verdadero potencial. Por un lado, las propuestas de los principales proveedores de computación en la nube apuestan por técnicas de auto-escalado reactivas, que carecen de la anticipación necesaria para no violar las SLAs. Además, estos sistemas no suelen detectar de forma automática que recurso es el que debe ser escalado. Del mismo modo, también se suele dejar del lado del consumidor la traducción de sus requisitos (SLAs) en configuraciones concretas de estos sistemas. Finalmente, los sistemas de auto-escalado que pueden ser utilizados en entornos *multi-cloud* son escasos, debido principalmente a la falta de un conjunto de métricas de monitorización que puedan ser medidas en múltiples proveedores.

Para abordar estas carencias y limitaciones, en el presente trabajo se ha diseñado e implementado FLAS (Forecasted Load Auto-Scaling), un sistema de auto-escalado para sistemas distribuidos centrado en las SLAs que detecta de forma automática los recursos computacionales que son el cuello de botella y decide la cantidad y el momento en que deben añadirse o liberarse recursos adicionales para cumplir las SLAs. Para ello, FLAS combina técnicas de auto-escalado proactivas y reactivas.

Además, se ha diseñado un sistema basado en el conocimiento capaz de traducir, de forma automática y transparente para el consumidor, las SLAs en condiciones expresadas mediante métricas de bajo nivel. Finalmente, se ha construido un repositorio de métricas de monitorización independientes del proveedor y que pueden ser medidas en estos proveedores (entornos *multi-cloud*) mediante un conjunto de funciones que permiten relacionar las métricas neutrales con las exportadas por cada proveedor.

Para evaluar este trabajo, se han llevado a cabo una serie de experimentos que han demostrado la validez del mismo. Por un lado, se ha evaluado FLAS como auto-escalador de un sistema distribuido publicador-subscriptor cloud. Los resultados de esta evaluación muestran como FLAS es capaz de asegurar el cumplimiento de los requisitos de las SLAs durante más del 99 % del tiempo, tomando las decisiones y acciones de escalado más apropiadas en cada instante.

Con todo ello, se puede concluir que es posible escalar de forma automáticas servicios cloud mediante el uso de sistemas de auto-escalado que permitan (i) cumplimiento y adherencia a las SLAs especificadas por los consumidores cloud, (ii) detección automática de los recursos computacionales que son el cuello de botella, y (iii) la monitorización de las métricas esenciales para la toma de decisiones de escalado en entornos *multi-cloud*.

Acronyms and Abbreviations

ANN	Artificial Neural Network
AP	Access Point
AR	Autoregressive model
ARMA	Moving Average Autoregressive model
ARIMA	Moving Average Integrated Autoregressive model
AWS	Amazon Web Services
CBPS	Content-based publish-subscribe
CP	Connection Point
EP	Exit Point
IaaS	Infrastructure-as-a-Service
IoT	Internet of Things
KPI	Key Performance Indicator
M	Matcher
MA	Moving average model
PaaS	Platform-as-a-Service
Pub/Sub	Publish-Subscribe
SaaS	Software-as-a-Service
SLA	Service Level Agreement
SLO	Service Level Objective
VM	Virtual Machine

Contents

1 Introduction **1**

1.1 Motivation 1

1.2 Statement of the problem 4

1.3 Definitions of terms 7

1.4 Description of the remaining chapters 13

1.5 Derived publications 14

2 State of the Art and Related Work **17**

2.1 Elasticity in cloud computing 17

2.1.1 Reactive auto-scaling 18

2.1.1.1 Threshold-based rules 18

2.1.2 Proactive auto-scaling 20

2.1.2.1 Time series analysis and forecasting 20

2.1.3 About existing auto-scalers 22

2.2 Multi-cloud metrics 26

2.2.1 Commercial solutions 26

2.2.2 Academic solutions 27

2.3 Content-based publish-subscribe systems 31

2.3.1 Improved version of E-SilboPS 31

2.3.1.1 General overview 32

2.3.1.2 Improving the performance of E-SilboPS . . 34

2.3.1.3 Discussion 41

3 Goals and Contributions 43

3.1 Objectives 43

3.2 Hypothesis and research questions 45

3.3 Contributions 46

3.4 Limitations 47

4 Combining proactive and reactive auto-scaling in an architecture for distributed systems: FLAS 49

4.1 System model 50

4.1.1 System configuration model 50

4.1.2 Workload and performance model 51

4.1.3 Problem statement 54

4.2 FLAS architecture 55

4.2.1 Scaling Time Forecaster 57

4.2.2 Workload Trend Forecaster 57

4.2.3 Performance Forecaster 58

4.2.4 Decider 59

4.3 FLAS for E-SilboPS 59

4.3.1 Scaling Time Forecaster and Workload Trend Forecaster implementation 60

4.3.2 Performance Forecaster implementation 62

4.3.3 Decider 65

4.4 Experimental evaluation 69

4.4.1 Test Case 1: Stationary peak 72

4.4.2 Test Case 2: Non-stationary peak 74

4.4.3 Test Case 3: Steady increase 74

4.4.4 Test Case 4: Isolated and close to SLA limit workload peak 76

4.4.5 Test Case 5: Consecutive and close to SLA limit workload peaks 77

4.4.6 Quantitative analysis of results 79

4.5 Discussion of results 84

5 Establishing a Set of Vendor-Neutral Cross-Provider Measurable Metrics 87

5.1 Proposed solution overview 88

5.2 From SLA to low-level conditions 91

5.3 Towards a set of vendor-neutral cross-provider measurable metrics 95

5.3.1 IaaS metrics 96

5.3.2 PaaS metrics 96

5.3.3 Context metrics 109

5.4 Evaluation 112

5.4.1 Multi-cloud publish/subscribe middleware 112

5.4.2 Multi-cloud RDBMS 117

5.5 Discussion of results 125

6 Conclusions **127**

7 Future Work **131**

7.1 About auto-scalers and FLAS 131

7.2 About Vendor-Neutral Cross-Provider Measurable Metrics . . 132

Bibliography **134**

Appendix A Vendor-Neutral Cross-Provider Measurable Metrics **143**

A.0.1 Metrics exported for each service by the main cloud providers 143

A.0.2 Repository of vendor-neutral metrics and their mappings with the metrics of the main cloud providers . . 216

List of Figures

2.1 PASCAL'S architecture 25

2.2 SALOON platform overview 29

2.3 LoM2HiS framework architecture 29

2.4 E-SilboPS architecture and operational flow 33

2.5 Exchange of messages during the scale-out of the Matchers layer 35

2.6 Throughput of the new E-SilboPS version with different deployment configurations. 37

2.7 Comparison of old and new E-SilboPS versions throughput with different deployment configurations 38

2.8 Comparison of E-SilboPS throughput and notification processing 39

2.9 Updated matcher architecture 40

4.1 Time points of interest in scaling actions 53

4.2 Functional diagram of the components that form the FLAS architecture 56

4.3 Comparison between a time series of the response time data before and after applying a Savitzky-Golay digital filter . . . 62

4.4 Comparison of the MAE value in the three time series analysis models tested by cross-validation for workload trend forecast 63

4.5 Forecasts of the values of $\frac{\delta RT}{\delta t_i}$ in a prediction time horizon using a harmonic regression model 63

4.6 Results of the cross validation carried out using different predictive FLAS models 66

4.7 Relative frequency histogram and standardized density function of the relative error of response time estimation 70

4.8 Test case 1: Stationary peak 73

4.9 Test case 2: Stationary peak 75

4.10 Test case 3: Steady increase 76

4.11 Test Case 4: Isolated and close to SLA limit workload peak . 77

4.12 Test Case 5: Consecutive and close to SLA limit workload peaks 78

4.13 Comparison of FLAS with other auto-scaling techniques in terms of percentage of time that the SLA is violated and percentage of time in over-provisioning. 83

5.1 Overview of the multi-cloud reference environment used . . . 90

5.2 CPU-bound vs Memory-bound brokers 93

5.3 Expert system to translate low-level metrics into high-level metrics 94

5.4 Metric mapping diagram between equivalent services from different cloud providers 99

5.5 Production rules that are part of the knowledge base for the multi-cloud publish/subcribe middleware use case 115

5.6 Production rules that form the knowledge base for the MySQL OLTP use case 119

List of Tables

2.1 Summary of academic cloud brokering solutions. 30

4.1 Results of the percentage of under-provisioning, over-provisioning and violation of the maximum response time imposed by the SLA for the different test cases. In the case of the Stationary Peak test case, the predictive and reactive X-Y versions are compared (being X the scale-out threshold and Y the size of the reactive window respectively). 81

4.2 Results of the scaling times (T) and the error in the prediction of these times for the scale-out and scale-in actions of the different test cases. 81

5.1 IaaS vendor neutral metrics. 97

5.2 Equivalence of main cloud provider services and their mapping relationship 101

5.3 Block Storage Vendor-Neutral Metrics 102

5.4 Object Storage Vendor-Neutral Metrics 103

5.5 PostgreSQL Vendor-Neutral Metrics. 104

5.6 MySQL Vendor-Neutral Metrics 105

5.7 Example 1 of mapping functions 107

5.8 Example 2 of mapping functions 107

5.9 Context vendor-neutral metrics 110

5.10 Mapping example of two regions defined by AWS and their respective values in vendor-neutral metrics 110

5.11 Relationship between the vendor-neutral regions and the regions defined by each cloud provider 111

5.12 Inference process for the translation of SLOs into conditions expressed as vendor-neutral metrics for the multi-cloud publish/subscribe middleware use case 116

5.13 Inference process for the translation of SLOs in conditions expressed as vendor-neutral metrics for the MySQL OLTP use case 121

5.14 Mapping functions that relate MySQL OLTP use case vendor-neutral metrics to the corresponding metrics of each of the main cloud providers 124

A.1 AWS RDS (Relational Database Service) 144

A.2 AWS EBS (Elastic Block Service) 145

A.3 AWS ApplicationELB (Elastic Load Balancer) 148

A.4 AWS NetworkELB (Elastic Load Balancer) 152

A.5 AWS S3 (Simple Storage Service) 154

A.6 AWS EFS (Elastic File System) 156

A.7 AWS Lambda 158

A.8 AWS ECS (Elastic Container Service 159

A.9 AWS DX (Direct Connect) 162

A.10 AWS EMR (Elastic Map Reduce) 162

A.11 AWS KinesisAnalytic 166

A.12 AWS Regions 172

A.13 GC Cloud SQL 174

A.14 GC Peersistent Disk 177

A.15 GC Load Balancing 178

A.16 GC Cloud Storage 182

A.17 GC Filestore 183

A.18 GC Cloud Functions 184

A.19 GC Kubernetes 185

A.20 GC Interconnect 188

A.21 GC Dataproc 190

A.22 GC Dataflow 193

A.23 GC Regions 195

A.24 Microsoft.DBforMySQL/servers 196

A.25 Microsoft.DBforPostgreSQL/servers 197

A.26 Microsoft Azure Disk Storage 198

A.27 Microsoft.Network/applicationGateways 199

A.28 Microsoft.Network/virtualNetworkGateways 200

A.29 Microsoft.Storage/storageAccounts/blobServices 202

A.30 Microsoft.Storage/storageAccounts/fileServices 204

A.31 Microsoft.Web/sites (functions) 206

A.32 Microsoft.ContainerInstance/containerGroups and /managed-Clusters 208

A.33 Microsoft.Network/expressRouteCircuits 209

A.34 Microsoft.Network/expressRouteCircuits 210

A.35 Microsoft.StreamAnalytics/streamingjobs 213

A.36 Microsoft Regions 214

A.37 VNM Regions 216

A.38 VNM Stream Processing Related Metrics 218

A.39 VNM Stream Processing Mapping 218

A.40 VNM Batch Data Processing Related Metrics 218

A.41 VNM Batch Data Processing Mappings 219

A.42 VNM Dedicated Interconnect Related Metrics 219

A.43 VNM Dedicated Interconnect Mappings 219

A.44 VNM Containers Service Related Metrics 219

A.45 VNM Containers Service Mappings 220

A.46 VNM MySQL Related Metrics 220

A.47 VNM MySQL Mappings (AWS and GC) 221

A.48 VNM MySQL Mappings (M. Azure) 222

A.49 VNM PostgreSQL Related Metrics 223

A.50 VNM PostgreSQL Mappings (AWS and GC) 223

A.51 VNM PostgreSQL Mappings (M. Azure) 224

A.52 VNM Block Storage Related Metrics 225

A.53 VNM Block Storage Mappings (AWS and GC) 225

A.54 VNM Block Storage Mappings (M. Azure) 225

A.55 VNM Object Storage Related Metrics 226

A.56 VNM Object Storage Mappings 227

A.57 VNM Network Load Balancer Related Metrics 227

A.58 VNM Network Load Balancer Mappings 227

A.59 VNM Application Load Balancer Related Metrics 228

A.60 VNM Application Load Balancer Mappings 228

Chapter 1

Introduction

The beginning is the most important part of the work.
— Plato, The Republic

1.1 Motivation

We have seen how in just a few years society has transformed and evolved towards an increasingly digitalized world, where all aspects of its daily life depend on technology and more specifically on Internet services. For all these reasons, it is not surprising that computer resources are now an essential utility in modern societies on a par with electricity, gas or water. As a result, cloud computing emerged as a way to provide computing resources as a service, that is, on-demand computing resources that users acquire on a pay-as-you-go basis.

Cloud computing has been consolidated as a support for the vast majority of current and emerging technologies, such as IoT and big data. For example, the widespread adoption of event-driven architectures [54], which are essential for real-time-sensitive digital business such as IoT (Internet of Things), has been possible because cloud computing is able to provide an infrastructure that meets the requirements demanded by high-performance distributed systems (publish/subscribe message brokers, distributed stream processing systems or distributed datastores), which are the cornerstones of these architectures.

A key feature of Cloud Computing is elasticity, which is the capability to acquire and release resources on demand to meet end-user requirements, which are formally expressed through Service Level Agreements or SLAs. However, it is not a trivial task to decide the exact amount of resources needed at any given time nor to decide when to scale to guarantee that those resources are available on time to meet these SLAs. There are several types of SLAs depending on the magnitude that end users want to manage such as performance, cost or energy consumption. Therefore, an auto-scaling system[1] is desirable to free the users from the burden of adjusting allocated resources to meet SLAs at any given time. The main objective of auto-scaling systems is to avoid both over-provisioning and under-provisioning of resources, which would increase the cost and violate the SLA respectively.

Many auto-scaling systems have been developed in both, the literature and the industry, proposing different approaches to the problem. These auto-scaling techniques are classified into two major groups: (i) reactive techniques, where the scaling action is in reaction to a change in the system, and therefore does not anticipate such a change; and (ii) predictive or proactive techniques, which attempt to anticipate future changes in the system by performing the necessary scaling actions before such changes occur [31, 42].

A scaling action is defined by the specific values of its dimensions, i.e. which resource is to be scaled (CPU, memory, network, etc), when to scale, how many resources are to be added or removed, and how to scale (horizontal or vertical scaling). An auto-scaling system can be seen as a system that returns a specific scaling action (with specific values to each of the dimensions) based on a series of parameters or input information provided to it (e.g. SLA, workload, application information to be scaled, predictive models, threshold-based scaling rules, etc) to ensure compliance with a SLA. Because of this, auto-scaling systems are quite complex and existing approaches usually focus only on one type of SLA (e.g. performance, cost or energy consumption), one or two dimensions of the scaling actions (e.g. when to scale and how much) and a specific application or type of application (e.g. distributed stream processing systems).

[1]Throughout this work the terms auto-scaler and auto-scaling system are used interchangeably.

The main cloud providers such as Amazon Web Services[2], Google Cloud[3] or Microsoft Azure[4], offer mainly[5] reactive auto-scaling systems with threshold-based rules on the use of certain computational resources (e.g. CPU, memory, network or disk), as shown in Example 1.1. Although these auto-scalers are easy to understand due to their intuitive nature, they can not anticipate changes in the needs of cloud applications. In addition, these systems put all of the burden on the cloud consumer to configure them, which requires a deep knowledge of (i) which metrics that can be monitored (normally cloud providers only export low-level or resource metrics) are the key performance indicators of the high-level parameters that the cloud consumer is interested in for its application (performance, cost, availability, etc.); (ii) when, how much and what type of resources must be added/deleted to meet the demand of the application at each moment. Moreover, this manual configuration made by the cloud consumer for an application cannot be replicated as it is for another application, since it is typically a per-application task.

$$\begin{aligned}
&\textbf{IF } CPU\ usage > 80\% \textbf{ FOR } 5\ minutes \\
&\textbf{THEN } add\ 1\ Virtual\ Machine \\
&\\
&\textbf{IF } CPU\ usage < 10\% \textbf{ FOR } 5\ minutes \\
&\textbf{THEN } remove\ 1\ Virtual\ Machine
\end{aligned} \tag{1.1}$$

On the other hand, there are many predictive (also called proactive) auto-scalers that have been proposed in the literature, which try to predict the future behavior of the system or the workload, in order to make decisions about scaling in advance. Different techniques have been used for this, including machine learning techniques or reinforcement learning. However,

[2] `https://aws.amazon.com/`

[3] `https://cloud.google.com/`

[4] `https://azure.microsoft.com/`

[5] During the development of this thesis, some of these providers have announced predictive auto-scaling services (`https://aws.amazon.com/es/blogs/aws/new-predictive-scaling-for-ec2-powered-by-machine-learning/` and `https://cloud.google.com/sdk/gcloud/reference/alpha/compute/instance-groups/managed/set-autoscaling`), but the auto-scaling solutions they offer today are eminently reactive.

although these solutions are capable of solving some of the main problems of reactive auto-scaling systems, there are several aspects that have not been addressed in these systems, such as the automatic determination of the resource to be scaled.

These limitations of current auto-scaling systems are a barrier that prevent the exploitation of the full potential of cloud computing technology, especially to allow the deployment of truly elastic applications that leverage the key feature of cloud computing, which is on-demand resources.

1.2 Statement of the problem

The aim of this section is to set out in a clear and concise manner the specific problems addressed in this thesis from among all those that have been glimpsed in the previous section.

As mentioned above, cloud computing SLAs are the contract between the cloud consumer and the cloud provider serving as the foundation for the expected level of service. The core of this contract is the specification of service level objectives (SLOs). SLOs are expressed by means of high-level user guarantee parameters such as throughput, availability, cost, response time or latency. SLA compliance is extremely important to fulfill the expectations of all participants in order to assure business continuity, customer satisfaction and trust. It is, therefore, a critical long-term success factor [35].

However, the main cloud providers most widely used these days do not offer cloud consumers the possibility of defining their SLAs or agreeing on the service level that they expect from the cloud provider. These cloud providers merely offer a set of, usually very low-level, monitoring metrics for the services that they provide together with resource provisioning and management mechanisms. The metrics monitored and offered by cloud providers are too close to machine level and do not have the primitives for defining service-relevant metrics [47]. According to [19], there is a gap between monitored metrics, which are usually low-level metrics, and SLA agreements, which are high-level. In addition, mapping SLA parameters to low-level metrics exported by cloud providers is a tricky problem, which requires the knowledge of experts who understand the relationship between these two metric levels [17]. Therefore, by assigning this responsibility to cloud consumers that

do not have the required expertise, and neither know nor care about how their applications behave at such a high-level (since they are interested in high-level SLA metrics, like cost, latency, throughput or availability), causes serious problems for many organizations when they try to formulate their SLA strategies and adjust the metrics to support such strategies [53, 35].

On the other hand, there are many predictive auto-scaling systems proposed in the literature that usually focus also on predicting only low-level metrics performance (mainly CPU usage). In the best case, some works try to predict the workload in future moments, but this is only a first step, since it is necessary to model how that workload prediction will affect the behavior of the SLA, which is what the cloud consumer is interested in.

However, in all cases the onus of implementing the mechanisms that ensure compliance with the desired service levels is put on cloud consumers [36]. Therefore, this is usually considered as a misguided and error-prone approach.

> **Problem 1.** *The current auto-scaling systems do not focus directly on compliance with service level agreements (SLAs), but rather on (i) establishing conditions on resource utilization (reactive approach) and/or (ii) predicting higher or lower level metrics such as workload or resource utilization metrics (predictive approach). Both approaches are insufficient, as a further step is needed to map these rules or predictions with the performance of SLA parameters.*

In order to achieve the desired elasticity of an application, several works and authors highlight the need to understand the relationship between the low-level behavior of that application and the high-level parameters of the SLA to be ensured [47, 19, 17, 35, 53, 27]. Therefore, auto-scale systems would have to be equipped with the necessary mechanisms that allow them to establish a relationship between the low-level behavior (i.e. at resource level expressed through resource metrics such as CPU usage, memory usage, etc) and the high-level behavior (i.e. SLA parameters of performance, cost, etc) of the application in order to take the appropriate scaling actions to ensure compliance with the corresponding SLA. However, few works address this problem, as they take for granted the resource that is the bottleneck and therefore the resource to be scaled. For example, many jobs take for

granted that the limiting resource or KPI (Key Performance Indicator) is the processing capacity and therefore their scaling action consists in increasing the processing capacity by adding more processors directly (scale-up) or more virtual machines (scale-out). It is not questioned that in these works the resource that they scale is the adequate one, since they usually demonstrate it empirically, but the study of this relation between low-level metrics and high-level metrics allows to characterize the system in a more precise way. In fact, there are several research works that point in this direction to improve their approaches in their future work [47, 16]. For example, although the resource to be scaled is processing capacity, the percentage of CPU usage may not be the most informative metric, and context changes, interruptions, or the percentage of time processors spend in user or kernel space may be more useful.

The inclusion of the information of this relationship between low and high-level metrics in an auto-scaling system considerably extends the range of applications to which such auto-scaling system can be applied, since it allows the detection of the resource that is the bottleneck and therefore the resource to be scaled regardless of the type of application and in a totally transparent way for the end user.

> **Problem 2.** *There is no auto-scaling system capable of automatically and application-agnostically establishing relationships (i.e. mappings) between high-level metrics (or SLA parameters) and low-level metrics (resource utilization metrics) in order to determine which resource is acting as a bottleneck, and therefore, to know which are the KPIs of that system and the resource to be scaled.*

In order to ensure compliance with some of the most demanded and important SLA service levels (e.g. cost optimization, legal restrictions or high availability), as they very often constitute the main reason for opting for cloud environments, it is necessary to use resources and services from multiple cloud providers, as a single cloud provider cannot offer these service levels. There are several reasons for opting for multi-cloud environments [37]: using external resources to deal with peaks in service or resource requests, optimize costs, adhere to constraints (e.g. new locations, laws or data sovereignty), achieve high availability by replicating the application or ensure back-ups to deal with scheduled inactivity or disasters. Therefore,

it is not surprising that there is an increasing tendency[6] in recent years to use multiple clouds. However, the use of multiple clouds introduces several challenges due to the number of different cloud providers and their intrinsic variability. This translates into numerous technological and administrative barriers that are an obstacle to the use of multiple clouds, favoring vendor lock-in. This is a consequence of the lack of widely adopted standards, motivated by the reticence of the main cloud providers who regard such standards as posing a threat to innovation and market advantage rather than as a business need or priority [40, 37]. All of this introduces an additional problem:

> **Problem 3.** *There is no vendor-neutral set of low-level metrics that can be monitored across cloud providers [36]. Since each cloud provider offers a different set of these low-level metrics, the strategies defined by cloud consumers to ensure SLA compliance are bound to a particular cloud provider, fostering provider lock-in and making it impossible to benefit from the service levels inherent to multi-cloud environments.*

Solving these problems is essential for designing efficient and effective auto-scalers systems to provide real elasticity to applications deployed in the cloud.

1.3 Definitions of terms

In recent years there has been an explosion of different cloud technologies, solutions and/or services that has led to different potential angles from which to address cloud computing concepts. This has blurred the boundaries between the different concepts. For the sake of clarity, what follows is the cloud computing terminology considered in this Thesis.

Cloud Computing

Although several definitions of cloud computing have been proposed in the literature published by both academia and industry, most authors agree with the common elements present in the definition provided

[6] `https://www.parkmycloud.com/wp-content/uploads/2017/06/EMA-Top10-2017-HybridCloud-DecisionGuide.pdf`

by the U.S. NIST (National Institute of Standards and Technology) [33]: *cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.* This cloud model is composed of five essential characteristics (i.e. on-demand self-service, broad network access, resource pooling, rapid elasticity and measured service). The NIST also define three service models:

1. **Software as a Service (SaaS):** in which users do not have control over the cloud infrastructure and therefore release their applications on a hosting environment (e.g. Gmail[7] or Google Docs[8])
2. **Platform as a Service (PaaS):** in which a platform supporting the full software lifecycle is provided for cloud consumers to develop their cloud services and applications (e.g. AWS Elastic Beanstalk[9])
3. **Infrastructure as a Service (IaaS):** where cloud consumers directly use IT infrastructures such as processing, storage or networks (e.g. Amazon EC2[10])

Cloud consumer

An organization or person that maintains a business relationship with, and uses the service from, a cloud provider [33]. It is the user of the services offered by the cloud provider (i.e. IaaS, PaaS or SaaS) [31].

Cloud provider

A person, an organization, or an entity responsible for making a service available to cloud consumers [33]. It refers mainly to the IaaS, PaaS and/or SaaS provider [31].

Multi-cloud

The usage of multiple and independent clouds by a client or a service. Multi-cloud and federated cloud are the two types of delivery models in multiple clouds. In the case of federated clouds, there is an agreement on resource sharing between the clouds involved, whereas

[7] `https://www.gmail.com`
[8] `https://www.google.es/intl/en/docs/about/`
[9] `https://aws.amazon.com/elasticbeanstalk/?nc1=h_ls`
[10] `https://aws.amazon.com/ec2/`

no such agreement exists in the case of multi-cloud. In this work, the multi-cloud model is adopted because it is hard to reach agreements between large cloud providers, especially since it is necessary to control the resources and services that an external provider expects to use for its benefit [37]. Additionally, multi-cloud differs from hybrid cloud in that it refers to multiple cloud services rather than multiple deployment modes (public, private or legacy).

Cloud broker

A cloud broker is, according to the NIST Cloud Computing Reference Architecture [33], an entity that manages the use, performance, and delivery of cloud services, and negotiates relationships between cloud providers and cloud consumers. Cloud brokers provide a single point of entry for managing multiple cloud services and a single consistent interface to multiple differing providers.

Service Level Agreement (SLA)

A formal contract between a cloud provider and a cloud consumer in the context of a particular service provision. They serve as the foundation for the expected level of service, as well as for reducing the area of potential conflicts and their resolution before these issues get materialized. More specifically, an SLA is composed of a set of high-level user guarantee parameters called SLA parameters. These SLA parameters are established by objectively measurable conditions called Service Level Objectives (SLOs), such as throughput, response time, latency or cost. Therefore, SLOs are a critical long-term success factor since they are extremely important for meeting the expectations of all participants in order to assure business continuity, customer satisfaction and trust[24, 32, 35].

In addition to the above definitions, other terms currently widely employed will be used recurrently in this work. It is precisely this extended use of them that may have partially distorted their meaning and therefore, it is necessary to give a concrete and consensual definition of them, focusing on the aspects most related to this work.

Internet of Things (IoT)

According to [25], Internet of Things for smart environments can be defined as *the interconnection of sensing and actuating devices providing the ability to share information across platforms through a unified framework, developing a common operating picture for enabling innovative applications. This is achieved by seamless ubiquitous sensing, data analytics and information representation with cloud computing as the unifying framework* (CloudIoT).

From a high-level perspective, the components of this new paradigm are:

- **Radio Frequency Identification (RFID):** permits the automatic identification of anything with a microchip for wireless data communication attached to it. The RFID tags are not battery powered and they use the power of the reader's interrogation signal to communicate the ID of the RFID reader.
- **Wireless Sensor Networks (WSN):** are composed by a large number of intelligent sensors, enabling the collection, processing, analysis and dissemination of valuable information, gathered in a variety of environments.
- **Addressing schemes:** are the methods used to uniquely identified "things" and to control remote devices through Internet.
- **Data storage and analytics**. New technologies are emerging to meet the challenge of store and process the huge amounts of data generated by the IoT paradigm.
- **Visualization**. The extraction of meaningful information from data is no trivial, and therefore visualization is critical for an IoT application to allow the users interact with the environment.

Publish/Subscribe (Pub/Sub) message pattern

In the publish/subscribe message pattern, producers publish information on a bus (an event manager) and consumers subscribe to the information they want to receive from that bus. The act of delivering that information is typically denoted by the term notification [21]. The main benefit of this message pattern is the decoupling between publishers and subscribers. This decoupling can be decomposed into dimensions:

- Space decoupling: the users do not need to know each other.

- Time decoupling: the users do not need to be actively participating at the same time.
- Synchronization decoupling: the publishers are not blocked when sending an message and consumers can get asynchronous notified of the occurrence of an event.

There are two main variations of the publish subscribe pattern used in the IoT environments:

- **Topic-based publish/subscribe**. Publishers publish messages in specific topics which are identified by a keyword and subscribers subscribe to the topics which they are interested. When a publisher sends a message to a specific topic, all the users subscribe to this topic receive the notification. This variation is usually extended by the use of hierarchical topics and wildcards in order to improve the flexibility and expressiveness of subscriptions. An example of this variant is RabbitMQ [11].
- **Content-based publish/subscribe**. In this variant, subscriptions are not classified according to any external and fixed criterion (topics), but according to the properties (content) of the messages. An example of this variant is E-SilboPS [53].

Scalability

As described in [26], scalability is a prerequisite for elasticity, but it does not consider temporal aspects of how fast, how often and at what granularity scaling actions can be performed. More specifically, "scalability is the ability of the system to sustain increasing workloads by making use of additional resources", and therefore, in contrast to elasticity, it is not directly related to how well the actual resource demands are matched by the provisioned resources at any point in time.

Elasticity

As defined in [26], "*elasticity is the degree to which a system is able to adapt to workload changes by provisioning and de-provisioning resources in an autonomic manner, such that at each point in time the available resources match the current demand as closely as possible*". In [4], the authors deduce the following equation that summarizes the

[11] `https://www.rabbitmq.com/`

concept of elasticity in cloud computing: "it means that the elasticity is built on top of scalability. It can be considered as an automation of the concept of scalability, however, it aims to optimize at best and as quickly as possible the resources at a given time".

$$Elasticity = \underbrace{scalability \ + \ automation}_{auto-scalling} + optimization$$

Middleware

According to [6], "middleware is a class of software technologies designed to help manage the complexity and heterogeneity inherent in distributed systems. It is defined as a layer of software above the operating system but below the application program that provides a common programming abstraction across a distributed system".

Throughput and Response time

In general, throughput is defined as the ratio of work or the amount of output that can be performed by a system in a given period of time. The term is used in a wide variety of contexts. In this work, it refers mainly to its meaning in the field of computer networks, and therefore, it is the ratio of the amount of messages that the system is capable of sending per unit of time (e.g. 100 messages/s). In some sections the term is used to talk about the throughput of some component of the system that does not send any message, and in that case the throughput refers to the ratio of the amount of messages that component is capable of processing per unit of time.

The term response time can be defined as the time that elapses between an inquiry to a system and obtaining a response to that inquiry. As with the term throughput, the definition of response time varies with respect to its application context. In this case, the response time is measured as the difference in time between a message being sent to the system and the response being received to that initial message as a result of being processed by the system. It is important to note that in this case, this time includes the transmission time since messages travel through the network and can be very significant (e.g. propagation delays or transmission errors).

These terms are presented together as they are used frequently throughout this document, and so they are presented together, with some sections focusing on throughput and others on response time, but it is important to keep in mind that both terms are related [28].

1.4 Description of the remaining chapters

The thesis is structured as follows:

Chapter 2

reviews the current state of the art and the related work. More specifically, this section is divided into three subsections. First, Sections 2.1 and 2.2 reviews the main contributions of both academic studies and major commercial solutions that identify and highlight the importance of addressing the three research questions motivating this work and analyzes the different solutions existing in both fields. Finally, Section 2.3 presents the state of the art of content-based publish-subscribe systems, as well as the main problems to be addressed in these systems, in order to focus on E-SilboPS, a content-based context-aware publish/subscribe middleware used in all this work as a representative distributed system typically deployed in cloud environments.

Chapter 3

introduces the specific objectives of this work in Section 3.1. Also, Section 3.2 presents the hypothesis of this work, as well as the specific research questions that guide this work and whose answer allows to validate the hypothesis. Finally, Sections 3.3 and 3.4 explain the contributions of this work and its limitations.

Chapter 4

introduces FLAS (Forecasted Load Auto-Scaling), an auto-scaler for distributed services that combines the advantages of proactive and reactive approaches according to the situation to decide the optimal scaling actions in every moment. More specifically, Section 4.1 explains the system configuration model, presents the workload and performance modelling, and finally states the specific problem based on the modelling presented. Section 4.2 presents the architecture of FLAS and each of its components, and Section 4.3 explains the integration of this architecture and its components for a specific case (i.e. E-SilboPS). To conclude this chapter, Sections 4.4 and 4.5 present, analyze and discuss the results of the FLAS evaluation.

Chapter 5

presents a solution to the problem of automatically translating SLAs into objectives expressed as metrics that can be measured across multiple cloud providers, abstracting the specificities of each cloud provider. More specifically, Section 5.1 provides a brief overview of the solution explained in more detail in the rest of the sections of the chapter. Section 5.2 describes a knowledge-based system capable of automatically determining the low-level metric conditions (i.e. key performance indicators or KPI) necessary to achieve a high-level metric condition in order to ensure compliance with the high-level metric objective (SLO). On the other hand, Section 5.3 presents a harmonized set of monitoring metrics, which aims to abstract the specificities between multiple cloud provider offerings, making the monitoring and enforcing of SLOs in a multi-cloud environment transparent for cloud consumers. Finally, Section 5.4 and 5.5 describe the evaluation of this system, and discuss the results of this evaluation.

Chapter 6

summarises and analyses the conclusions reached in this work.

Chapter 7

lists the open questions for future research on the topic.

1.5 Derived publications

Some key results of this Thesis have already been published in conferences and journals relevant to the subject. This has allowed both to be discussed and contrasted them with the global community of researchers in the field.

Conferences

- Víctor Rampérez, Javier Soriano, David Lizcano, and Juan A. Lara. 2018. An Innovative Approach to Improve Elasticity and Performance of Message Brokers for Green Smart Cities. In Proceedings of the Fourth International Conference on Engineering & MIS 2018 (ICEMIS '18). Association for Computing Machinery, New York, NY, USA, Article 34, 1–5. DOI: `https://doi.org/10.1145/3234698.3234732`

Journals

- Víctor Rampérez, Javier Soriano, and David Lizcano, "A Multidomain Standards-Based Fog Computing Architecture for Smart Cities", Wireless Communications and Mobile Computing, vol. 2018, Article ID 4019858, 14 pages, 2018. DOI: `https://doi.org/10.1155/2018/4019858`.
- Víctor Rampérez, Javier Soriano, David Lizcano and Juan A. Lara, "FLAS: A combination of proactive and reactive autoscaling architecture for distributed services", Future Generation Computer Systems, 18 pages, 2020. DOI: `https://doi.org/10.1016/j.future.2020.12.025`.

Chapter 2

State of the Art and Related Work

Nothing new that is really interesting comes without collaboration.
— James D. Watson

This chapter reviews the current state of the art and the related work. More specifically, this section is divided into three subsections. First, sections 2.1 and 2.2 reviews the main contributions of both academic studies and major commercial solutions that identify and highlight the importance of addressing the three research questions motivating this work and analyzes the different solutions existing in both fields. Finally, section 2.3 presents the state of the art of content-based publish-subscribe systems, as well as the main problems to be addressed in these systems, in order to focus on E-SilboPS, a content-based context-aware publish/subscribe middleware used in all this work as a representative distributed system typically deployed in cloud environments.

2.1 Elasticity in cloud computing

This section discuss the two major categories of auto-scaling techniques in which auto-scalers are classically classified, reactive and/or proactive. It is important to note that although not all auto-scaling techniques can be classified as purely reactive or proactive (e.g. control theory, reinforcement

learning and queuing theory)[31], it has been decided to include this classification for the sake of clarity, since the solution proposed in this work combines purely reactive and proactive techniques. In addition, within each of the categories, the techniques used in this work are explained in more detail. Finally, in the last part of this section the similarities and deficiencies of those auto-scaling systems more related to the one proposed in this work are analyzed.

2.1.1 Reactive auto-scaling

The reactive auto-scaling techniques, widely studied in the past, are those in which the scaling action is decided in a reactive way to a change in the workload, that is, the change in the workload has to have taken place in order to be able to react to it by means of a scaling action. Deciding on the scaling action to be performed once the change in workload has occurred, combined with the fact that scaling actions require time, often results in the scaling action being delayed from the time when the change in workload occurred. This delay usually leads to economic costs and/or violations of the SLA more or less considerable depending on the delay of the corresponding scaling action.

2.1.1.1 Threshold-based rules

The most popular reactive auto-scaling technique are threshold-based rules because it is the solution adopted by the vast majority of cloud providers today due to its simplicity and intuitive nature. However, the correct configuration of its thresholds is a task that requires a deep understanding of workload trends and their relationship to the desired SLA for each application. (i.e. per-application task)[31].

This technique usually consists of one or more rules for scale-up/out and one or more rules for scale-down/in as shown in 2.1 extracted from [31].

$$
\begin{aligned}
&\textbf{IF } x_1 > thrU_1 \textbf{ AND/OR } x_2 > thrU_2 \textbf{ AND/OR} \ldots \\
&\qquad \textbf{FOR } durU \textit{ seconds } \textbf{THEN} \\
&\qquad\qquad n = n + s \textbf{ AND} \\
&\qquad\qquad \textit{do nothing } \textbf{FOR } inU \textit{ seconds} \\
&\textbf{IF } x_1 < thrL_1 \textbf{ AND/OR } x_2 < thrL_2 \textbf{ AND/OR} \ldots \\
&\qquad \textbf{FOR } durL \textit{ seconds } \textbf{THEN} \\
&\qquad\qquad n = n - s \textbf{ AND} \\
&\qquad\qquad \textit{do nothing } \textbf{FOR } inL \textit{ seconds}
\end{aligned}
\tag{2.1}
$$

Each rule has two parts: the condition and the action to be executed when the condition is met. The condition part uses one or more performance metrics x_1, x_2, ..., such as the input request rate, CPU load or average response time. Each performance metric has upper $thrU$ and lower $thrL$ thresholds. If the condition is met for a given time ($durU$ or $durL$), then the corresponding action will be triggered. For horizontal scaling, the application manager should define a fixed amount s of VMs to be acquired or released, while for vertical scaling s refers to an increase or decrease of resources such as CPU or RAM. After executing an action, the auto-scaler inhibits itself for a small cooldown period defined by inU or inL.

An example of a scale-up/out rule would be: *add two medium virtual machines (VMs) when CPU usage is above 80% for 60 seconds and do nothing for 5 minutes.*

The most basic version of this technique has been presented here for the sake of clarity. However, in the literature some modifications to this technique can be found to improve or mitigate some of its limitations such as: *smart kill* of VM [13], dynamic thresholds [30], and many others analyzed in the work of Lorido-Botran et al. [31].

2.1.2 Proactive auto-scaling

On the other hand, proactive auto-scaling, also known as predictive auto-scaling, are based on predicting the future demands on the system to carry out the corresponding scaling actions before those changes occur. Therefore, this approach allows to scale in advance and deal with the ever fluctuating resource usage pattern. Within this category are all techniques that use past data to predict future metrics or workload performance. These past data are usually time series, and therefore, within this category are mainly the techniques of time series analysis.

2.1.2.1 Time series analysis and forecasting

Time series analysis includes methods for analyzing time series data and extracting relevant statistics and other characterizations from the data, which will be used to build predictive models capable of predicting future values based on previously observed values (i.e. time series forecasting).

A time series results from observing the same phenomenon during a series of regular time intervals. More formally, a time series $x_1, x_2, \ldots, x_T$ is the finite (univariate) realization of a stochastic process X_t. This stochastic process $\{X_T : t = 1, 2, \ldots, T\}$ is a sequence of random variables. According to [34], "Time series analysis accounts for the fact that data points taken over time may have an internal structure (such as autocorrelation, trend or seasonal variation) that should be accounted for".

Some of the time series forecasting models are: Naïve, SNaïve, Seasonal decomposition (+ any model), Exponential smoothing, STL decomposition, ARIMA o SARIMA. These models are beyond the scope of this work, and therefore only ARIMA models will be briefly presented below, since they are the ones mainly used in this work. The definitions presented below are based on those presented in [34].

On the one hand, autoregressive model (AR) of order p, is essentially a linear regression of the current value of the time series against one or more past values of the time series:

$$X_t = \delta + \phi_1 X_{t-1} + \phi_2 X_{t-2} + \cdots + \phi_p X_{t-1} + A_t$$
$$\delta = (1 - \sum_{i=1}^{p} \phi_i)\mu \tag{2.2}$$

where p is the order of the model, A_t is white noise, X_t is the time series and μ is the process mean.

On the other hand, a moving average model (MA) of order q is also a simply linear regression of the current value of the time series against the white noise or random shocks (from a normal distribution) of one or more prior values of the series. The difference in this model is that the random shocks are propagated to future values of the time series.

$$X_t = \mu + A_t - \theta_1 A_{t-1} - \theta_2 A_{t-2} - \cdots - \theta_q A_{t-q} \tag{2.3}$$

where q is the order of the model, A_{t-i} are the white noise terms, $\theta_1, \ldots, \theta_q$ are the parameters of the model, X_t is the time series and μ is the mean of the series.

Finally, $ARMA(p, q)$ models are a combination of the previous AR and MA models:

$$\begin{aligned} X_t =& \delta + \phi_1 X_{t-1} + \phi_2 X_{t-2} + \cdots + \phi_p X_{t-1} + \\ & A_t - \theta_1 A_{t-1} - \theta_2 A_{t-2} - \cdots - \theta_q A_{t-q} \end{aligned} \tag{2.4}$$

The autoregressive component is responsible for stationary, for correlation and forecasting exponential structure. The moving average component is responsible for the first autocorrelation and forecast.

Therefore, a non-seasonal $ARIMA(p, d, q)$ model is described by:

- p is the number of autoregressive terms.
- d is the number of non-seasonal differences.

- q is the number of moving-average terms.

ARMA models come from the combination of autoregressive (AR) and moving average (MA) models. The Box-Jenkins model assumes that the time series is stationary, and if not, they recommend differentiating one or more times the non-stationary series until it becomes stationary, which produces an ARIMA model, with the "I" standing for "Integrated". More specifically, SARIMA models or ARIMA models with additional seasonal parameters or factors that operate across multiples of lags (i.e. the number of observations in a season):

$$X_t \approx ARIMA(p, d, q)(P, D, Q)_s \tag{2.5}$$

- d is the number of regular differences to obtain stationary.
- p and q are the order of the regular ARMA model.
- D are the number of seasonal differences to obtain stationary.
- P and Q are the order of seasonal ARMA model.

2.1.3 About existing auto-scalers

The vast majority of auto-scalers tend to focus on the temporal dimension (when to scale) and the quantitative dimension (how much to scale), making it obvious which resource to scale. Usually this dimension of scaling is not analyzed because it is considered trivial as a result of previous knowledge of the application to be scaled. Moreover, usually this resource is the processing capacity, taking as Key Performance Indicator (KPI) the percentage of CPU usage or some similar metric in this sense [29, 16, 39, 57, 59, 58, 49]. Nevertheless, there are many works that in their future lines mark the need to study other different scaling metrics, including some of the authors that highlight the limitation of the previous approach [29, 16]. For example, in [29] Lombardi, F. et al. consider the CPU as the KPI since it is the most prominent bottleneck for the type of application they scale, however they also note the intention to include memory and bandwidth in a more complete model as future work. In the same vein, the authors of [16] recognize

that many resources can potentially be the bottleneck, but also focus on the CPU resource alone, justifying it as frequently being the key resource in determining performance. In [14], the authors consider a module to detect the operator that is the bottleneck in order to perform the corresponding scale-out of that operator. They monitor the user and system CPU times of the different VMs in order to determine the "stolen" CPU time when other VMs on the same physical node were scheduled. When more than a certain number of consecutive times an operator is reported to be above a user-defined threshold, that operator is considered a bottleneck.

Many studies have identified the need to establish some kind of relationship or mapping between the high-level metrics, in which cloud consumers are interested, and the low-level metrics offered by cloud providers, in order to establish mechanisms to ensure that the service levels demanded by cloud consumers are met.

According to [19], there is a gap between monitored metrics (low-level entities) and SLAs (high-level user guarantee parameters) and none of the approaches discussed in their work deal with the mappings of low-level monitored metrics to high-level SLA guarantees necessary in cloud-like environments.

In the same vein, Paschke et al. [35] highlight the problem of the poor translation of SLAs into low-level metrics, claiming that the metrics used to measure and manage performance compliance to SLA commitments are the heart of a successful agreement and that inexperience in the use and automation of performance metrics causes problems for many organizations as they attempt to formulate their SLA strategies and set the metrics needed to support those strategies.

Debusmann et al. [17] again also clearly identify the need to address this problem, stating that a key prerequisite for meeting these goals is to understand the relationship between high-level SLA parameters (e.g., availability, throughput, response time) and low-level resource metrics, such as counters and gauges. However, it is not easy to map SLA parameters to metrics that are retrieved from managed resources.

In [16] the authors claim that specifically domain experts are usually involved in translating these SLOs into lower-level policies that can then be used for design and monitoring purposes, as this often necessitates the application of domain knowledge to this problem.

In [12], the authors go further and establish correlation models between absolute resource utilization metrics (i.e. "measures report about the cumulative activity counters in the operating system") and relative resource utilization metrics (i.e. "those performance measures which values are based on the data collected from the `/cgroup` virtual file"), demonstrating that the use of relative resource utilization metrics underestimates the capacity required and therefore are not appropriate for determining the amount of resources needed to meet performance SLA (e.g. Response Time).

There are many works that make this mapping between high and low-level metrics using black-box prediction techniques (e.g. Artificial Neural Networks or ANN), which even performing very accurate predictions, do not allow to really understand the relationships between these levels of metrics [29, 27]. It is essential to be able to understand these relationships in order to characterize and classify the applications, and that is why in this work it has been opted for a statistical method based on regression that allows to clearly interpret the established mappings with adequate predictive accuracy.

In addition, Scryer[1, 2] is an auto-scaler developed by Netflix that allows them to provision the right number of AWS instances needed to handle their customers' traffic. According to the authors, they have chosen this solution because it allows them to perform predictive auto-scaling, instead of the reactive approach offered by the AWS auto-scaling service. However, Scryer does use AWS's reactive auto-scaling service as a means of contingency when predictive scaling predictions fail or simply phenomena occur that cannot be predicted.

F.Lombardi et al. present in [29] a work that is closely related to the solution presented in this work (i.e. FLAS). They introduce PASCAL, which is a predictive auto-scaling system for distributed systems by predicting workload patterns, estimating the minimum configuration required by the application and making decisions on the corresponding scaling action based on this information at each moment.

More specifically, PASCAL predicts the workload input rate and estimates the application's performance at each moment in order to estimate the minimum required configuration and take the scaling decisions that will allow reaching that minimum configuration.

Both FLAS and PASCAL work in two phases, a monitoring and learning phase for the generation of the predictive models and an auto-scaling phase in which the decisions about the scaling actions to be performed are made. As for the predictive part, its objective is to predict the workload input rate, while in the approach of this work the objective is to predict the trend of the relevant SLA parameters (e.g. throughput or response time), which allows to predict how quickly a state of SLA violation can be reached. In addition, their performance estimation model currently only takes into account CPU usage, which very limited, as stated above, compared to FLAS which establishes mappings between high-level metrics and a large set of low-level metrics.

Furthermore, PASCAL uses models based on Artificial Neuron Networks for its predictions, which although it provides them with very good prediction results, it does not allow the understanding of the relationships between these levels of metrics. The auto-scaler proposed in this work, through statistical methods based on regression, allows to understand these relationships, and therefore detect the resource that is the bottleneck, as well as the metric(s) that monitor it (KPI) and thus the resource to be scaled.

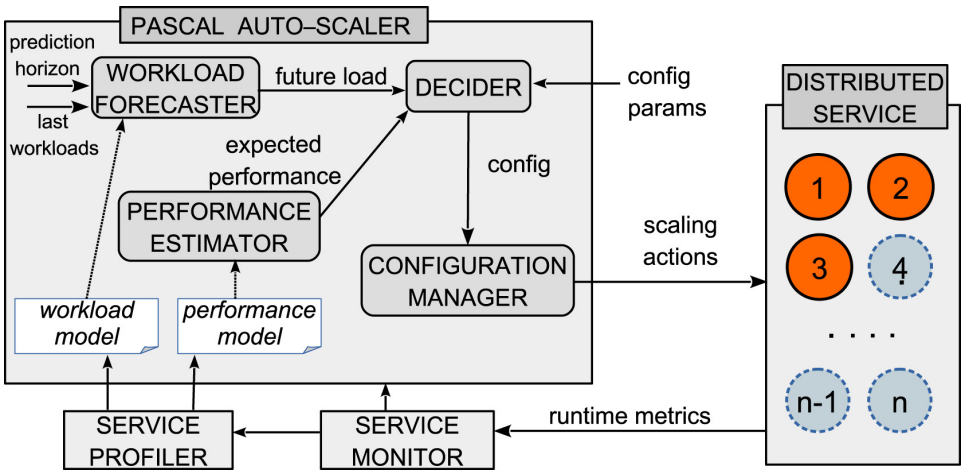


Figure 2.1: Extracted from [29]. PASCAL'S architecture.

2.2 Multi-cloud metrics

Many authors have identified the need to establish a set of vendor-neutral metrics (i.e. they can be measured in multiple cloud providers) in order to offer a unified platform-independent set of metrics to monitor the cloud services, reducing the semantic interoperability between heterogeneous cloud providers.

For example, Patel et al. [36] argue that there is no universal set of metrics that can be monitored across cloud providers. They recognize that there are attempts to standardize the clouds and underscore the importance of such efforts in the light of monitoring capabilities. They also suggest a set of basic metrics and best practices for measurements be established.

Quinton et al. [40] recommend using several clouds to deploy a multi-cloud configuration to deal with this problem, although they introduce several challenges due to the number of providers and their intrinsic variability.

In the same vein, Ferrel et al. present a survey of cloud solutions and address the problem of heterogeneous clouds, claiming that the ability to run and manage multi-cloud systems can exploit the peculiarities of each cloud solution [23]. However, these cloud solutions are typically heterogeneous and the features that they provide are often incompatible. This diversity is an obstacle the proper exploitation of the full potential of cloud computing.

2.2.1 Commercial solutions

At the time of writing, there were many commercial multi-cloud solutions available, including Flexera (RightScale)[1], Scalr[2] and Morpheus[3].

[1] https://www.flexera.com/products/agility/cloud-management-platform.html
[2] https://www.scalr.com
[3] https://www.morpheusdata.com

They all make it possible to deploy and manage applications in multiple clouds, acting as multi-cloud brokers. Cloud consumers can choose between a catalog of different types of instances (i.e. VM) defined in templates with different computing, RAM or storage capacities. Although cloud consumers can benefit from some of the features of a multi-cloud solution (high availability, cost optimization, geo-localization awareness, etc.), none of these commercial solutions address the issues that raised in this work.

First of all, these solutions do not require a vendor-agnostic set of metrics, since they make use of solutions such as `collectd`[4] or Nagios[5], which offer a harmonized set of metrics. However, this approach limits the use of multi-cloud-to-IaaS services, since the installation of these monitoring frameworks requires access to the management of the instance or VM. Therefore, they are not able to exploit other high-value (e.g. PaaS-level) services from the main cloud providers.

In addition, all of these commercial solutions follow, according to the classification by [5], a Trigger-Action approach, in which the cloud consumer specifies a set of triggers and their associated action(s) in order to ensure compliance with their SLAs.

This approach of putting the onus of translating the SLAs into low-level conditions on cloud consumers has, as already mentioned, many drawbacks, as it requires expert knowledge about the application domain that they do not normally possess.

To the best of our knowledge, there is currently no commercial solution offering a SLA-based multi-cloud broker service.

2.2.2 Academic solutions

On the other hand, there are many academic projects and papers dealing with several of the above problems. The systematic review approach proposed in [5] is applied, using the group taxonomy that it specifies. In this manner, this broad survey is extended, as shown in Table 2.1.

[4] `https://collectd.org/`
[5] `https://www.nagios.org/`

SALOON [41] is a solution based on the combination of ontologies with feature models (FMs) to identify multi-cloud configurations whose functionalities and resources match the technical requirements of cloud applications (Figure 2.2). It highlights the use of ontologies to formally capture and model the knowledge of cloud platforms and deal with the wide variety of terms used by different cloud providers to refer to the same concept, as a first step to reduce the semantic interoperability of multi-cloud environments.

LoM2HiS is a novel framework for mapping low-level resource metrics to high-level SLA parameters [19]. This framework is embedded in the FoSII (Foundations of Self-Governing ICT Infrastructures) infrastructure for developing infrastructures for autonomic SLA management and enforcement [20]. To be precise, it detects SLA violation threats based on predefined threat thresholds and notifies the enactor component to avert the threats. Although it addresses the issue of mapping high-level metrics into low-level metrics, this solution is rather limited, as it is designed for only IaaS-level cloud services and not suitable for a multi-cloud environment, because monitoring requires the instrumentalization of instances (Figure 2.3).

Similarly, the mOSAIC project [38] offers an open source API and platform for the development and deployment of applications in multi-cloud environments. For this purpose, it is possible to define the service requirements of an application, at both the IaaS and PaaS levels, in terms of a cloud ontology, with the platform being responsible for finding the services that best match the requirements of the application. Nonetheless, this solution does not offer a multi-cloud service at the PaaS level. Instead, it requires the customization of IaaS resources to meet PaaS requirements and does not address the problem of the gap between high- and low-level metrics.

Cloud4SOA is a solution for multi-cloud application management, monitoring and migration at the PaaS level [56]. More specifically, it offers a multi-cloud broker that addresses the challenges of semantic interoperability at the PaaS level in public, private and hybrid deployment models. In order to monitor the heterogeneous PaaS resources offered by a multi-cloud environment and avoid vendor lock-in, Zeginis et al. [56] propose a set of harmonized metrics and adapters that bridge the business logic between the harmonized API and the PaaS offerings. However, the set of harmonized vendor-neutral metrics that are proposed are very limited. Therefore, it is not possible to define cloud consumer SLAs in terms of these metrics.

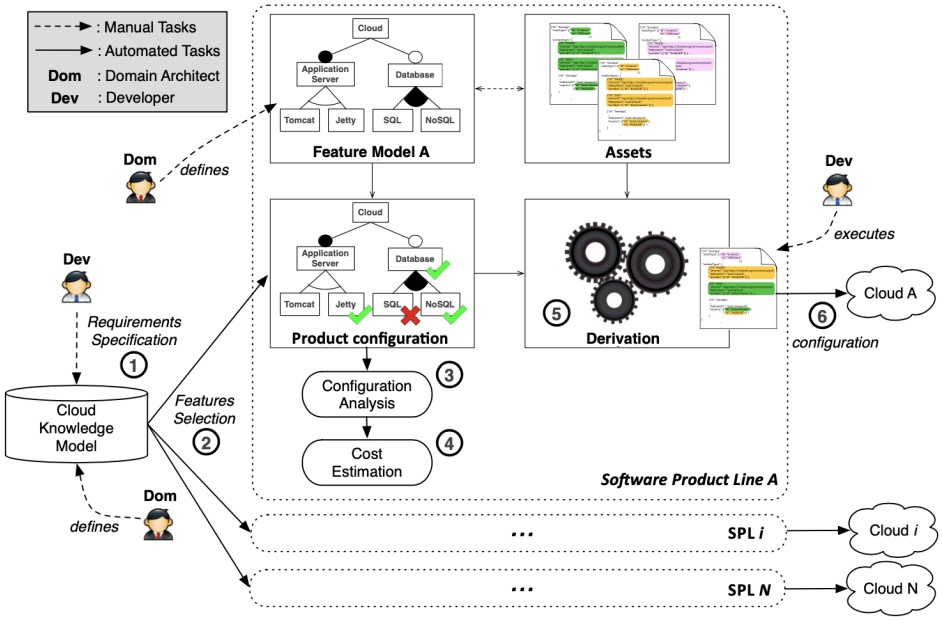


Figure 2.2: Extracted from [41]. SALOON platform overview.

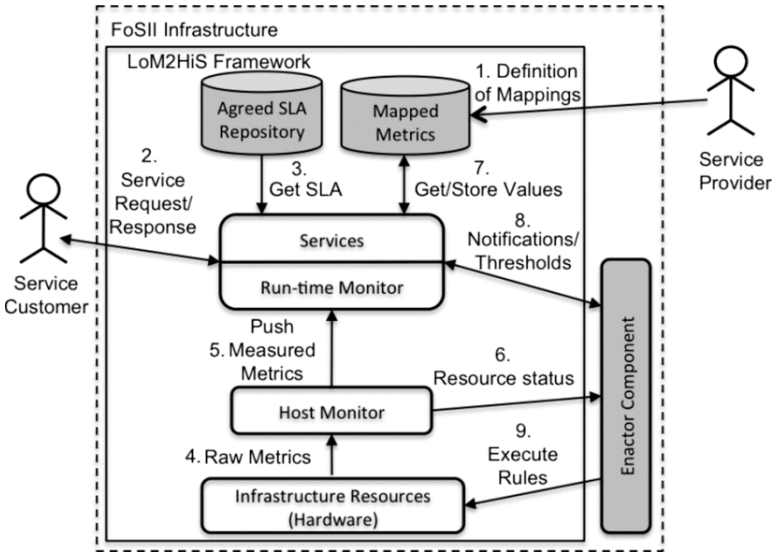


Figure 2.3: Extracted from [19]. LoM2HiS framework architecture.

	Organization Type	Architecture	Brokering Type	Application Type	Awareness	Cloud Services
SALOON	Ministry of Higher Education and Research, Nord–Pas de Calais Regional Council, the FEDER through the Contrat de Plan Etat Region Campus Intelligence Ambianteand the EU FP7 PaaSage project.	Independent Service	SLA based	Singular jobs, periodical jobs, compute-intensive and data-intensive interactive application	Pricing, location	PaaS IaaS
LoM2HiS	Vienna Science and Technology Fund under grant agreement Foundations of Self-governing ICT Infrastructures (FoSII)	Independent Service	SLA based	Singular jobs, periodical jobs, compute-intensive and data-intensive interactive application	-	IaaS
mOSAIC	Private and Public European research organizations funded by EU	Independent Service	SLA based	Singular jobs, periodical jobs, compute-intensive and data-intensive interactive application	Pricing	PaaS IaaS
Cloud4SOA	Partially funded by the European Commission in the context of the Cloud4SOA project	Independent Service	SLA based	Singular jobs, periodical jobs, compute-intensive and data-intensive interactive application	Pricing	PaaS

Table 2.1: Summary of academic cloud brokering solutions.

2.3 Content-based publish-subscribe systems

Event-driven architectures are becoming more prevalent recently in multiple technological paradigms, with message brokers being the cornerstone of these architectures [54]. One of the best implementations of these message brokers are content-based publish-subscribe systems (CBPS) because of their ability to allow subscribers to specify their interests and only receive notifications according to those interests, as opposed to the processing overhead that subscribers to topic-based publish-subscribe systems have to perform [53, 55, 44, 52].

Therefore, it has been opted for a content-based publish-subscribe distributed system to evaluate the implementation of this work. More specifically, E-SilboPS[44, 53, 52, 45] has been used, which is a content-based publish-subscribe system specifically designed to be elastic due to its scaling algorithm. Its architecture is inspired by other CBPS like SIENA [11, 10] and E-StreamHub [8].

Despite their importance, an obstacle to the research of these systems is the lack of real and publicly available workloads, due to the privacy issue involved in disclosing the interests (subscriptions) of users and other commercial interests of the companies. The authors of [55] note this problem and address it by proposing a wide-area workload generator for content-based publish-subscribe systems. For this purpose, both subscriber interests and geographic locations are generated through statistical summaries of public data traces. However, despite indicating its intention to make this generator public, it is not currently available.

2.3.1 Improved version of E-SilboPS

This section presents the latest and improved version of E-SilboPS, a context-aware content-based publish-subscribe middleware. From an architectural point of view, E-SilboPS is a distributed system specifically designed to be elastic, leveraging the elasticity offered by Cloud Computing to deliver high performance capabilities.

Much of the evaluation of this work has been done on E-SilboPS, as an archetype of a distributed system typically deployed in a cloud environment to offer its services. For this reason, and for the sake of clarity, it is necessary to explain this publish-subscribe middleware in some detail and to emphasize the improvements that have been made to it with respect to previous versions [52, 53] (which are recommended for more detailed information), as well as an evaluation of its performance to provide a baseline for a better understanding of the rest of the experimental work presented in this document.

2.3.1.1 General overview

As already mentioned, E-SilboPS is a context-aware content-based publish/-subscribe middleware specially designed to be elastic. This middleware is the elastic version of SilboPS and was born as a result of a doctoral thesis [51] of which this work is its natural continuation. In addition, it has acquired great relevance as shown by its integration with technological paradigms that demand high performance and are transforming our lives such as Internet of Things (IoT), Smart Cities, Fog Computing [52, 53, 44], and more generally, in any event-driven architecture [54].

Inspired by StreamHub's architecture [7], E-SilboPS is composed of four operator layers. Each of these layers is formed by a variable number of instances of the corresponding operator. Figure 2.4 shows this architecture as well as its operational flow, which is detailed below.

Connection Points (CP) are the entry and exit points of the system, since they are in charge of managing the connections with the clients and sending the notifications to the corresponding interested subscribers.

- Subscriptions are forwarded to any *Access Point* (anycast).
- Notifications are forwarded to any *Access Point* (anycast).

Access Points (AP) forward messages, whether notifications or subscriptions, to the *Matchers* layer. More specifically:

- Subscriptions are forwarded to a specific *Matcher* instance (unicast) determined by a selector.

- Notifications are forwarded to all the *Matchers* (broadcast).

Matchers (M) store incoming subscriptions and process the notifications, that is:

- Subscriptions are stored in the internal structure.
- Notifications are matched against the stored subscriptions and dispatched the local result set to the corresponding *Exit Point* (unicast).

Exit Points (EP) forward the notifications to the *Connection Points*.

- Subscriptions never reach this point.
- Collects all partial result sets from the matchers to merge them into a final result set composed of all *Connection Points* instances to which notification must be sent.

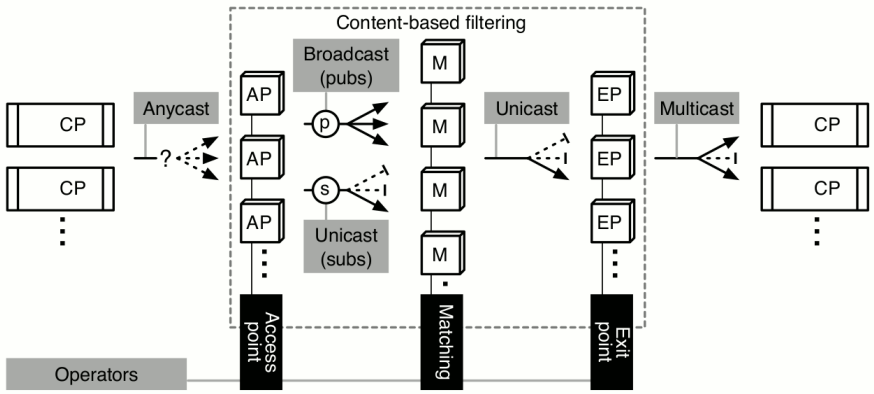


Figure 2.4: Extracted from [7]. StreamHub and E-SilboPS architecture and operational flow.

Each of these operator layers can scale independently by adding and/or removing instances of those operators to match the demand for the workload at each moment. In this way, the system has a global property of elasticity [53], since it can start from a basic topology and scale out/in each one of the operator layers to be able to optimally manage the workload at each moment, avoiding the need to size the system *a priori* from the beginning.

One of the most valuable features of this system is, undoubtedly, its scaling algorithm that "is able to provide an uninterrupted service to end users even during the scaling operation since its internal state repartitioning is transparent for publishers or subscribers". Moreover, "the scaling operation is time-bounded and depends only on the dimension of the state partitions to be transmitted to the different nodes". More specifically, this algorithm is based on the Chandy-Lamport algorithm [15], commonly known as the snapshot algorithm, which allows to create a consistent cut of the overall system state. Figure 2.5 summarizes the message exchange during the scale-out operation of the *Matcher* layer. All information regarding the scale-out and scale-in algorithms of each of the operator layers can be found in [43, 51, 53]. As shown in Figure 2.5 by dashed lines, some coordination between the different layers is necessary to maintain consistency of system state. In particular, the implementation uses Zookeper[6] to ensure distributed coordination between the operator slices.

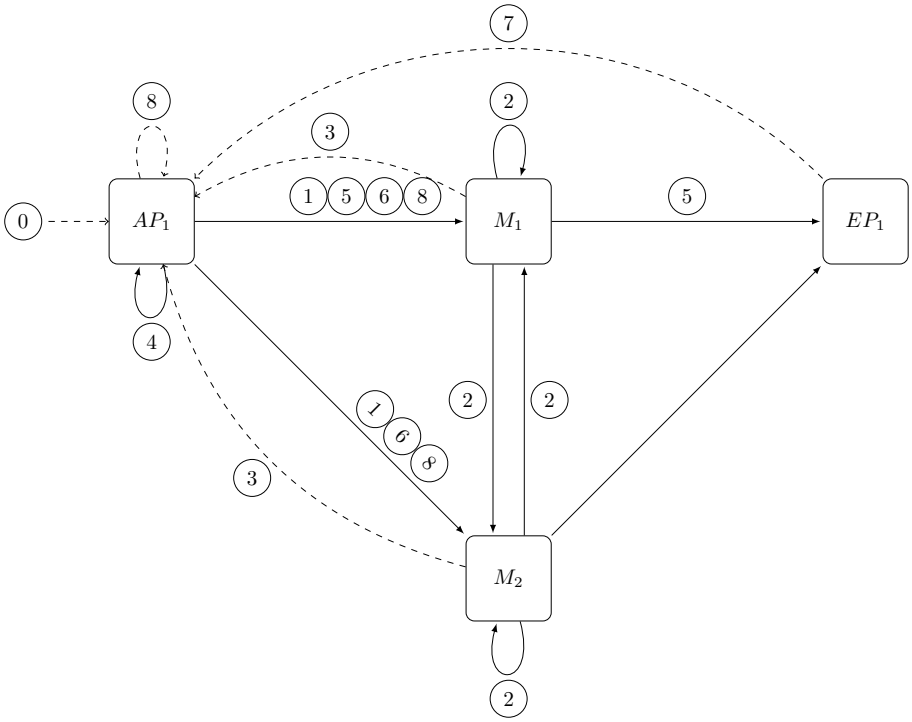
NOTE: throughout this document the form *X-Y-Z* is frequently used to describe the E-SilboPS topology with *X*, *Y* and *Z* being the number of instances of corresponding *AP/CP*, *Matcher* and *EP*, respectively.

2.3.1.2 Improving the performance of E-SilboPS

Some important updates have been made in the context of this work to improve the performance of E-SilboPS with respect to the latest publications of this system [53]. Therefore, this section is dedicated to explain these changes and how they impact on the performance of the system. As explained, E-SilboPS is one of the systems on which the proposals of this thesis are evaluated, mainly at the performance level. This section is intended to illustrate these changes and to provide the reader with a baseline of system performance.

In any system, the maximum throughput is limited by the lowest throughput of each of its parts. In other words, the maximum throughput of the system is that of the component that acts as the bottleneck. In the case of E-SilboPS, the maximum throughput is marked by the component responsible for the highest processing load, which in this case corresponds to the processing of the notifications to determine which subscriptions are

[6] `https://zookeeper.apache.org/`



⓪ Start Scale-In/Out

① Start Scale-In/Out + Selector

② Add matcher state

③ Change-Ready

④ Change-Matcher-Ready

⑤ Change-Matcher-On-Exit Point

⑥ Change-Matcher

⑦ Exit Point-Matcher-Updated

⑧ Scale-End

Figure 2.5: Exchange of messages during the scale-out of the Matchers layer (being $M2$ the new *Matcher* instance deployed). Solid lines represent the communications established between operators for sending notifications and subscriptions. The dashed lines represent the messages used by the Distributed Coordinator to synchronize the layers.

matched. In other words, the theoretical bottleneck of the system was in the *Matcher* instances, more specifically, in the processing of notifications, and therefore the maximum throughput of the system would be the maximum throughput of these components.

Note that the concrete values are meaningless as they depend on the infrastructure where the system is running, but let's take the concrete values obtained from the experimentation in the circumstances described in [53] to be able to compare and better understand the performance differences. At the beginning of this work, and using the previous test environment, it was empirically measured that the maximum throughput of the notification processing of a matcher instance was 1,2M notifications/s, and therefore it could be expected that the maximum throughput of the whole system was also 1,2M notifications/s. However, it was empirically proven that the system, even with the simplest 1-1-1 topology was far from reaching that global throughput (i.e. 20k notifications/s on average at most). The rest of the chapter is devoted to explaining the reasons for this behavior and how they were solved.

2.3.1.2.1 Improving the performance of connections As it is a distributed system, as explained above, the first point that could explain this loss of performance was the connections between the different operators' instances. The performance tests demonstrated this hypothesis, as the maximum throughput of a connection was the same as that of the entire system (i.e. 20k notifications/s on average at most). The experiments have been ran on an Intel Core i7-4790K 4.00GHz and 16 GB of RAM running Linux 5.3.0-28-generic and openjdk 11.0.7

To try to mitigate this loss of performance on the one hand, some aspects of implementation (at the code level) were improved, and the serialization of messages was identified as the cause of this loss of throughput. To solve this last issue, other serialization libraries were tested such as *Kryo*[7] and *FAST*[8] and the performance they offered was analyzed. Based on the results obtained from this analysis, it was decided to use the FAST library for the serialization of the messages that were sent by the connections between the instances of the operators.

[7] `https://github.com/EsotericSoftware/kryo`
[8] `https://github.com/RuedigerMoeller/fast-serialization`

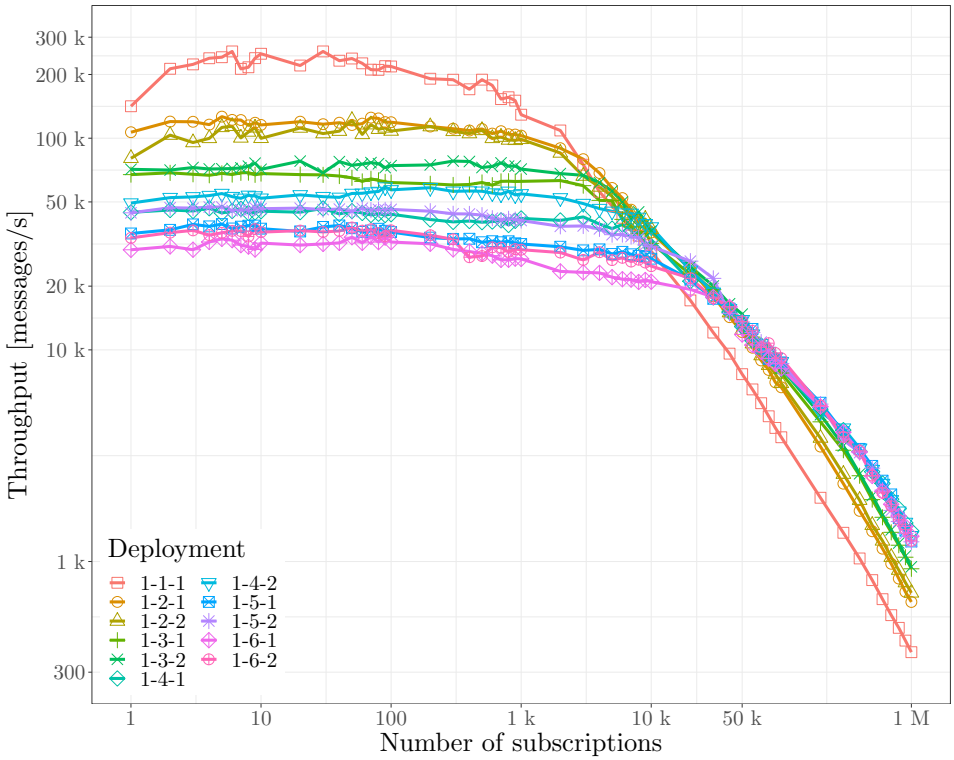


Figure 2.6: Throughput of the new E-SilboPS version with different deployment configurations.

Figure 2.6 shows the performance (throughput) of the new version of E-SilboPS (code level improvements and the use of the FAST library for message serialization) with different deployment configurations. On the other hand, Figure 2.7 shows a comparison of the throughput between the new and the old version of E-SilboPS using different deployment configurations. As can be seen, the new E-SilboPS version of this work achieves a throughput improvement of up to 5 times.

However, this remarkable improvement only increases the throughput of the connections, but as it can be seen, it is not enough to make the global throughput of the system equal to the maximum throughput of the *Matcher's* notifications processing. This problem will be discussed in the next section.

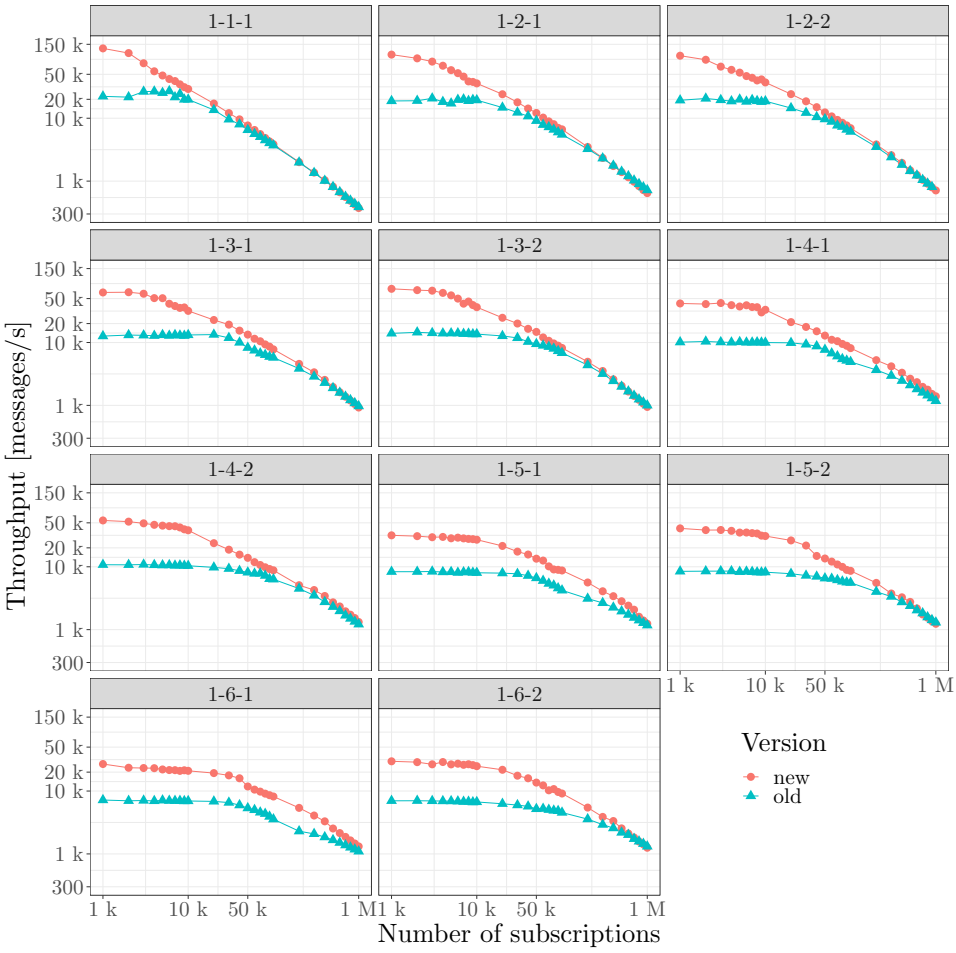


Figure 2.7: Comparison of new and old E-SilboPS versions throughput with different deployment configurations.

2.3.1.2.2 Updating the architecture As mentioned in the previous section, the improvement of the throughput of the connections, although significant, is not enough to achieve that the global throughput of the system is that of the processing of the Matcher notifications. This can be clearly seen in Figure 2.8, which shows how the maximum throughput of *Matcher*'s notifications processing reaches a maximum of 1,2M messages/s, while the global throughput of E-SilboPS is the maximum of a single connection, approximately a maximum of 200k message/s. Of course, this would not be a problem in a 1-1-1 topology, since each operator layer has only one connection, and therefore that connection is the bottleneck, explaining that the total throughput of the system would be that of that connection. However,

it was experimentally proven that this same behavior occurred with other topologies such as 3-1-3, in which the expected behavior would be that the global throughput would ideally be multiplied by 3, since it has 3 parallel connections between the *AP/CP-Matcher* and *Matcher-EP* layers.

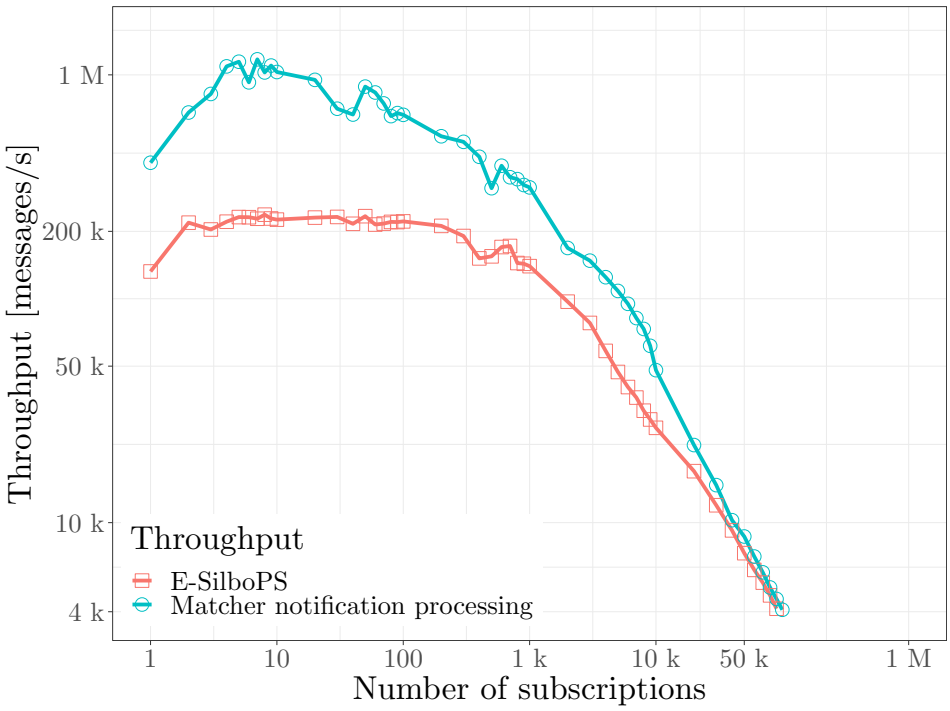


Figure 2.8: Comparison of E-SilboPS throughput and notification processing Note that the E-SilboPS throughput is that of a single connection, regardless of the topology used.

By combining performance testing and architecture analysis, the problem was isolated in the *Matcher* architecture, more specifically in the connections to the *Exit Points* layer. More specifically, the problem was that by having a single output buffer, it was not possible to parallelize the sending of the local result sets of subscriptions that matched the incoming notification to the different *Exit Points* instances. Finally, the solution adopted is presented in Figure 2.9, and consists in adding an output buffer for each connection to an instance of the Exit Points, so that it is possible to parallelize the sending in the same way as in the connections between the layers of *CP/AP-Matcher* operators.

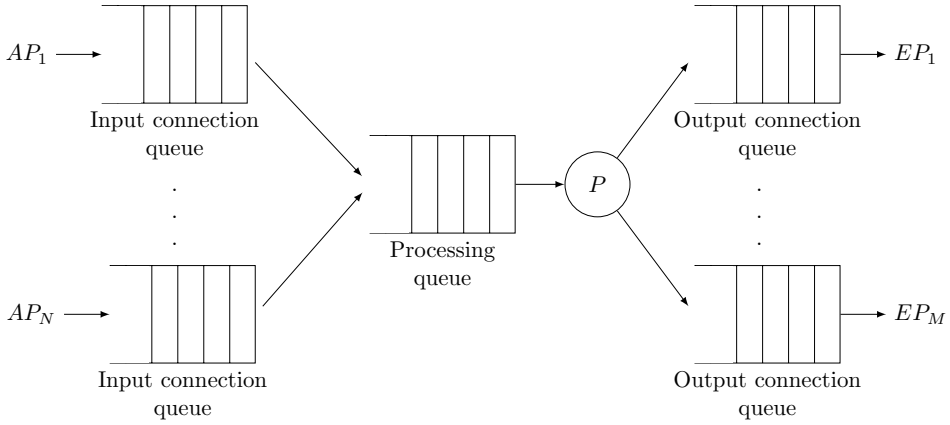


Figure 2.9: Improved architecture of the E-SilboPS *Matcher* with an output buffer for each *Exit Point* connection.

Obviously, in order to achieve the theoretical maximum throughput, a suitable E-SilboPS configuration is necessary. The following rule serves as a first approximation for the calculation of such a configuration:

let $Matcher_{max\ th}$ be the maximum throughput for processing notifications of a *Matcher* instance, and $E\text{-}SilboP_{max\ th}$ be the maximum throughput of the system (i.e. the theoretical maximum throughput of the system), calculated as $E\text{-}SilboPS_{max\ th} = X \times Matcher_{max\ th}$ and $connection_{th}$ the maximum throughput of a connection, then for the system to reach $E\text{-}SilboPS_{max\ th}$ an N-X-N configuration is needed, being X the number of matchers deployed and N calculated as $N = \frac{E\text{-}SilboPS_{max\ th}}{connection_{th}}$.

With the data offered above and let $X = 1$, $Matcher_{max\ th} = 1,2M\ messages/s$, $E\text{-}SilboP_{max\ th} = Matcher_{max\ th} = 1,2M\ messages/s$ and $connection_{th} = 200k\ messages/s$, then $N = \frac{1,2M\ messages/s}{200k\ messages/s} = 6$. Therefore, the minimum deployment configuration of E-SilboPS should be 6-1-6.

2.3.1.3 Discussion

By means of the solutions presented in the previous sections, the global throughput of the system was able to reach the maximum throughput of *Matcher*'s notification processing, which is the upper limit that the system can theoretically give. In other words, by improving the performance of serialization and the architecture of the Matcher, it has been possible, with the appropriate deployment configuration, to overcome the bottleneck of the performance of the connections to achieve the maximum throughput, limited by the processing capacity (i.e. CPU bound) and not by the performance of the connections of the distributed system. This allows E-SilboPS to offer maximum performance that only depends on the infrastructure on which it is deployed, not being limited internally at the system level by other bottlenecks.

It is important that the E-SilboPS configuration is the minimum necessary at any given time to avoid wasting resources, since a higher than necessary number of N instances of *CP/AP* and *EP* is an unnecessary extra cost, while the same maximum performance could be obtained with an N-X-N configuration without wasting resources. On the other hand, it is necessary to ensure that resources are sufficient, since an ideal N-X-N topology will not be able to deliver maximum performance if one of the resources is scarce (e.g. CPU saturated at 100% utilization). This may seem obvious, but it is of vital importance in Cloud environments.

All this developed work has allowed to have an extensive knowledge of E-SilboPS and its performance. This knowledge about the implementation and performance of the solution, together with the fact that it offers a similar performance to the existing commercial solutions at the time of writing of this work, have been the reasons for choosing E-SilboPS as an archetype of a distributed system typically deployed in a cloud environment for the evaluation of part of this work.

Chapter 3

Goals and Contributions

The only way to test a hypothesis is to look for all the information that disagrees with it.
— Karl Popper

3.1 Objectives

Although the topic of auto-scaling services has been widely discussed in the past, it is still very active today due to new technologies that open new horizons on this topic, such as containerization and management services like Docker[1] or Kubernetes (K8s)[2], respectively. The aim of this thesis is to address and solve several aspects of auto-scaling that have not been addressed in the literature, and which are essential to achieve true auto-scaling systems.

One of the main problems with auto-scaling systems today is the complexity of configuring them to ensure compliance with SLAs that cloud consumers are interested in. Auto-scalers tend to focus on metrics that are far removed from SLA parameters such as CPU usage or workload, as opposed to the economic cost or response time that the user may be interested in. Predicting (in the case of predictives) or establishing rules (in the case of

[1] `https://www.docker.com/`
[2] `https://kubernetes.io/`

reagents) on those metrics is only a necessary step, but not sufficient, to ensure compliance with SLAs. The missing step in defining the relationships between these metrics is usually either left to the cloud consumer, which is prone to errors, or is done *ad-hoc* for a particular service, to the detriment of the generality of that auto-scaler. One way to achieve this is, for example, in the case of predictive systems, to make predictions about how SLA parameters will behave in the future, instead of predicting the workload and then having to study how that might affect the SLA.

Another major problem of today's auto-scaling systems is that they do not automatically address the detection of the resource that is the bottleneck, and therefore do not automatically determine the metric or metrics that are KPIs to make the corresponding scaling decision. The vast majority of auto-scalers are *ad-hoc* for a given service, and therefore, take for granted the resource to be scaled and the metrics that are KPIs. However, this does not favour the adoption of these systems in a generic way by other services, since the limiting resources may be other (e.g. cpu-bound vs memory-bound). Among the aims of this thesis is the study of the relationships between low level metrics, or resource use metrics, and high level metrics, or SLA parameters, in order to understand their relationships and to be able to establish a mechanism that allows, automatically, to establish a mapping between these two levels of metrics. In this way, this mechanism will allow a auto-scaling system not only to automatically determine, for any service, the KPIs to be monitored and their relationship with the SLA parameters, but also the resource(s) to be scaled in each moment to ensure compliance with the SLAs.

Finally, some cloud services need to be deployed in multi-cloud environments to meet certain SLAs, such as geographic restrictions on where data can be stored in order to comply with applicable laws. Multi-cloud environments bring with them a multitude of complexities that have been studied in other works and are beyond the scope of this thesis. However, in this work it has been wanted to address those aspects that are directly and more closely related to the auto-scaling systems on which this thesis is focused. More specifically, one objective of this thesis is to propose a generic enough and extensible set of vendor-neutral and cross-provider measurable metrics that allows the proposed auto-scaling system to monitor resources in multi-cloud environments.

3.2 Hypothesis and research questions

The novelty of our proposal lies in how we address and validate the following hypothesis. For this purpose, we state four specific research questions, whose answers represent the solution to the above mentioned problems and constitute the backbone around which the remainder of the work is structured:

Hypothesis. *It is possible to automatically scale cloud services by means of an auto-scaler system that allows (i) compliance and adherence to an SLA specified by the cloud-consumer, (ii) automatic detection of the resource acting as a bottleneck, and (iii) monitoring of the metrics needed to make the necessary scaling decisions can be performed in multi-cloud environments.*

RQ 1. *Is it possible to design a auto-scaler focused on the fulfillment of the SLAs defined by the cloud-consumer capable of leveraging the combination of predictive auto-scaling techniques, based on data analysis, and reactive techniques as a contingency method for possible failures of the predictive approach? (Problem1)*

RQ 2. *Is it possible to design an auto-scaling system capable of automatically detecting the resource that is the bottleneck, and therefore has to be scaled, as well as the KPIs to be monitored, by establishing mappings between high level metrics (SLA parameters) and low level metrics (resource utilization metrics)? (Problem2)*

RQ 3. *Is it possible to automatically establish the relationship between high-level objectives (or SLOs) and conditions expressed as low-level metrics for a given cloud application? (Problem2)*

RQ 4. *Is it possible to define a generic enough and extensible set of vendor-neutral and cross-provider measurable metrics? (Problem3)*

3.3 Contributions

The main contributions of this thesis are the following:

1. The design of an auto-scaling framework for distributed services that combines the advantages of proactive and reactive approaches according to the situation to decide the optimal scaling actions in every moment. This auto-scaling framework works by learning and predicting patterns in the performance behavior of distributed systems in order to take the appropriate scaling decisions at any given time to ensure compliance with SLAs. The main novelties introduced by this auto-scaling framework are (i) a predictive model of the high-level metrics trend which allows to anticipate changes in the relevant SLA parameters (e.g. performance metrics such as response time or throughput) and (ii) a reactive contingency system based on the estimation of high-level metrics from resource use metrics, reducing the necessary instrumentation (less invasive) and allowing it to be adapted agnostically to different applications (RQ1).

2. An implementation of the auto-scaling framework described above, called FLAS (Forecasted Load Auto-Scaling), for the use case of a content-based publish-subscribe middleware (E-SilboPS) that is the cornerstone of an event-driven architecture. This is the first auto-scaling system for content-based publish-subscribe distributed systems (although it is generic enough to fit any distributed service).

3. A FLAS evaluation based on several test cases recreating not only the expected contexts of use, but also the worst possible scenarios (following the Boundary-Value Analysis or BVA test methodology), the approach has been validated and demonstrated the effectiveness of the solution by ensuring compliance with performance requirements over 99% of the time.

4. A data-driven, application-agnostic model capable of automatically mapping low-level metrics (or resource utilization metrics) to high-level metrics (or SLA parameters), which is part of the FLAS implementation. This model allows (i) to automatically determine the resource that is the bottleneck of a service, and that therefore, must be scaled, and (ii) the KPIs responsible for the behaviour of the SLA parameters (RQ2).

5. A knowledge-based system that is capable of automatically determining the low-level metric conditions (i.e. key performance indicators or KPI) necessary to achieve a high-level metric condition in order to ensure compliance with the high-level metric objective (SLO), the enforcement of which ensures compliance with these SLAs (RQ3).

6. A harmonized and extensible set of vendor-neutral metrics and how they can be measured in the different cloud providers. Furthermore, a validation of this set of metrics is presented through a couple of representative use cases (IaaS and PaaS respectively) in a multi-cloud environment formed by leading cloud providers. This validation demonstrates how the proposed harmonized set of monitoring metrics is able to abstract the specificities between multiple cloud providers offerings, making the monitoring and enforcing of SLOs in a multi-cloud environment transparent for cloud consumer (RQ4).

3.4 Limitations

The following limitations of this work are mainly due to the need to limit the problem to establish the scope and limits of the problem to be studied and solved, and to be able to offer a document that can be self-contained for the sake of greater clarity.

One of the main limitations of the auto-scaling proposed in this work is that it does not address the dimension of the how much to scale. This dimension is fundamental in deciding how many resources should be added or released in a given scaling action. For the experimentation of this work, a very *naïve* policy has been taken to add twice or to halve the resources at the time of an scaling action. Obviously, this approach is far from optimal and is therefore included in future work.

On the other hand, this work focuses on the performance SLAs that are most used today, i.e. those that set requirements on SLA parameters such as response time or throughput. It is a limitation of this thesis not to experiment with SLA parameters related to economic cost, or to energy consumption (the latter is gaining some importance in the literature lately). However, although these parameters have not been addressed in the evaluation of this work, the proposed solution should work indistinctly with these parameters, since they are, after all, continuous variables and could therefore

be predicted with the models presented here. The greatest difference would be found in the management of the amount of resources to be added and released, but as commented on in the previous paragraph, this dimension is not considered in this work.

The generic enough and extensible set of vendor-neutral and cross-provider measurable PaaS metrics proposed in the last part of this work only takes into account the three main cloud providers currently (Amazon Web Services, Google Cloud and Microsoft Azure). There are many other cloud providers today, but these three providers account for the vast majority of the PaaS cloud services market today.

Other limitations of this work, but of lesser importance, is that the prediction models used in this work are mostly statistical models, since in this first approach it is considered important to be able to understand how these models predicted on the basis of the predictor variables and to better understand the functioning of auto-scaling. However, it is possible to use any other predictive model (e.g. ANN) as long as the interface of the auto-scaling solution presented here is respected.

Chapter 4

Combining proactive and reactive auto-scaling in an architecture for distributed systems: FLAS

Essentially, all models are wrong, but some are useful
— George Box

This chapter introduces FLAS (**F**orecasted **L**oad **A**uto-**S**caling), an auto-scaler for distributed services that combines the advantages of proactive and reactive approaches according to the situation to decide the optimal scaling actions in every moment [46]. The main novelties introduced by FLAS are (i) a predictive model of the high-level metrics trend which allows to anticipate changes in the relevant SLA parameters (e.g. performance metrics such as response time or throughput) and (ii) a reactive contingency system based on the estimation of high-level metrics from resource use metrics, reducing the necessary instrumentation (less invasive) and allowing it to be adapted agnostically to different applications. A FLAS implementation is proposed for the use case of a content-based publish-subscribe middleware (E-SilboPS) that is the cornerstone of an event-driven architecture. This is probably the first auto-scaling system for content-based publish-subscribe distributed systems (although it is generic enough to fit any distributed service). Through an evaluation based on several test cases recreating not only

the expected contexts of use, but also the worst possible scenarios (following the Boundary-Value Analysis or BVA test methodology), this approach has been validated and demonstrated the effectiveness of this solution by ensuring compliance with performance requirements over 99% of the time.

More specifically, this chapter address Problems 1 and 2 offering a response to RQs 1 and 2.

4.1 System model

4.1.1 System configuration model

Lets consider a cluster of M nodes, understanding a node as an abstract entity of infrastructure that allows to execute a software (i.e. physical or virtual machines of a cluster, containers, etc), in which the different operators of a distributed system are executed. Each operator can have several instances, and the number of instances of each operator can be increased and decreased independently. Thus, a 1-3-2 configuration indicates that there is 1 instance of operator 1, 3 instances of operator 2 and 2 instances of the third operator. In principle, they are not considered restrictions in the cardinality of instances of each operator deployed in each node.

Through a scaling action or *sa*, the operator instances of a certain configuration can be increased or decreased. Continuing with the notation of F. Lombardi et al. in [29], a scaling action requires a time T_{sa} that depends on the workload, and in case a reconfiguration of the internal state of the instances is necessary, it will also depend on the size of the state that has to be exchanged and the number of instances before and after performing the scaling action.

According to [50], a scaling action is defined by three points in time: (i) *sa* demand point (DP_{sa}) which is the point at which a new configuration is required, i.e. a scaling action; (ii) *sa* triggering point (TP_{sa}) which is the point at which a scaling action is activated, and (iii) *sa* reconfiguration point (RP_{sa}) is the point in time at which the scaling action has been completely terminated. Therefore, as shown in Figure 4.1, the time of a scaling action T_{sa} is calculated as the difference between RP_{sa} and TP_{sa}, $T_{sa} = RP_{sa} - TP_{sa}$.

In addition, a scaling action is clearly defined by specific values of four dimensions, namely:

When. It refers to the time at which the scaling action should be performed. As mentioned above, there are two approaches, (i) reactive techniques in which the scaling action is taken in reaction to a change (a certain condition is met) and (ii) predictive (also known as proactive) techniques that aim to anticipate changes before they occur in order to add or release the necessary resources by the time that change occurs.

How. It represents the type of scaling (horizontal or vertical) and the specific scaling action (scale-out/in or scale-up/down for horizontal and vertical scaling respectively).

What. It refers to which resource must be scaled to meet a given SLA. Some applications may be CPU bound while others may be memory bound or limited by other resources.

How much. It denotes the amount of resources that must be added or released to satisfy the SLA.

Although FLAS is a generic solution, for the sake of clarity this work is focused on a specific case combining predictive and reactive techniques (*when*), horizontal scaling (*how*), a CPU-bound type application, which implies that the resource to be scaled is the processing capacity (*what*) and as a first approximation is is going to add or reduce the resources to double or half in each scaling action (*how much*).

4.1.2 Workload and performance model

As indicated above, there is a wide variety of SLAs to reflect different end-user interests. This work focus on performance SLAs. In particular, this work focus on the two performance metrics par excellence in this area (high-level metrics or SLA parameters) which are throughput and response time, although economic cost and energy consumption are also considered to the extent that over-provisioning is minimized. However, FLAS could work with other types of SLAs as economic cost or energy consumption by modifying the SLA parameters and providing the corresponding sources of monitoring of those parameters.

Following the modeling described in [29, 28], end users or clients interact with the distributed system by sending messages. This input rate or workload $\lambda(t)$ is defined as the number of messages received by the system per unit of time in a given instant t. The system has a capacity to process a certain number of messages per time unit called service time S. Depending on the application, the service time can be constant or can be variable depending on the system state at a time t, $S(t)$. The system is said to be in saturation or overload when it receives more messages per time unit than it can process in that same amount of time, i.e. $\lambda(t) > S(t)$. When the system is in saturation, the successive messages are queued, since they cannot be processed immediately. The response time $RT(t)$ is the time required by the system to process a message, which will depend on the occupation of the system, since if the system is saturated the response time will be greater because it has, in addition to the processing time $S(t)$, to wait a time in the queue before being processed. On the other hand, the throughput $X(t)$ is defined as the amount of messages that the system can process per unit of time. The behavior of both metrics will be determined by the state of the system:

Normal: input rate is less than or equal to the service rate, i.e. $\lambda(t) \leq S(t)$ and therefore the response time can be equivalent to the service rate, i.e. $RT \simeq S(t)$ and the throughput will be equal to the input rate, i.e. $X(t) \simeq \lambda(t)$ (ignoring propagation delays).

Saturation or overloaded: occurs when the input rate is higher than the service rate, i.e. $\lambda(t) > S(t)$ and therefore messages have to be queued. This causes the response time to increase exponentially (Figure 4.1) and the throughput to remain constant at a value close to the service rate $X(t) \simeq S(t)$.

Although the solution proposed in this work is not based on queuing theory, it is very interesting for the modelization of this problem. More specifically, as indicated by k.Lazowska in [28], asymptotic bound analysis provides optimistic and pessimistic limits for throughput and response time that provide rapid insights that are essential for determining the main factors affecting performance.

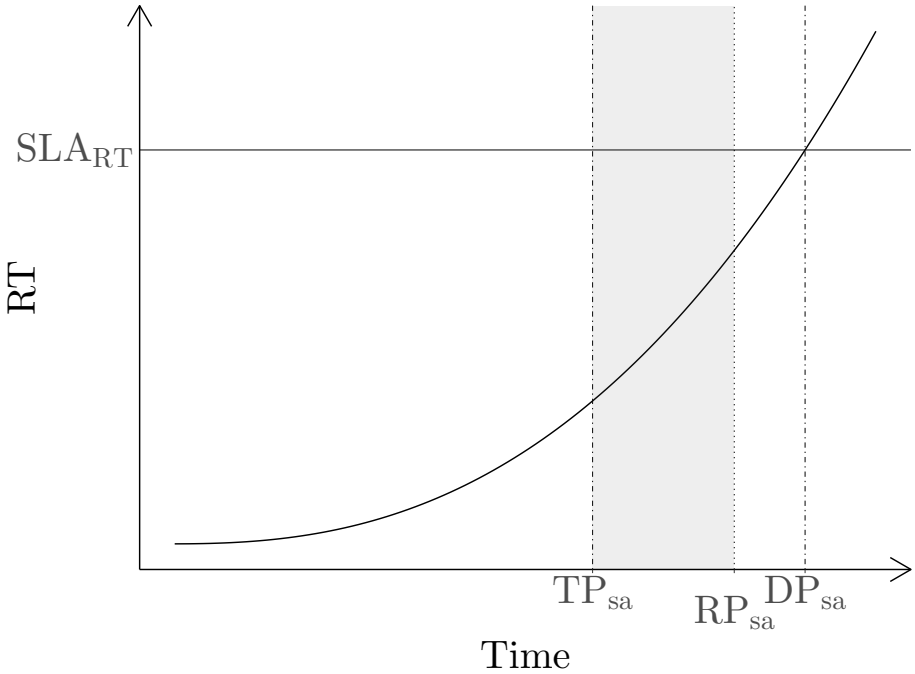


Figure 4.1: Example in which the response time of a distributed system increases rapidly as a consequence of saturation. To avoid exceeding the maximum response time imposed by the SLA it is necessary to perform a scale-out action that is triggered in TP_{sa} and ends in RP_{sa}. The scaling time will be $T_{sa} = RP_{sa} - TP_{sa}$. As it is a scale-out operation that ends before the DP_{sa} instant, the system will be in over-provisioning a time equal to $DP_{sa} - RP_{sa}$.

When the system reaches its saturation point, the system begins to act in a saturated state and performance degrades and therefore some service level objective of the SLA is often violated, especially if the trend continues. To avoid this, a scale-out action is usually performed, which allows the necessary resources to be added so that the service offered does not degrade. On the other hand, a scale-in action is necessary when the current resources are greater than those required to provide service without violating the SLA, thus saving costs or energy consumption.

Considering that the objective of the scaling actions is to return or maintain the system in a normal operating state to avoid non-compliance with SLAs, the scaling action should ideally be completed at the same time as it is demanded. Therefore, one of the objectives of the scaling systems in terms of the time dimension would be to ensure $RP_{sa} = DP_{sa}$ through reactive or proactive techniques. When this is not fulfilled, one of these two alternatives occurs:

- $RP_{sa} > DP_{sa}$: in this case the scaling action will be completed after it is needed. In the case of a scale-out it would mean an under-provisioning of resources (a configuration with less resources than required) and in the case of a scale-in it would mean an over-provisioning of resources (a configuration with more resources than required).

- $RP_{sa} < DP_{sa}$: in the opposite case to the previous one, the scaling action ends before the instant it is necessary. In this case, if the scaling action was a scale-out or a scale-in it will result in over-provisioning or under-provisioning of resources respectively (Figure 4.1).

4.1.3 Problem statement

In general, the problem of auto-scaling is to calculate the specific scaling action (i.e. specific values at each of its dimensions) needed at each moment to ensure compliance with an SLA. Since this problem is too vast to cover its entire domain, and because elasticity is a per-application task [2], this work focuses on a subset of applications that share a common set of characteristics. More specifically, it is focused on a generic auto-scaling solution for high performance distributed systems that represents by itself a quite wide and diverse set of applications.

As mentioned above, the focus has been on performance SLAs and the dimensions of when to scale and what resource to scale. The dimension of how to scale is application dependent and how it is designed, so it cannot be addressed in a generic solution and will have to be application specific. However, it is not a limitation of this solution as shown in the evaluation of this work. As for how much to scale, it has been opted for a first approach of multiplying or dividing by two the resources to scale that allows to verify and evaluate this solution without introducing too much complexity. However, future work points in that direction.

More specifically, the aim is to minimize the distance between the moment when a new configuration is demanded and the moment when the scaling action covering that demand is concluded, i.e. minimize $|DP_{sa} - RP_{sa}|$, to minimize the time of over- and under-provisioning of resources. In addition, as part of the solution to this problem, it is intended to develop a model that allows the mapping of low-level behavior of the application, represented by the behavior of its low-level or resource metrics, and the SLA parameters, or high-level metrics, to automatically detect which is the resource to be scaled and which low-level metric(s) are the most descriptive and useful when monitoring the system (KPIs). All of this with the main objective of not violating the SLA or minimizing the time that is being violated.

4.2 FLAS architecture

This section presents in a generic and agnostic manner the architecture of FLAS, as well as the components that compose it and its workflow. The following section (Section 4.3) describes the functional flow of the components that compose the architecture presented here for a specific integration with a distributed system.

FLAS, like other auto-scaling systems [29], works in two phases, a monitoring phase and an auto-scaling phase. In the first phase, the system collects the necessary data to acquire the knowledge needed to generate prediction models. More specifically, it collects data on the workload, the evolution of trends over time of this workload, the behavior in terms of low-level resources and the high-performance variables used in the SLA. Once

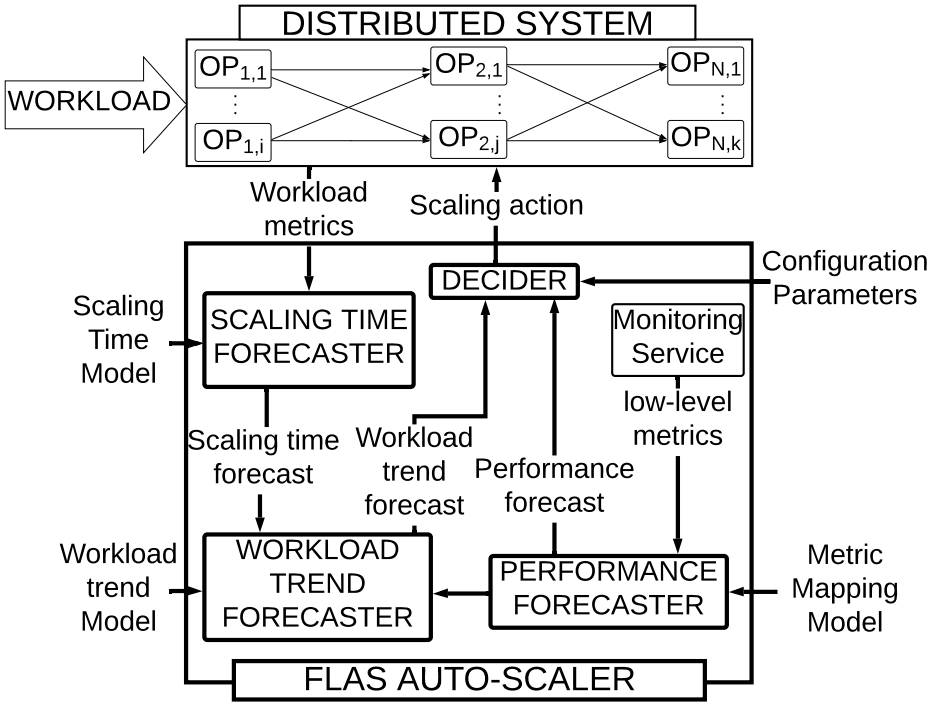


Figure 4.2: Functional diagram of the components that form the FLAS architecture and the integration with a distributed system.

these models have been generated, in the auto-scaling phase, the different modules will make use of these models to provide the necessary predictions to the decision maker, who will ultimately be responsible for deciding which auto-scaling actions should be applied, if any.

As shown in Figure 4.2, the FLAS architecture consists of 4 functional modules: (i) *Scaling Time Forecaster*, (ii) *Workload Trend Forecaster*, (iii) *Performance Forecaster* and (iv) *Decider*. In the following subsections these functional modules will be explained in more detail, however, the implementation of each of them may require slight adjustments to integrate with the different existing distributed systems. An example of this integration is explained in more detail in sections 4.3 and 4.4. In addition, each of these modules works as a black-box within the FLAS architecture, which allows replacing the implementation of each of these modules in a transparent manner as long as they respect the definition (interface) of these modules.

4.2.1 Scaling Time Forecaster

This module is in charge of predicting the time that a scaling action will take depending on the workload of the distributed system, i.e. $T'_{sa}(workload)$, being T'_{sa} the prediction of the scaling time and T_{sa} the actual time of that scaling action. Although the scaling time is mostly influenced by the workload, the predictions can be more accurate if other variables are included on which this scaling time could depend on the target distributed system. For example, in some distributed systems it is likely that the configuration before and after the scaling operation is a factor that influences this time, especially if the operators that are scaled are operators with internal state that has to be reconfigured.

4.2.2 Workload Trend Forecaster

This module is responsible for predicting the trend of system performance (i.e. SLA parameters) in the near future. Since a single prediction in a single future instant can be misleading, it is not only predicted in a single future instant, but in a time horizon h (also called a forecast window) composed of several consecutive future instants. This prediction time horizon (h) is used to see how these predictions will evolve at different time points in the future so that the trend can be consolidated and shown in a way that is not misleading. However, the choice of this h-value must be made carefully as if it is too short many fluctuations may be observed, and if it is too wide relevant details of the behavior may be lost. The future instant from which these h predictions are made (indicating the value of h the number of consecutive predictions in time to be made, or window size) is usually calculated by means of the scaling time predicted by the *Scaling Time Forecaster* module. For example, if $h = 4$, t_0 is the current instant and $T'_{sa}(t_0)$ is the predicted scaling time for the current instant, then the *Workload Trend Forecaster* will make predictions in the future instants $t_i = t_0 + T'_{sa}(t_0) + i,\ \forall i \in \{0, \ldots h-1\}$

The objective of this module is to provide the *Decider* module with information on the trend of the performance of the distributed system in the prediction time horizon, indicating, for example, if during the prediction time horizon the response time will increase exponentially or if the throughput will remain constant in the next instants, both being symptoms that the system is heading towards its saturation point.

To accomplish this, during the profiling phase, the behavior of the performance metrics or SLA parameters must be recorded as a function of time to generate a time series model based on which to make these predictions in the auto-scaling phase.

4.2.3 Performance Forecaster

This module is responsible for the prediction of the performance metrics (high-level metrics) that compose the SLA. More specifically, it aims to monitor the use of low-level resources to extract a model capable of capturing the relationships between low-level and high-level metrics. This is essential because it allows to know the key performance indicators (KPIs) of the application, which indicate the bottleneck resource of the application, since if these resources are saturated (there are no more available), they lead the application to enter a state of saturation and therefore a service degradation. This knowledge is essential, because it indicates (i) the resource(s) to be monitored and by means of which metrics, and (ii) the resource to be scaled so that the application is not saturated, and therefore to avoid SLA violation. As mentioned, there are few solutions that address this problem in the literature, since they assume the resource to be scaled, and therefore the metric to be monitored, but this not only restricts the range of application of these solutions, but also, a model that reflects the behavior at low level could show that it is more effective to monitor other metric(s) (KPIs) [29, 16, 39, 57, 59].

In order to establish the relationship between the behavior at the resource level and the performance level in terms of SLA of the distributed system, the *Performance Forecaster* is in charge of collecting the low-level metrics (low-level resource use metrics) and high-level metrics (SLA parameters) in the profiling phase to generate a model capable of mapping them. This model will be the one used in auto-scaling phase to make the estimations of the high-level metrics that will be provided to the *Decider*.

Low-level metrics are provided to the *Performance Forecaster* through a monitoring service, which periodically collects utilization metrics. In the profiling phase, this service collects a wide variety of resource metrics, which after being pre-processed and transformed will be the predictors of the predictive performance model. In the auto-scaling phase, the model is able

to predict the system's performance based on the values of the resource utilization metrics that the monitoring service delivers periodically. These performance estimations are sent to the *Decider* along with the *Workload Trend Forecaster* predictions.

4.2.4 Decider

The *Decider* is the module in charge of determining if it is necessary to trigger any scaling action. This decision is taken based on the information received from the previous modules, this is, the performance trend predicted in the prediction time horizon h and the current performance of the system estimated in that moment. In addition, it needs some configuration parameters that establish the thresholds to decide which scaling action to perform, allowing flexibility and adaptation of this solution to different distributed systems. For the sake of clarity, this module is explained in more detail in Section 4.3.3, where the specific implementation of this module for a particular distributed system is explained.

4.3 FLAS for E-SilboPS

In recent years we have seen the increasing importance of publish-subscribe systems as a consequence of the strong adoption of event-driven architectures, where these systems are the cornerstone since they are in charge of sending the information asynchronously in the form of events [54, 55]. Compared to topic-based systems, content-based publish-subscribe systems allow subscribers to indicate their interests through predicates in a multi-dimensional system, which significantly reduces the processing of notifications by end users. To achieve this, they are usually implemented as distributed systems matching the incoming notifications to the stored subscriptions by determining which subscribers should receive each of the incoming notifications based on the interests described in each subscription.

This section describes how FLAS integrates with a high performance distributed system such as E-SilboPS. As mentioned, E-SilboPS has been chosen to validate FLAS due to its greater complexity in its scaling actions that supports transparent, publisher-wise dynamic state repartitioning without client disconnection and with minimal notification delivery interruption

for subscribers. This functionality makes the scaling time dependent on both the input workload and the current internal state of the system, which is a major challenge for the evaluation of an auto-scaling system such as FLAS since the estimation of the scaling time is quite variable.

More specifically, E-SilboPS was conceived as a content-based publish-subscribe middleware specially designed to be elastic [53, 44, 52]. It is a distributed system composed by four layers of operators (Connection Point, Access Point, Matcher and Exit Point) forming a directed acyclic graph (DAG). Each of these operators can have a different number of instances, which can be increased or decreased independently by means of horizontal scaling operations (scale-out/in). The scaling algorithm allows dynamic distribution without disconnecting clients and with minimal interruption of the notification service.

It is important to emphasize that the prediction models are architecture agnostic, so it is a generic solution, since the implementation can differ as long as the defined API is respected. Nevertheless, the following sections describe the specific implementations that have shown good results for the case study exposed, as it can be seen from the analysis of the results of this evaluation.

4.3.1 Scaling Time Forecaster and Workload Trend Forecaster implementation

The *Scaling Time Forecaster* module is responsible for predicting the scaling time (T'_{sa}) based on the workload at a given instant, $T'_{sa}(workload)$. More specifically, as the load of content-based publish-subscribe systems is determined by the ratio of notifications per unit of time and stored subscriptions, N and S respectively, then the best possible function is pursued to calculate the time of a scaling action based on these parameters, $T'_{sa}(N, S)$. For the generation of the predictive model, in the profiling phase, the scaling times of several scaling actions with different workloads have been collected, in order to obtain a dataset that reflects the different scaling situations. Scaling actions during this phase are triggered in a reactive manner using different threshold-based rules on workload. With this training dataset, a linear regression model has been built that allows in the auto-scaling phase

to determine the time of a scaling action based on the current workload. With this training dataset, a linear regression model has been built that allows in the auto-scaling phase to determine the time of a scaling action based on the current workload.

The *Workload Trend Forecaster* is responsible for predicting the performance trend over a future prediction time horizon h. For the sake of simplicity and clarity, the focus has been on response time as an SLA performance metric. In this case, the trend of performance is translated into the trend of response time expressed as the first order derivative of response time with respect to the time $\frac{\delta RT}{\delta t}$. In the profiling phase, time series of the response time are collected (top of Figure 4.3). A positive value of $\frac{\delta RT}{\delta t}$ will indicate an increasing trend and a negative value will indicate a decreasing trend with a more or less pronounced slope depending on the absolute value of the prediction. In this way, it is not directly forecast the response time or the workload, but rather the trend of the response time, which allows to know how fast a response time that violates the SLA could be reached. To smooth this function and avoid fluctuations of the first order derivative, a Savitzky-Golay filter has been applied to the first order derivative of the data, which is a digital filter for smoothing the data, increasing its accuracy without distorting its trend (bottom of Figure 4.3). These smoothed data were used to generate various time series analysis models, after which the model with the lowest prediction error was chosen. This model is in charge of making the predictions on the trend of the response time in the future prediction time horizon h in the auto-scaling phase. To generate these models, some time series analysis techniques have been evaluated, such as ARIMA, STL decomposition with an ETS model for the seasonally adjusted data and harmonic regression. In order to test these models, a cross-validation was performed with the models, choosing the one with the lowest prediction error (4.4).

During the auto-scaling phase, the *Workload Trend Forecaster* is in charge of forecasting in t_0 the values of $\frac{\delta RT}{\delta t_i}$ in the future instants $t_i = t_0 + T'_{sa}(t_0) + i,\ \forall i \in \{0, \ldots h-1\}$ by means of the prediction model obtained in the profiling phase (Figure 4.5). In this way, a forecast of the response time trend in the h future instants after finishing a possible scaling action that started at the current instant t_0 has been obtained. These forecast values are the ones that will be sent to the *Decider*. As already mentioned, the decision of which h-value (also known as forecast window size) to

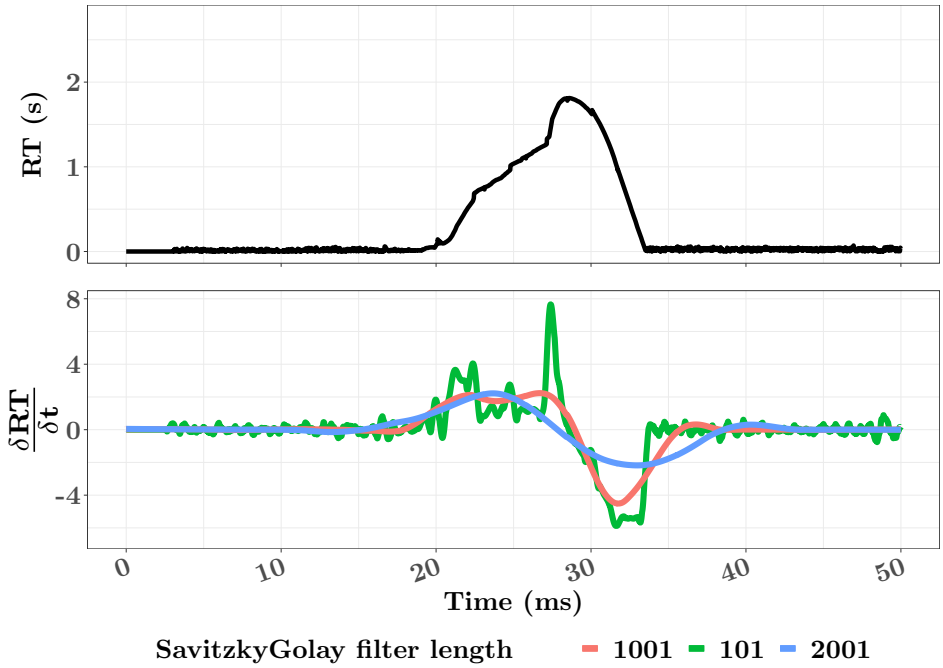


Figure 4.3: Part of a time series of the response time data before (top) and after applying a Savitzky-Golay digital filter for smoothing the data with different lengths (bottom).

take is a complex one that depends on the degree of detail desired. On the one hand, a very small h-value can cause large fluctuations in the predicted values, while a too large value can omit fluctuations that are significant and should trigger a scaling action.

4.3.2 Performance Forecaster implementation

As previously mentioned, the monitoring service is in charge of collecting performance metrics in the monitoring phase and resource usage metrics in both the profiling and auto-scaling phases to send them to the *Performance Forecaster*. This service is executed periodically every second (the period is configurable) and collects a wide variety of resource usage metrics. For this implementation it has been used the *dstat*[1] service, collecting more than 30 usage metrics of various resources such as processor (system, user, idle, wait, hardware interrupt, software interrupt, context switch metrics),

[1] `https://linux.die.net/man/1/dstat`

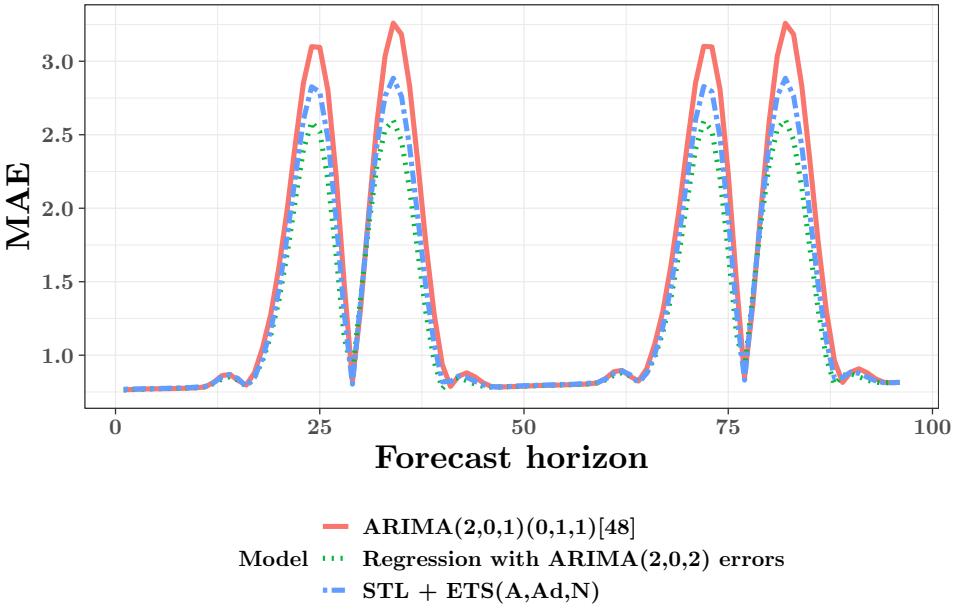


Figure 4.4: Comparison of the MAE value in the three time series analysis models tested by cross-validation for workload trend forecast.

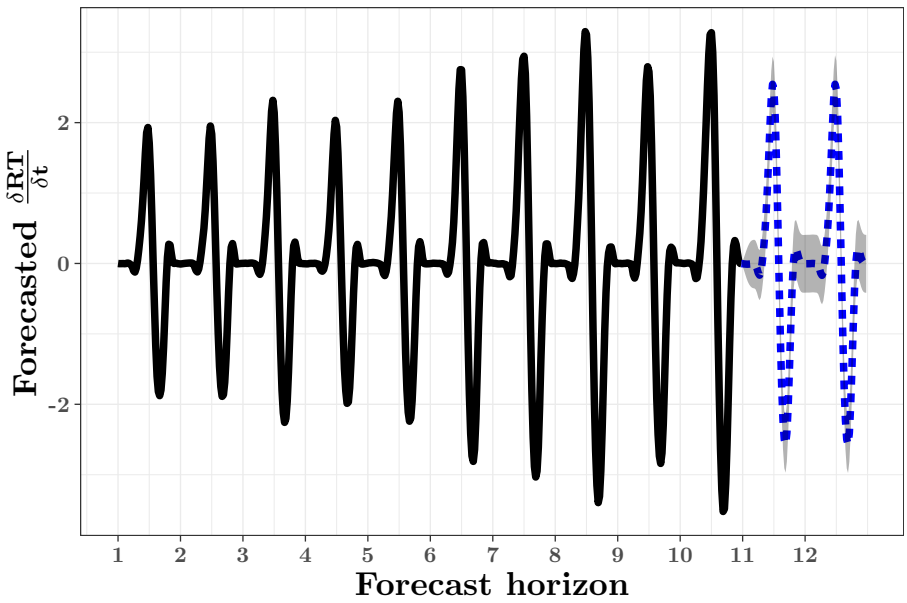


Figure 4.5: Forecasts of the values of $\frac{\delta RT}{\delta t_i}$ in a prediction time horizon using the harmonic regression model $ARIMA(2,0,2)$(dotted line).The horizontal axis represents the periods of seasonality.

memory (used, buffers, cache, free metrics), disk (read, write metrics), network (receive, send metrics), etc. Once these metrics are collected, the data is cleaned and pre-processed, adding compound metrics such as utilization percentages that will be reported to the *Performance Forecaster* in both execution phases.

The treatment of outliers is especially important, and that is why FLAS includes several mechanisms to treat them. In the cleaning and pre-processing phase of the data, the outliers are detected and removed. In addition, the values of the resource monitoring are average values of the sampling period. Finally, and as explained, FLAS scaling decisions require that appropriate conditions are maintained over time, and not at a single point.

A regression-based model has been chosen since statistical models, besides allowing to make performance predictions in the auto-scaling phase, allowing to infer and understand the relationships between low and high-level metrics, being able to detect the KPIs of the application and the resources to be scaled in each scaling action.

More specifically, statistically valid linear regression models have been created for the two main performance metrics, throughput and response time. The results show that, as in the case of throughput, the low-level metrics that contribute the most information to the model (KPIs) are the amount of free RAM, the number context switches and the network usage (received and sent).On the other hand, the main KPI of the response time is the percentage of memory use, and the number of context switches, although to a lesser extent.

Once the mapping algorithms/models are chosen and trained for a given application type and for the KPIs considered in this paper, they remain static. They could need to be changed or trained again for a different set of KPIs (e.g associated to a different application type). They are dynamic in this sense, and this is why they are parameterizable in our system by design.

To exemplify this, we carried out a series of tests using four different publish-subscribe systems: E-SilboPS[53], RabbitMQ, ActiveMQ and Be-Tree[48] to analyze the type of relationship between low-level or resource-use metrics and high-level metrics or SLA parameters. From the results of these tests it was clear that content-based publish-subscribe applications (i.e. E-SilboPS and Be-Tree) were CPU bound, and in the case of topic-based publish-subscribe (RabbitMQ and ActiveMQ) they were memory bound.

Therefore, we can consider these relationships are static for the same application type (being it content-based or topic-based publish-subscribe), but not for different application types. Even more, we cannot ensure anything beyond that, not even within applications of the same paradigm, as is the case with publish-subscribe.

In order to evaluate the predictive capacity of these two models, a k-fold cross-validation (k=10) was performed with more than 40 predictive models generated for comparison of several types such as Random Forests (RF), Artificial Neural Networks (ANN), Generalized Additive Models (GAM) or Generalized Linear Models (GLM). In a very summarized way, due to the lack of space, Figure 4.6 shows the results of the 10-fold cross-validation (R^2 and MAE) comparing the FLAS predictive models for response time and for throughput with other types of predictive models (i.e. the model with the best results has been chosen as the representative model of each category).It can be seen how FLAS models have a very good predictive capacity (R^2) with a very low prediction error (MAE).

4.3.3 Decider

The *Decider* is the module in charge of gathering all the information from the previous modules to decide if any scaling action should be triggered, and if so, to decide the specific values of each of the scaling dimensions. This implementation of the *Decider* exploits the benefits of both approaches, predictive and reactive, since it initially checks the future predictions of the performance trend in order to take a decision in advance (proactive), but it also checks the current values of the estimated performance and compares them with some thresholds (reactive) as a contingency plan against possible failures of the predictive model.

The *Decider*, like the rest of modules in this implementation, is a service that runs periodically every second and executes the algorithm described in Algorithm 1. As can be seen, the main function of this module receives as parameters the current instant (t_0), the workload, a vector with the response time estimates in the moments prior to t_0 (RT'), and the Decider configuration, which contains a series of adjustable parameters of the *Decider* (i.e. h, $reactW$, $incTrendTH$, $decTrendTH$, $reactUpperTH$, $reactLowerTH$ and $majority$). The first verification it makes is that it is not currently in the cool-down time (line 1). After a scaling operation, a cool-down time is

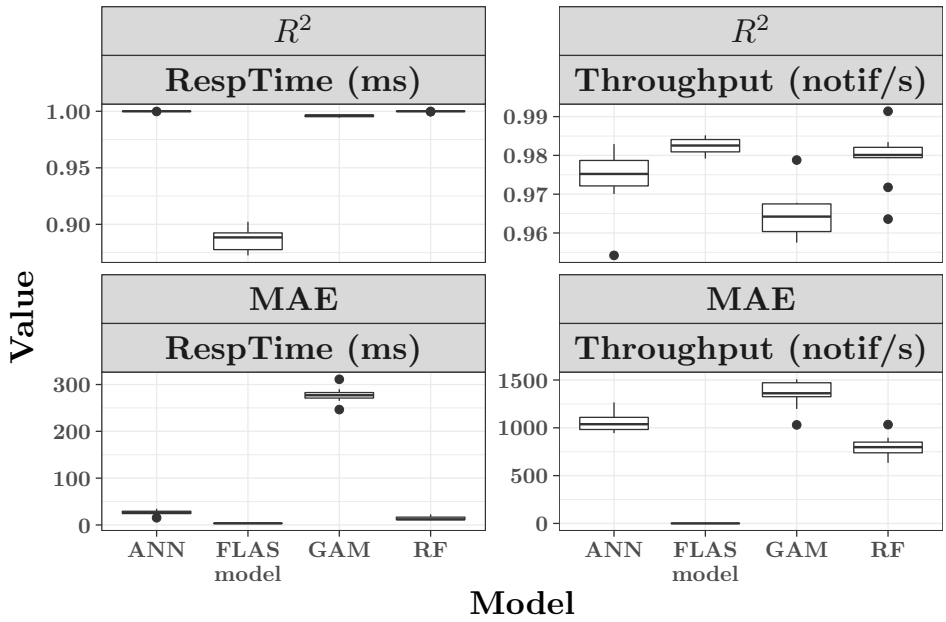


Figure 4.6: R^2 and MAE values obtained from the 10-fold cross-validation performed with more than 40 predictive models of different types for comparison with the predictive models implemented in FLAS, both for response time and throughput (i.e. the model with the best results has been chosen as representative of its category, which is shown). The different categories of models are: Artificial Neural Networks (ANN), Generalized Additive Models (GAM) and Random Forests (RF).

required to allow the system to stabilize and not trigger successive scaling actions continuously. The time of a possible scaling action (T'_{sa}) is then predicted (line 2) on the basis of the load (rate of incoming notifications per second and number of stored subscriptions). The response time trend prediction is obtained from the *Workload Trend Forecaster* by passing the current instant (t_0), the forecast scaling time (T'_{sa}) and the value of h as an argument. As a result, a prediction vector is obtained of $\frac{\delta RT}{\delta t}$ for each of the time instants of h (line 3). In addition, the resource usage metrics obtained from the *Monitoring Service* are passed as an argument to the *Performance Forecaster* to obtain an estimate of the response time at the current instant RT'_{t_0} (lines 4 and 5) and this response time estimation is added to the vector RT' (line 6) containing the previous RT forecasts.

Having made the corresponding predictions, the algorithm then decides whether any scaling action needs to be triggered. First, it checks if the predictions of $\frac{\delta RT}{\delta t}$ vector follow an upward trend (line 8). More specifically, the $incTrend()$ function checks that at least as many predictions of the time horizon h as indicated by the $majority$ configuration parameter are above an $IncTrendTH$ defined also in the configuration. In this way, it can be verified whether, despite occasional fluctuations, the predicted values follow an increasing trend above a particular value. If the predictive condition of the scale-out is not fulfilled, the reactive condition is checked by calling the $RTAboveTH()$ function. This function checks whether the last N response time estimations (reactive window, $reactW$ configuration parameter) are above a certain threshold ($reactUpperTH$) expressed in terms of the maximum response time specified by the SLA. If either of these two conditions are met, a scale-out action is triggered, doubling the number of Matcher instances and measuring the real time it takes to complete the scaling action, T_{sa}(line 9). After any scaling action, the cool-down time is activated (calculated as a function of the T_{sa} time previously measured) in which no scaling action can be performed (line 10). In addition, the response time estimation vector is cleaned so that the reactive condition can be reassessed (line 11). Similarly, the conditions for carrying out a scale-in action are assessed using the respective thresholds of the configuration (lines 15 to 20).

For the sake of clarity, some implementation details such as synchronization between scaling operations have been omitted. This case reflects the horizontal scaling of the matchers (CPU-bound operator) which is the most complex case, since the scaling of the rest of operators is trivial as it has no state [53].

Algorithm 1: *Decider* auto-scaling algorithm.

```
Input: t_0, workload, RT', h, reactWindow, incTrendTH,
       decTrendTH, reactUpperTH, reactLowerTH, majority
1  if coolDown == 0 then
2      T'_sa ← forecastT(workload.N, workload.S);
3      δRT/δt ← forecastRTTrend(t_0, T'_sa, h);
4      lowLevelMetrics ← monitoringService(t_0);
5      RT'_t0 ← estimateRT(lowLevelMetrics);
6      RT'.add(RT'_t0);
7
       // Scale-out evaluation
8      if incTrend(δRT/δt, incTrendTH, majotiry)
        || RTAboveTH(RT', reactUpperTH, reactW) then
9          T_sa ← startScaleOut();
10         coolDown ← getCoolDownTime(T_sa);
11         RT'.clear();
12         return
13     end
14
       // Scale-in evaluation
15     if decTrend(δRT/δt, decTrendTH, majotiry)
        || RTBelowTH(RT', reactLowerTH, reactW) then
16         T_sa ← startScaleIn();
17         coolDown ← getCoolDownTime(T_sa);
18         RT'.clear();
19         return
20     end
21 else
22     coolDown--;
23 end
```

As seen, the reactive FLAS approach does not use the real response time (RT) metric, but the response time estimated by the *Performance Forecaster* (i.e. RT'). The approach $RT'(t) \simeq RT(t)$ for all t instants allows that during the auto-scaling phase the application does not have to be monitored and therefore it is not necessary to instrumentalize the application in this phase, which makes FLAS a less invasive solution and reduces the monitoring overhead. However, it has been concluded that this approach is valid since the relative error of the response time estimation is limited to relatively low values for approximately 98% of the estimations (percentile 99), that is, values outside this range can be considered neither statistically frequent nor relevant, as shown in Figure 4.7. Although the relative error may seem high, it should be noted that the domain of response time estimation is very large (from values close to 0 to tens of thousands of milliseconds or more). In addition, the model makes its estimation based on a snapshot of the resource usage provided by the low-level metrics, which may cause that at a given moment there is a measurement of a punctual peak usage due to their large variability, which causes an estimation far above the real response time. However, it has been demonstrated that these cases are statistically not frequent and irrelevant (Figure 4.7).

4.4 Experimental evaluation

This section presents the evaluation of FLAS as an auto-scaling system for a distributed content-based publish-subscribe system such as E-SilboPS. In addition, the results of this evaluation are analyzed, showing how FLAS allows to minimize the time of violation of the performance SLA in different situations.

Unfortunately, there are no real public workloads available due to privacy concerns and commercial interests, which often hinders the validation of content-based publish-subscribe systems. Some works have been done as in [55], where the authors describe a possible solution, but it is not available for use. For this reason, several test cases with synthetic workloads have been generated for this evaluation. These test cases recreate several typical and expected functioning scenarios of the proposed solution (i.e. test cases 1 to 3). However, following the Boundary-Value Analysis (BVA) test methodology, other test cases are proposed that recreate some of the worst possible scenarios (i.e. test cases 4 and 5), thus complementing the evaluation made

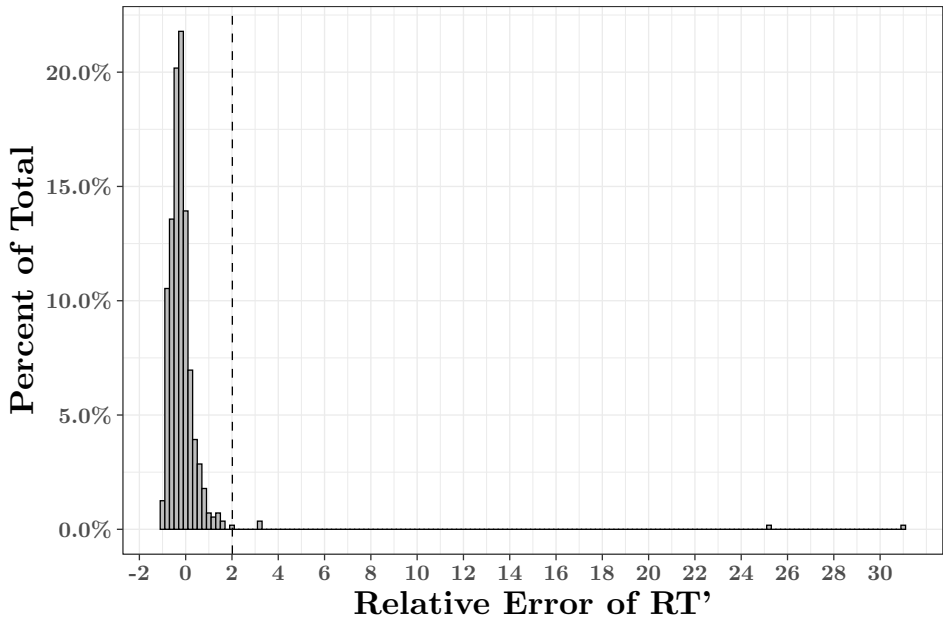


Figure 4.7: Relative frequency histogram and standardized density function of the relative error of response time estimation. The vast majority of the response time estimation error is constrained to low values (99 percentile shown by the vertical dashed line), and therefore the $RT'(t) \simeq RT(t)$ approach is used for reactive scaling.

and showing how this solution works even in the worst possible cases, which are difficult to present in real contexts. The following subsections present and explain the different test cases proposed and the last subsection analyses the quantitative results obtained after multiple executions of the different test cases.

The experiment has been ran on an Intel Core i7-4790K 4.00GHz and 16 GB of RAM running Linux 5.3.0-28-generic and openjdk 11.0.7. All test cases have been performed with an SLA that imposes a maximum response time of 1 second. Regarding the workload, the notifications are specially designed to match at least one subscription of those processed by E-SilboPS, which is a worst case scenario, since in a real scenario not all the notifications match at least one subscription. The publication rate of these notifications is constant in all test cases with a value of 10k notifications/s and the subscription dispatch rate is the one that varies throughout the test cases, since E-SilboPS is specially designed to scale in the number of subscriptions. This type of workload is typical of applications in contexts where the number of subscriptions is much higher than the number of notifications, such as stock market applications, or more recently, centralized applications for the notification and tracking of COVID-19 patients. Furthermore, all test cases start with a minimum configuration of 1-1-1 (1 Access/Connection Point, 1 Matcher and 1 Exit Point). In addition, the decider configuration parameters used in all the test cases have been the following:

- $h = 4$
- $incTrendTH = 0.9$
- $decTrendTH = -0.9$
- $reactUpperTH = 750ms$ (75% of the maximum RT allowed by the SLA)
- $reactLowerTH = 10ms$ (1% of the maximum RT allowed by the SLA)
- $reactW = 2$ (2 estimations must exceed the *ReactiveUpperThreshold* value to trigger a scaling action)

These specific values of the configuration parameters have been obtained empirically and have demonstrated the best results for this particular case. All the graphs shown in this section have been generated with these configuration parameters that have been empirically demonstrated to be the ones

that produce the least number of SLA violations for this set of test cases. However, as can be seen in Section 4.4.6, other values of these parameters can generate different scaling decisions. More specifically, Section 4.4.6 compares the quantitative results of some of the alternative configurations tested and their comparison with the chosen configuration.

4.4.1 Test Case 1: Stationary peak

In this first test case there is a huge increase and later decrease in the number of subscriptions (up to 80k subs/s) in a time interval of a few seconds (peak), as shown by the dotted line in Figure 4.8. This peak occurs seasonally every certain period of time, representing a massive sending of subscriptions and subsequent unsubscriptions as a result, for example, of interest in an event that is repeated seasonally with the same period and that after some time loses interest (e.g. shopping channels with discount notifications before Christmas or the Black Friday, traffic news channels before holiday periods, etc). This situation, although infrequent, could cause a saturation that would degrade the service of other end users using the same E-SilboPS instance.

In this test case both scale-out and scale-in actions have been taken proactively due to the predictions of the *Workload Trend Forecaster*. As can be seen in Figure 4.8, the scale-out action is triggered before the response time grows exponentially, as a consequence of the forecast performance trend. Once the scale-out action is completed, it goes from a 1-1-1 topology to 1-2-1, doubling the number of Matcher instances, which doubles the processing capacity of the workload and reduces the response time. Later, when there is a forecast in which the trend is decreasing during h, the scale-in action is triggered, which reduces the Matcher instances to save resources, thus returning to a 1-1-1 topology. By means of the scale-out action, the maximum value of 1 second response time indicated by the SLA is not reached as in the case it would not have scaled (dashed line). Moreover, when the peak has passed and the processing demand is lower, the system releases the unnecessary resources by performing a scale-in action.

As shown in Figure 4.8, the time intervals between the vertical lines and the stroke and point lines indicates the forecast scaling time (T'_{so} and T'_{si} for the scale-out and scale-in respectively) and the time intervals between the two vertical lines and light shading indicates the actual time of the

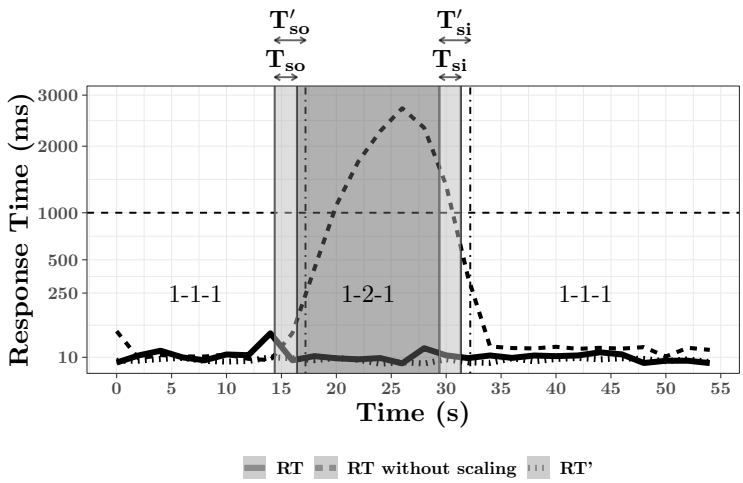


Figure 4.8: Test case of a stationary peak of subscriptions that FLAS is able to cope with by means of a predictive scale-out before the peak is reached and a scale-in also predictive when the workload is going to decrease permanently to save resources. The values of T'_{so} and T'_{si} indicate the forecast scaling time for the scale-out and scale-in actions respectively, while the values T_{so} and T_{si} indicate the actual times that these scaling actions have taken.

scaling action (T_{so} and T_{si}). Both scaling actions end slightly earlier than predicted. In addition, due to the lack of a greater resolution cannot be seen how there is a slight increase in the response time when the scaling actions are initiated, this is because the scaling algorithm of the Matcher in E-SilboPS requires to distribute the internal state between the total Matcher instances of the new configuration. This dynamic repartitioning requires sending the corresponding status to each instance and its processing, which implies an overhead to the processing of the workload that continues to be received during the scaling operations.

Finally, the dotted line represents the estimation of the response time in each one of those instants (RT') that as can be seen is quite approximate to the value of the real response time measured in those instants (RT). A figure similar to this (Figure 4.8) can be observed in each of the following subsections where the test cases and their results are explained, and therefore, the way to interpret it is the same as explained here.

4.4.2 Test Case 2: Non-stationary peak

In this test case, there is a workload peak similar to the previous test case, but in a non-stationary manner. This change cancels out the predictive part of FLAS, since it is not able to predict when that peak will occur. However, FLAS is able to make the corresponding scaling decisions in a reactive manner to ensure compliance with the SLA. In a real context, this can occur if there is a programming error in the application using E-SilboPS, a denial of service (DoS) type attack, or simply a sudden and ephemeral massive interest in a certain entity.

Both scaling decisions (scale-out and scale-in) have been taken in a reactive manner using threshold based rules (Figure 4.9). When the RT predicted by the *Performance Forecaster* exceeds a certain threshold for a certain amount of time, the scaling action is triggered. Several executions have been performed with different values both of the threshold value (i.e. 75%, 80% and 90% of the maximum response time imposed by the SLA in the case of scale-out), and of the amount of time that should be exceeded to trigger the scaling action (i.e. reactive window size of 1, 2 and 3 consecutive estimations). These results have allowed to empirically obtain the best FLAS configuration as detailed in Section 4.4.6 Quantitative analysis of results, where a quantitative comparison of all these variations can be seen.

In general, the reactive approach is less efficient than the predictive one, since it tends to incur longer periods of under-provision in the scale-out and over-provision in the scale-in as a result of its lack of anticipation. The higher the threshold value of the reactive scaling rules, the more these differences are accentuated. On the other hand, adding the restriction of exceeding the threshold value for more than one estimation prevents the triggering of scaling actions as a consequence of a false positive of the *Performance Forecaster*, but once again, it delays the decision making, and if it is not a false positive it could mean a higher SLA violation.

4.4.3 Test Case 3: Steady increase

This test case reflects a scenario in which subscriptions increase slowly but steadily over time (+500 subscriptions/s) until more than 100k subscriptions are stored. In a real context, this small but constant increase in load may correspond to a certain fashion or trend that causes subscriptions related to

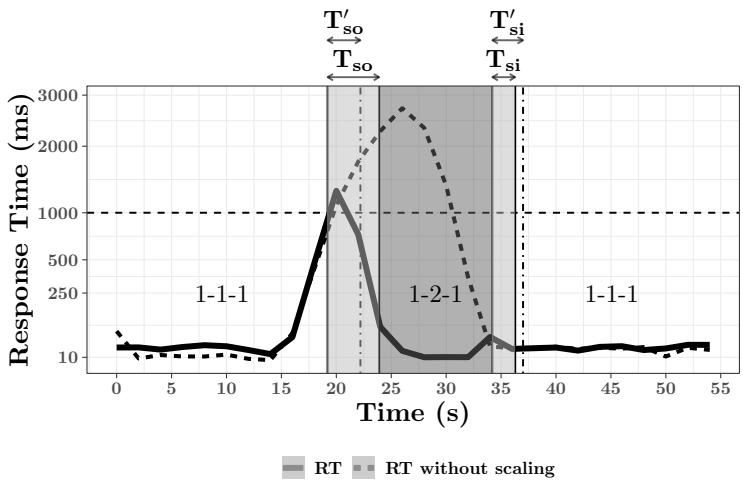


Figure 4.9: Test case similar to that shown in Section 4.4.1 but using a reactive rather than predictive approach because it's non-stationary. A threshold of 75% of the maximum RT established by the SLA (i.e. 750 ms) has been used. Compared to Figure 4.8 it can be seen how the scaling action occurs later and therefore the response time increases.

a certain entity to gradually increase as this entity becomes more popular, which means that large peaks are not reflected as in the previous cases (Figure 4.10). An example of this behavior can be social network profiles whose interest is slowly but steadily increasing over time or subscriptions to a platform that is gradually becoming popular.

This type of situation means that the slope predicted by the *Workload Trend Forecaster* is not sharp enough to trigger a scale-out in the first few moments, and when this slope is clearly accentuated, the maximum response time value imposed by the SLA is already being exceeded by a long margin. In this situation, a predictive scale-out is not possible because the scaling decision would be very late, once the SLA is violated which would boost even more the response time by adding the overhead of the scale-out process. Instead, the reactive approach already discussed is able to react earlier in these cases, especially with low threshold values (i.e. 75% of the maximum response time set by the SLA). Moreover, even if the reactive approach determines a little late that a scale-out should be performed, in this context it is not so harmful because the increase of the response time is quite slow and therefore it will take longer to violate the maximum response time imposed by the SLA.

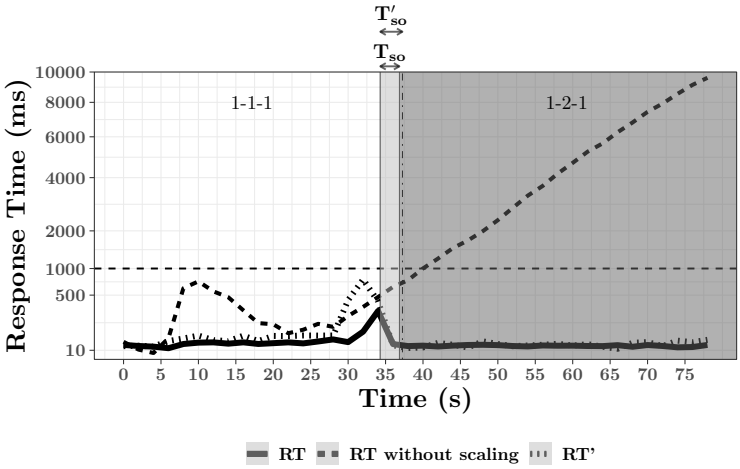


Figure 4.10: A test case in which the subscription rate increases slowly, which causes the response time to increase gradually and not generate a peak. In this case the predictive approach of FLAS is not able to predict sufficiently in advance the scale-out action, but the reactive approach allows FLAS to trigger the scale-out action without violating the SLA.

4.4.4 Test Case 4: Isolated and close to SLA limit workload peak

This test case is very similar to the first one, but unlike the first one, the peak of the workload occurs in a much shorter instant of time, going from a workload of 30k subscriptions/s to 120k subscriptions/s, to return the next instant to 30k subscriptions/s (Figure 4.11). As previously mentioned, this test case and next one represents a worst case scenarios following a BVA analysis, and therefore, it will be very difficult to find these type of case in a real context, but it allows to evaluate FLAS even in the worst imaginable conditions.

Although this test case seems a little artificial, it serves to illustrate how the system is capable of adequately managing peaks so punctual that it is not worth triggering a scaling action, since the duration of the scale-out operation itself could be double or triple the duration of the peak, in addition the overhead introduced by the scaling action itself, could make the response time at that instant even greater. In this case, it is not efficient to trigger a scale-out and scale-in operation and FLAS is able to detect it through the condition that the trend has to be maintained during a certain number of the predictions of the prediction time horizon h. That is, the

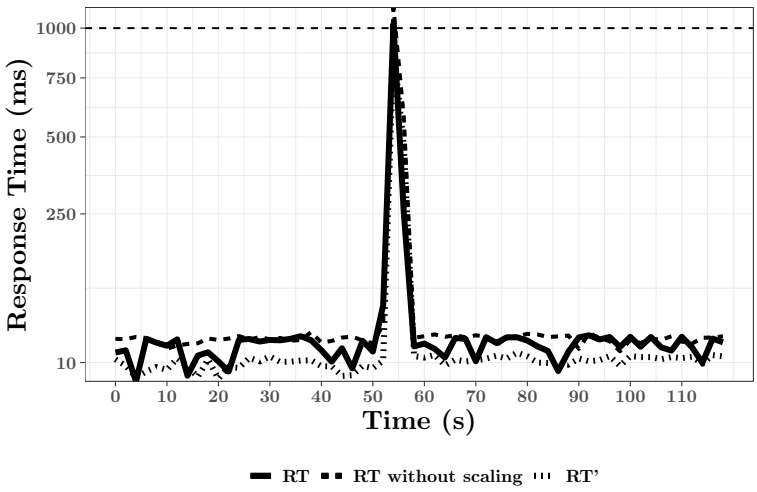


Figure 4.11: Test case with a very punctual peak workload in which FLAS determines that it is not efficient to trigger scaling actions (neither predictively nor reactively), therefore it has a 1-1-1 configuration during the whole test case.

Workload Trend Forecaster detects the peak, but since it does not show a trend maintained over time, the *Decider* resolves that it should not trigger any scaling action. Moreover, in this case, the reactive window has to be large enough not to trigger a scale-out action as a result of the peak in response time estimation when the workload peak occurs. As shown in Figure 4.11, only the maximum response time SLA of one second is violated during a very short period of time (approximately 1 second).

4.4.5 Test Case 5: Consecutive and close to SLA limit workload peaks

This test case is based on the previous one, but instead of being a single punctual peak, several peaks occur in a row during a certain time interval. This behavior, as in the previous cases, is repeated periodically. In this case, it would be considered advisable to perform a scale-out action, since otherwise the maximum response time imposed by the SLA would be violated for quite some time as shown in Figure 4.12.

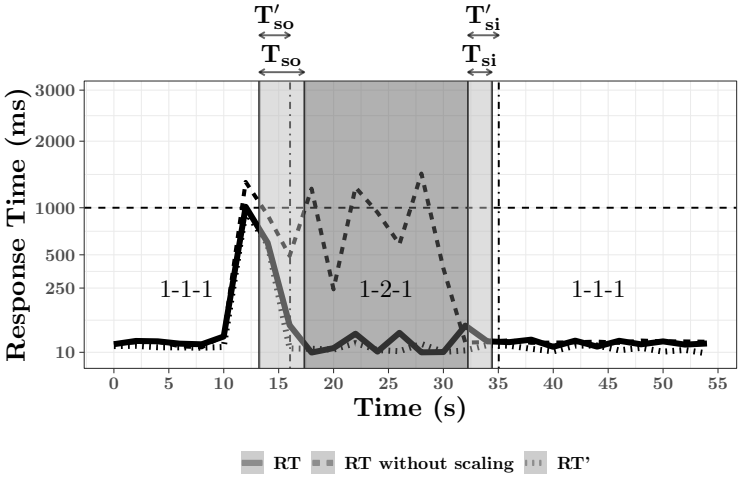


Figure 4.12: Test case with continuous punctual load peaks. FLAS is able to perform a scale-out to reactively to cope with the increased workload. The scale-in is predictive, since it predicts a continuous decrease of the response time. Only one estimation above the reactive threshold can be appreciated but this is due to the aggregation of the data for the sake of clarity.

However, as in the previous test case, it is not possible to give a predictive response, as the upward trend is not stable over time, but fluctuates continuously. However, FLAS is still able to determine that it must perform a scale-out since it detects that the estimated response time is above a certain threshold (75% of the maximum established by the SLA, in this case, 750 ms) for two consecutive estimations. In this way, as shown in Figure 4.11, a reactive scale-out is produced. Some SLA violations can still occur in one or two seconds, but it is far less than if no scaling action were taken. Finally, scale-in is predictive, since it can detect a clear trend of decreasing response time once the succession of workload peaks is over.

4.4.6 Quantitative analysis of results

This section presents and discusses the results of the evaluation of the case studies described above (Tables 4.1 and 4.2). The results presented here are average values obtained from 20 executions of each test case. The default setting for the reactive scaling is the one mentioned above (75% of the maximum response time imposed by the SLA as a threshold for the scale-out and reactive window size of 2). It is later demonstrated that this is the most efficient configuration for this case.

When interpreting Tables 4.1 and 4.2, the distinction between NA and 0.00 values must be taken into account. NA values indicate that there is no data for that case, since it does not apply that measure for that particular case, while the other case indicates that the value of that metric is 0.00.

For each test case a single version has been executed without any scaling action in order to obtain a baseline of the response time behavior for that test case. This baseline has been used to determine the Demand Point of each scaling action (DP_{sa}). In each execution, the difference between a demand point for an scaling action and the completion of the corresponding scaling action (RP_{sa}), i.e. $DP_{sa} - RP_{sa}$, has been calculated. Positive values of this difference indicate over-provisioning in the case of a scale-out action and under-provisioning in the case of a scale-in action. On the other hand, negative values of this difference indicate under-provisioning and over-provisioning if the scaling action is scale-out and scale-in respectively. In this way, the percentage of time in over-provisioning and under-provisioning can be calculated as the sum of all over-provisioning and under-provisioning times for each scaling action divided by the total runtime.

Finally, Table 4.1 also shows the percentage of time that the SLA has been violated, that is, the percentage of time that the RT has been greater than 1 second. Table 4.2 shows the scale action times calculated as $\sum |TP_{sa} - RP_{sa}|$ for all scale-in and scale-out actions respectively. In addition, for each type of scaling action it shows the relative error of the prediction of T, calculated as $\sum \frac{T_{forecast} - T}{T}$. The sign of this relative prediction error indicates if the predicted T value is greater than the real value of the scaling time (positive) or if, on the contrary, the predicted scaling time is less than the real time of the scaling action (negative).

The Test case 5 has a very low SLA maximum response time violation rate (0.36%). The over-provisioning of this test case is mainly due to the predictive scale-in actions and the under-provisioning to the reactive scale-out actions, being the latter the ones that cause these small SLA violations. In turn, the Test case 3 does not violate the maximum response time of the SLA at any time due to the early reactive scaling as seen in the percentage of over-provisioning time, the slow increase in response time of this test case and the null value of under-provisioning.

The Test case 4 is the only case where no scaling action is taken and yet the SLA violation percentage is very low (less than 1%). It should be remembered that in this case, since the peak is so small, it is not profitable to perform a scale-out, since the overhead of the scale-out action would produce an increase in response time that would result in a larger SLA violation than if this scale-out action is not performed. Due to the reactive scaling window, FLAS is able to detect this situation and conclude that it is not necessary to trigger the scale-out action.

For the case of the Non-stationary peak test (Test case 2) several reactive versions have been test to be able to be compared with the predictive one (Test case 1) and show how the configuration values of the *Decider* have been fixed empirically. Each version of Test case 2 is identified as X-Y, where X is the scale-out threshold (as a percentage of the maximum RT defined by the SLA), and Y is the size of the reactive window. Therefore, the higher the X and Y values, the later the scaling action will be triggered. As can be seen, the predictive version (Test case 1) does not violate the SLA at any time, since the under-provisioning is very low and the over-provisioning is somewhat higher (due to the prediction error). Looking at the reactive versions, for the same value of the scaling threshold, the total under-provisioning time increases as the size of the reactive scaling window increases, and therefore, the time in violation of the SLA increases. In view of this data, the reactive configuration 75-1 and 75-2 are the ones that generate less SLA violations, and specifically, the implementation of configuration 75-2 has been chosen in the *Decider* since it allows to efficiently treat cases such as Test case 4, while with configuration 75-1 it would not be possible (although with configuration 75-2 SLA violations slightly increase compared to 75-1, they still do not exceed 1% of the time).

Test case	Reactive Window Size	Total over-prov. time (%)	Total under-prov. time (%)	Time in SLA violation (%)
Test case 5	2	10.16	7.39	0.36
Test case 3	2	3.98	0.00	0.00
Test case 4	2	NA	NA	0.61
Test case 1	2	7.66	0.84	0.00
Test case 2 ($reactUpperTH = 750ms$)	1	7.65	0.84	0.18
	2	7.61	2.16	0.78
	3	9.18	8.13	6.14
Test case 2 ($reactUpperTH = 800ms$)	1	8.87	3.20	1.56
	2	7.02	4.92	5.21
	3	12.10	11.02	5.71
Test case 2 ($reactUpperTH = 900ms$)	1	9.00	4.72	2.27
	2	8.08	4.45	2.73
	3	10.07	9.54	7.79

Table 4.1: Results of the percentage of under-provisioning, over-provisioning and violation of the maximum response time imposed by the SLA for the different test cases. In the case of the Stationary Peak test case, the predictive and reactive X-Y versions are compared (being X the scale-out threshold and Y the size of the reactive window respectively).

Test case	Reactive Window Size	avg T s-in (s)	avg T s-out (s)	R.E. of T' in s-in (%)	R.E. of T' in s-out (%)
Test case 5	2	1.99	2.88	40.71	-2.84
Test case 3	2	NA	2.93	NA	2.86
Test case 4	2	NA	NA	NA	NA
Test case 1	2	1.97	2.11	42.31	36.81
Test case 2 ($reactUpperTH = 750ms$)	1	1.99	2.92	41.05	5.31
	2	2.29	3.19	28.87	-0.57
	3	2.02	5.39	38.51	-41.03
Test case 2 ($reactUpperTH = 800ms$)	1	2.06	3.27	36.07	0.11
	2	1.41	4.32	99.07	-10.40
	3	1.99	5.33	41.13	-38.50
Test case 2 ($reactUpperTH = 900ms$)	1	1.99	3.79	41.07	-12.70
	2	1.97	3.89	41.91	-16.91
	3	2.14	5.80	33.12	-43.81

Table 4.2: Results of the scaling times (T) and the error in the prediction of these times for the scale-out and scale-in actions of the different test cases.

With respect to Table 4.2, it can be seen how the scale-out time (*avg T scale-out*) increases with the size of the reactive window and the scale-out threshold. This is due to the fact that, as mentioned, with higher values of the reactive window and the scale-out threshold, the scale-out action is triggered later, and therefore the E-SilboPS load is higher and in saturation, which increases the scale-out time. This same phenomenon can be observed in the relative error of prediction of T in the scale-out (last column), since it is observed that for reactive cases, the error goes from positive values (i.e. $T' > T$), to negative values (i.e. $T' < T$).

In Table 4.2 it can be seen how for the scaling actions triggered in a predictive way, the relative error of prediction of T oscillates between approximately 30% and 40%. This *a priori* could be translated in a bad accuracy of the predictive model of the *Scaling Time Forecaster*, but it is necessary to analyze it in this context. The scaling horizon h of the *Workload Trend Forecaster* depended on the prediction of T, so if the predicted T is always slightly higher, this causes that h is also larger, and therefore the scaling decisions are taken slightly in advance of what would be the optimum. This leads to a slight increase in the percentage of time in over-provisioning in those same cases, as can be seen in Table 4.1. However, it must be taken into account that in this context the objective of FLAS is to minimize performance SLA violations, and therefore, that the scaling actions are slightly advanced (as a consequence of a predicted T greater than the real T) entails a higher cost derived from a higher over-provisioning. However, it should be borne in mind that a slightly delayed scaling action (predicted T less than actual T) usually entails an increase in SLA violations (as seen in the results of Table 4.1), which is a much greater penalty than a slight over-provisioning, and therefore, in this context it seems cautious to have that margin of error.

Following the evaluation methodology of similar works[29], Figure 4.13 compares FLAS with each one of auto-scaling techniques that compose it (reactive and predictive) and with a reactive technique using threshold-based rules. The latter has been chosen because it is the most widely used auto-scaler in practice, since it is the one offered by the main Cloud providers. In this case, it has been configured so that the scale-out is triggered if CPU usage is above 80% during more than two monitoring periods, and

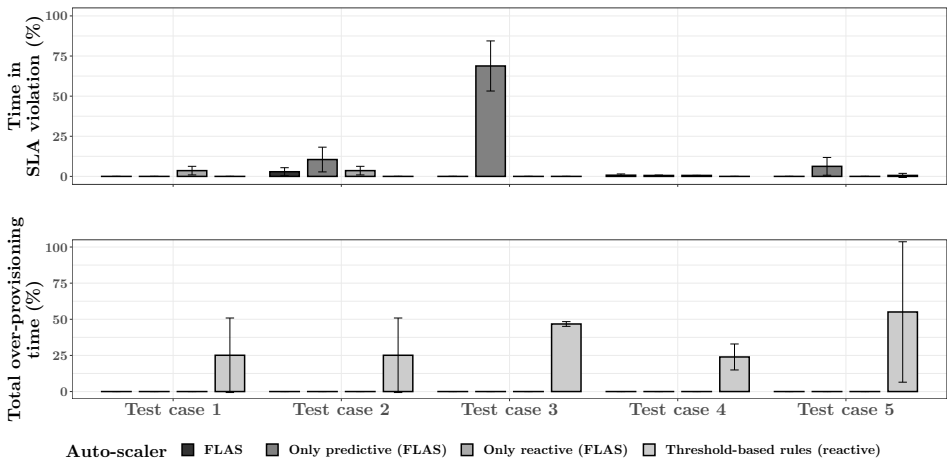


Figure 4.13: Comparison of FLAS with other auto-scaling techniques in terms of percentage of time that the SLA is violated and percentage of time in over-provisioning.

the scale-in is triggered in an analogous way with a threshold of 40%. This configuration has been chosen because it has been empirically proven to minimize SLA violations. After any scaling action of any of the auto-scalers, the same cool down time has been used.

As can be seen in Figure 4.13, the percentage of time in violation of FLAS SLAs is the lowest of the two techniques that compose it separately, that is, FLAS is capable of coordinating the two auto-scaler techniques to choose the most favorable one in any situation, even in the most extreme cases represented here. On the other hand, it can also be seen how the reactive technique that uses threshold-based rules has a percentage of SLA violation very similar to that of FLAS, and therefore better than the auto-scaling techniques of FLAS separately in some test cases. However, it achieves this with an over-provisioning much higher than FLAS and its separate parts in all test cases, which results in an increase of the economic cost and energy consumption compared to FLAS.

Finally, considering these results we can conclude that FLAS meets its objective of minimizing performance SLA violations, especially since some of the test cases presented here are the worst possible scenarios. In these circumstances (taking into account that reactive versions with configuration different from 75-2 are shown only for comparison), it is verified that the

percentage of time in which the maximum response time imposed by the SLA of 1 second is violated is less than 1% (i.e. 0.78% in the worst case), which ensures a compliance with the SLA of 99.22% of the time. Furthermore, it has been demonstrated that FLAS has a better performance than each of the auto-scaling techniques that compose it, being able to coordinate both to choose the most convenient one in each situation. When compared to other auto-scalers, FLAS has demonstrated to obtain similar results in terms of SLA compliance, but with a quite inferior over-provisioning, which results in a lower economic cost and energy savings using FLAS.

4.5 Discussion of results

As has been shown throughout this chapter, FLAS is an auto-scaler whose design focuses on compliance with the SLAs of cloud-consumers by combining both predictive (in this case using statistical prediction methods) and reactive (threshold-based rules with optimization) auto-scaling techniques. As we have seen in the previous evaluation section, FLAS is capable of achieving compliance with the SLA imposed for more than 99% of the time, due to the complementarity of both auto-scaling techniques. On the one hand, the predictions of the proactive techniques allow FLAS to anticipate the changes in the trend of the SLA parameters and to perform the necessary scaling actions in advance in order not to saturate the system and violate the SLAs. On the other hand, when the predictions of the proactive techniques fail (e.g. non-stationary performance of the SLA parameters), FLAS is still able to perform the necessary scaling decisions in order not to violate the SLAs, or at least to minimize the time of violation. All this constitutes the justification that makes it possible to answer in the affirmative the first of the research questions raised in this thesis RQ1, thus providing a solution to the first of the problems addressed in this work and explained at the beginning of this document (Problem1).

In addition, FLAS allows for the automatic detection of the resource that is the bottleneck, and which must therefore be monitored through its KPI, and when the time comes, scaled so as not to violate the SLA defined by the cloud-consumer. As has been seen in this chapter, the *Performance Forecaster* module of FLAS is in charge of carrying out precisely this task. To this end, it receives as input the values of a wide range of low-level metrics from the monitoring service and the predictive models (statistical in this

case) that allow the behavior of the high-level metrics (SLA parameters) to be predicted based on the values of the resource metrics (low-level metrics) monitored. On the one hand, the predictive statistical models identify the metrics that will act as KPIs (i.e. the amount of free RAM, the number context switches and the network usage for the throughput SLA parameter, and the percentage of memory use and the number of context switches for the response time SLA parameter) and the predictions of these models are the dynamic mappings between the different levels of metrics that provide an essential part of the knowledge to carry out the necessary scaling actions at each moment. All this allows to answer satisfactorily the second research question raised in this work (RQ2), and therefore, to solve a necessary, but not sufficient, part of the second problem addressed by this thesis (Problem2).

Chapter 5

Establishing a Set of Vendor-Neutral Cross-Provider Measurable Metrics

Cloud is about how you do computing, not where you do computing.
— Paul Maritz

On the one hand, the major cloud providers currently put the onus of implementing the mechanisms that ensure compliance with the desired service levels on cloud consumers. However, consumers do not have the required expertise.

On the other hand, any of the most important service levels demanded by cloud consumers in their SLAs (e.g. cost optimization, legal restrictions, high availability or vendor lock-in avoidance) cannot be adequately satisfied through the use of a single cloud provider. Therefore, it is necessary to use resources and services from multiple clouds. However, this poses an additional problem when mapping high-level metrics to low-level metrics, since there is no vendor-neutral set of metrics that can be monitored across multiple cloud providers [40, 37, 36]. This fosters provider lock-in and prevents consumers from benefiting from the advantages of multi-cloud environments.

This chapter presents a solution to the problem of automatically translating SLAs into objectives expressed as metrics that can be measured across multiple cloud providers, abstracting the specificities of each cloud provider. To this end, firstly, a knowledge-based system capable of automatically translating high-level SLAs defined by cloud consumers into a set of conditions expressed as vendor-neutral metrics is proposed. Secondly, it is presented a harmonized and rather generic set of vendor-neutral metrics that can be measured in multiple clouds. Each of these cross-provider metrics would then have to be mapped to the corresponding metric in each cloud provider by means of mapping functions, which is also described in this chapter. Finally, it is reported a validation of the proposed solution through two use cases (IaaS and PaaS) in a multi-cloud environment formed by leading cloud providers. This evaluation has demonstrated that, thanks to the complementarity of the two solutions, cloud consumers can automatically and transparently exploit the multi-cloud in many application domains.

More specifically, this chapter address Problems 2 and 3 offering a response to RQs 3 and 4.

5.1 Proposed solution overview

The aim of this section is to provide a brief overview of the solution explained in more detail in the rest of this chapter, focusing on the stated research questions and the synergies arising out of their joint response and also discussing issues that are out of scope.

As shown in Figure 5.1, the proposed solution focuses on the cloud consumer viewpoint. As already mentioned, cloud consumers use cloud environments (and especially multi-cloud) in order to satisfy the requirements and service levels of their applications. For this reason, it is considered that the solution should be designed based on the SLA. The SLA is where cloud consumers state these needs, which are a key long-term success factor. SLAs are composed of a set of service-level objectives or SLOs, which are SLA parameter (slap) or high-level metric conditions. These conditions are very wide ranging, including, for example, throughput greater than 30k notifications/s, all resources deployed in USA regions (due to restrictions

or data sovereignty laws), monthly cost never greater than $100, etc. This work does not deal with the formalization of SLAs and SLOs, since there are many works offering frameworks and standards valid for this purpose [32].

These high-level metric conditions (SLOs) are then translated into lower-level metric conditions (RQ3). This translation cannot be limited to static mapping, as the relationship between SLA parameter conditions and measurable metrics has to take into account numerous contextual factors, such as the application domain. The following section focuses on describing the proposed solution to this problem. In addition, it is proposed a repository of metrics that can be measured across multiple cloud providers, which does not depend on any specific cloud provider (RQ4). This vendor-neutral cross-provider measurable metrics repository is divided into IaaS, PaaS and Context metrics (e.g. geo-location, cost, or energy consumption metrics). In this case, the relationship between this vendor-neutral metric repository and the metrics exported by each cloud provider has been established through static mapping functions, as explained in more detail later. These mapping functions, which bridge the gap between the harmonized standard repository of metrics and the metrics exported by each cloud provider, will be offered by a cloud provider-specific adapter implementing a common API exported by the vendor-neutral metrics repository. These adapters are also responsible for communicating with the corresponding cloud provider. To do this, they use one of the multiple inter-cloud libraries available for various programming languages (e.g. jclouds[1] for JAVA or Apache LibCloud[2] for Python), which have been widely tested through use in multiple multi-cloud projects [38]. These libraries abstract the heterogeneity of the different cloud providers' management APIs and allow resource provisioning.

As a result, the proposed solution can ensure compliance with the service levels required by cloud consumers by abstracting the specificities between the services offered by multiple cloud providers. This provides cloud consumers with a high level of specification of their objectives, which can be evaluated in a standardized and cross-provider manner.

[1] `https://jclouds.apache.org/`
[2] `https://libcloud.apache.org/`

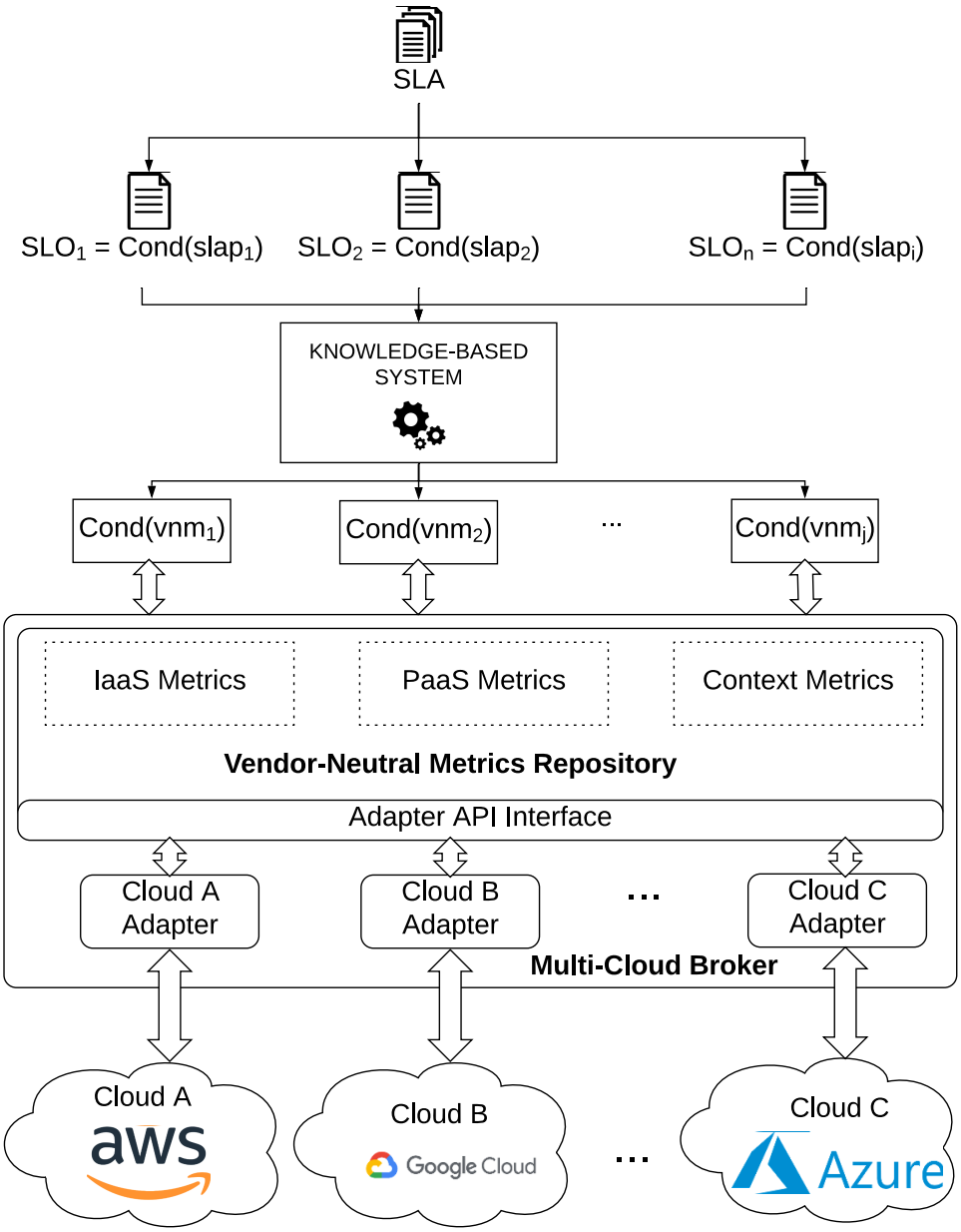


Figure 5.1: Overview of the proposed solution focused on the vision of the cloud consumer. The top section shows how the SLA and its respective SLOs are converted into a set of low-level conditions, which are expressed as vendor-neutral metrics that can be monitored in multiple-cloud environments.

There are many other challenges that a multi-cloud broker must address (e.g. provisioning, allocation and portability of resources or conditions enforcement). For the sake of clarity, they have not been included in this work, as the novelty of this research resides in the innovative solution to the research questions. However, these aspects are orthogonal to this research, which could be rounded out by integrating some of the diverse solutions to each of these challenges proposed in the literature.

5.2 From SLA to low-level conditions

This section describes the solution that it is proposed to deal with RQ3 above and the empirical grounds for adopting this solution. As previously explained, the solution to RQ3 implies offering a knowledge-based system that is capable of automatically determining the low-level metric conditions (i.e. key performance indicators or KPI) necessary to achieve a high-level metric condition in order to ensure compliance with the high-level metric objective (SLO).

The relationship between high-level objectives or SLOs and low-level objectives raises the following needs, which reflect the requirements that a solution to RQ3 must satisfy. It must:

N 1. *Capture and use expert knowledge. The translation of high-level objectives or SLOs into their corresponding low-level objectives is a difficult problem and requires the application of expert knowledge [16]. Therefore, the proposed system must be able to capture and use this expert knowledge to decompose SLOs into their respective low-level metric conditions or KPIs.*

N 2. *Be flexible and adaptive to changes. Because SLAs are defined at a very high level, there is a gap between these agreements and the underlying reality, which changes enormously and very rapidly. For example, a user wants the resources supporting the contracted cloud services to be in a particular region (e.g. Europe) in order to comply with the applicable data laws. However, the data center where these resources are hosted may, in the future, be in a country that no longer belongs to the region that complies with these regulations. For example, "The local laws that apply in the jurisdiction where the content is located are an important consideration for some customers. However, customers also need to consider whether laws in other jurisdictions may apply to them" [3]. Another example of these possible changes is the variation in the pricing policy of a given cloud provider, which may compromise the assurance of complying with a given cost-related SLO. Since these changes are unpredictable and inherent to our reality, the proposed system must be able to adapt to these changes, if not automatically, with the least possible change, favored by a design conceived to deal with this variability.*

N 3. *Offer dynamic mapping. The relationship between high-level and low-level conditions cannot be expressed by a simple static mapping, as this relationship is strongly influenced by the context and specific domain of each case. However, there are two key factors to be taken into account when establishing these relationships, which therefore must be considered by the proposed solution. The first factor is the application type, since the same SLO cannot generally be expressed by the same low-level metric conditions for different types of applications built for different purposes. For example, an application throughput SLO will be defined by different low-level metric conditions for a database-type application (whose limiting factor may be storage) than for publish/subscribe middleware (whose limiting factor may be processing capacity or bandwidth). The second factor is the application domain, which accounts for the fact that the same application can perform very differently at resource level depending on the application domain. This obviously has to be considered to ensure compliance with the SLOs defined by the cloud consumer. Figure 5.2 shows how four middleware applications, implementing the same publish/subscribe paradigm, have different resource-level performance depending on whether the model used to determine the matching between a subscription and a notification is based on a predefined topic (i.e. topic-based publish/subscribe or TBPS) or on the content of the message (i.e. content-based publish/subscribe CBPS). More specifically, two of the middlewares implemented the topic-based model (RabbitMQ[3] and ActiveMQ[4]) and the other two implemented the content-based model (E-SilboPS[53] and BE-Tree[48]). The experiment measures the throughput (notifications/s) of each of the middlewares with a publisher sending notifications at maximum rate to n previously loaded subscribers in order not to influence the matching time. This study shows that, in the case of content-based brokers (E-SilboPS and BE-Tree), the throughput saturation point is due to CPU exhaustion (i.e. they are CPU-bound applications). On the contrary, the resource that limits the throughput in the case of topic-based brokers (RabbitMQ and ActiveMQ) is memory (i.e. they are I/O-bound applications). This goes to show that systems implementing the same paradigm can perform very differently at resource level depending on the purpose for which an application is to be used. As a result, the same high-level metrics (throughput) may require different low-level metric mapping functions depending on the task KPIs, since, it is the KPIs (CPU usage and memory usage for the CBPS and TBPS middlewares, respectively) that determine the saturation point of the high-level metric, and this could lead to non-compliance with the SLO. There are*

[3] `https://www.rabbitmq.com/`
[4] `http://activemq.apache.org/`

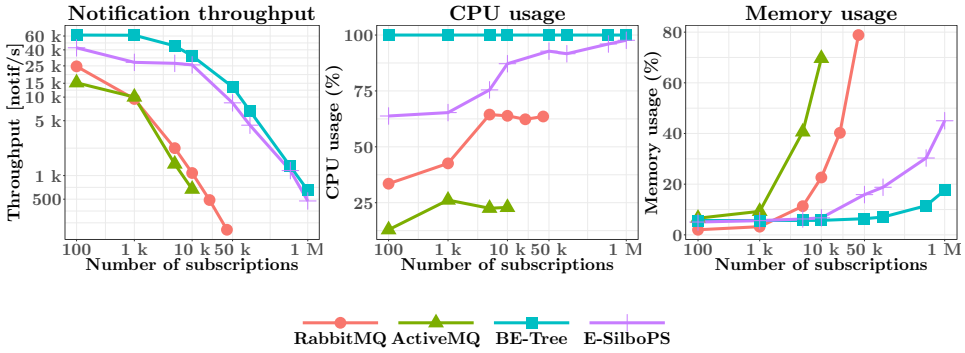


Figure 5.2: Four publish/subscribe middleware applications widely used in cloud environments, showing that the throughput saturation point in content-based publish/subscribe middlewares or CBPS (i.e. E-SilboPS and BeTree) is due to CPU exhaustion, whereas it is due, in the case of topic-based publish/subscribe or TBPS (i.e. ActiveMQ and RabbitMQ), to lack of memory.

many similar examples to confirm that the application domain influences the low-level performance of the same application, such as the difference in the resource-level performance of the same relational database type application depending on whether it is used in an OLTP (on-line transaction processing) or OLAP (on-line analytical processing) context.

In order to meet all the above requirements, it is proposed a knowledge-based system as shown in Figure 5.3. Knowledge-based systems use knowledge and inference procedures to solve problems that are so complex as to require human experience [22]. Besides, problem complexity again calls for best instead of optimal matching in most cases [37]. The system knowledge base should represent the expert knowledge required to translate SLOs into specific objectives to be met by low-level metrics. To do this, the production rules that form the knowledge base can use all the information available to provide the system with the necessary expressiveness to represent the expert knowledge (N1). Below there is an example of the production rules that populate the knowledge base. In addition, this production rule reflects part of the expert knowledge extracted from the evaluation results illustrated in Figure 5.2 for the case of CBPS middlewares.

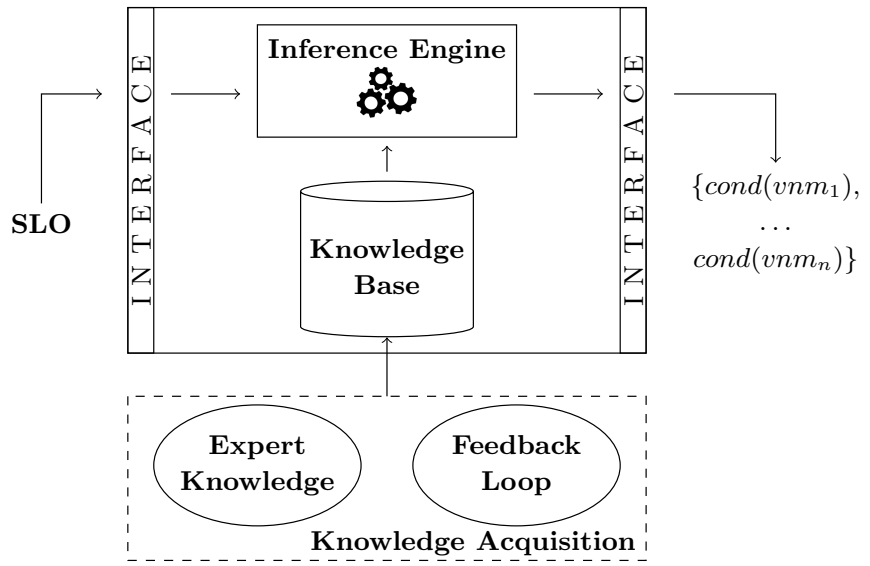


Figure 5.3: The knowledge-based system converts SLOs into a set of low-level metric conditions, using an inference engine fed by a knowledge base populated with expert knowledge and feedback from previous translations.

$$\textbf{IF } AT = CBPS \wedge throughput \geq 10k\ notif/s$$
$$\textbf{THEN } CPU_usage \leq 80$$

Section 5.4 shows the operation of this knowledge-based system in two real use cases that demonstrate how this solution satisfies all the abovementioned needs. However, the use of a knowledge-based system not only fulfills the abovementioned needs and is therefore a valid solution to the RQ3, but it also offers other advantages that add significant value. Knowledge-based systems can discover new knowledge. For instance, a correct classification and hierarchy of application types can lead to the discovery of new relationships between low-level and high-level metrics and between SLO and low-level metric objectives. In addition, this is highly flexible, as the inclusion of new rules can lead to a substantial system improvement. The source of this new knowledge acquisition is either: (1) a feedback loop that modifies or adds information to the knowledge base in the event of deviations in the SLOs or (2) experts who voluntarily expand or refine the knowledge base. Another advantage is that knowledge-based systems offer the opportunity

to aggregate knowledge, allowing all cloud consumers to benefit from that expert knowledge. For example, many of the cloud applications follow a similar architecture (e.g. back-end and front-end with a database) benefiting from the production rules that have been found to perform such translations successfully. Finally, when the application to be deployed is experimental as opposed to a solution widely used by the community whose low-level performance is quite well known, the use of a knowledge-based system can be very useful since knowledge-based systems are tolerant to and can handle uncertainty or missing data.

For the sake of clarity and simplicity, the most basic type of knowledge-based system has been used to describe the proposed solution. However, other types of knowledge-based systems incorporating neural networks and fuzzy logic problem solving approaches [9] or different types of inference engines that are possibly much more suitable for specific contexts, could be used.

5.3 Towards a set of vendor-neutral cross-provider measurable metrics

This section presents and explains the solution proposed to solve RQ4, highlighting its advantages and its justification through some examples of its operation that are extended in the real use cases presented in the evaluation section. As mentioned above, the answer to RQ4 is to offer a harmonized set of monitoring metrics, which aims to abstract the specificities between multiple cloud provider offerings, making the monitoring and enforcing of SLOs in a multi-cloud environment transparent for cloud consumers. This set of vendor-neutral and cross-provider measurable metrics will be used by the solution to RQ3 explained above to express the low-level conditions that ensure the enforcement of SLOs (see Chapter 4).

Many authors note that there is no set of metrics that can be monitored for all cloud providers, as each of them exports a different set of these metrics [36, 40, 23]. To solve this and offer a solution to RQ4, it is proposed a repository of cloud provider-agnostic metrics (vendor-neutral metrics or vnm) that can be monitored for the main cloud providers most commonly used today. This vendor-neutral metrics repository and the mapping functions that relate each of the metrics to those exported by each cloud provider provide a

solution for abstracting the specificities of each cloud provider, offering a set of provider-agnostic and measurable metrics in multi-cloud environments. In addition, the set of metrics offered by this repository is very large and diverse, ranging from IaaS metrics, through PaaS metrics, to context metrics. This variety and number of metrics perfectly complements the above solution to RQ3, since the wider the variety of metrics, the more optimal the translations of SLOs to vendor-neutral metric objectives will be. In the following sections, it is explained the proposed vendor-neutral metrics and how they can be built in order to be cross-provider measurable for IaaS, PaaS and context.

5.3.1 IaaS metrics

The evident heterogeneity of the metrics exported by the main cloud providers has led the major multi-cloud services and brokers to consider alternative solutions to this problem. The instrumentalization of IaaS resources (i.e. VM or instances), using tools like collectd[5] or Nagios[6], which export a harmonized set of metrics independent of the instance provider, has been the most adopted solution. Some cloud providers like AWS even require cloud consumers to instrumentalize their instances through the use of scripts for more detailed monitoring (since otherwise it is impossible to get metrics such as RAM usage[7]). For this reason, it is argued that, in the case of IaaS, the use of tools such as Nagios or collectd is the best possible solution to RQ4, as these tools avoid the vendor lock-in caused by these proprietary instrumentalization scripts, offering a vendor-neutral set of low-level metrics. Collectd, in particular, has been used as a tool for instrumentalizing IaaS resources, where the metrics exported by collectd are the IaaS vendor-neutral metrics of the proposed repository, some of which are shown in Table 5.1.

5.3.2 PaaS metrics

The approach proposed for IaaS is not valid in the case of PaaS, since PaaS solutions do not usually offer access to the underlying infrastructure resources, such as VM or instances. On the other hand, some multi-cloud PaaS solutions do use this approach since the actually provide a false PaaS

[5] `https://collectd.org/`
[6] `https://www.nagios.org/`
[7] `https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/mon-scripts.html`

Metric Type (plugin)	Metric
CPU	cpu-#.cpu-idle.value
	cpu-#.cpu-interrupt.value
	cpu-#.cpu-nice.value
	cpu-#.cpu-softirq.value
	cpu-#.cpu-steal.value
	cpu-#.cpu-system.value
	cpu-#.cpu-user.value
	cpu-#.cpu-wait.value
Disk	disk-<dn>.disk_merged.read
	disk-<dn>.disk_merged.write
	disk-<dn>.disk_octets.read
	disk-<dn>.disk_octets.write
	disk-<dn>.disk_ops.read
	disk-<dn>.disk_ops.write
	disk-<dn>.disk_time.read
	disk-<dn>.disk_time.write
Memory	memory.memory-buffered.value
	memory.memory-cached.value
	memory.memory-free.value
	memory.memory-used.value
Interface	interface-<int>.if_errors.rx
	interface-<int>.if_errors.tx
	interface-<int>.if_octets.rx
	interface-<int>.if_octets.tx
	interface-<int>.if_packets.rx
	interface-<int>.if_packets.tx
Other	uptime.uptime.value

Table 5.1: Some examples of the proposed IaaS vendor-neutral metrics.

by merely using IaaS services on top of the software installation services necessary to offer PaaS solutions. Nevertheless, although this solution benefits from instrumentalization (not only at resource level as before, but also by exporting a vendor-neutral set of higher level metrics), it forgoes all the

advantages of real PaaS (service optimization, lower cost, vendor lock-in avoidance and release from low-level resources management). These limitations are responsible for the small number of PaaS multi-cloud solutions and the fact that existing solutions are not real multi-cloud PaaS.

Firstly, the services offered by the three main PaaS cloud providers (i.e. Amazon Web Services, Google Cloud and Microsoft Azure) are analyzed. The conclusion of this analysis was that they mainly offer solutions for the same services. It is derived the mapping between these equivalent solutions, shown in Table 5.2, following the service mappings offered by the cloud providers themselves as documentation[8] for cloud consumers to migrate their services from other providers.

For each service, the metrics exported by the corresponding solution of each cloud provider were then analyzed (Figure 5.4). This analysis corroborated that all major cloud providers export similar monitoring metrics for each comparable service they offer, as they all measure the same magnitudes. There are slight differences between the metrics exported by each provider, the most common of which are differences in name, unit of measure (e.g. percentage or absolute value), sampled period, or composition of metrics (i.e. some cloud providers offer composite metrics that can be calculated as functions of simpler metrics exported by other cloud providers). On the one hand, it is classified mappings where the relationships between the metrics exported by the services is established directly (metrics that measure the same magnitudes) or require negligible transformation (e.g. simple operations combining several directly exported metrics) as direct (mapping column of Table 5.2). On the other hand, mappings that, due to the complexity of the service, require experts for their definition, which usually requires defining custom metrics based on very in-depth knowledge of the service, are classed as indirect. For the sake of clarity, this work focuses on direct mappings, which are in actual fact the most demanded multi-cloud services. Nevertheless, it is consider that the provision of these mappings by different service experts would be very valuable for the solution proposed in this work. This could by done by engaging cloud providers in a similar manner as they are currently involved in providing migration guides between services from different providers. Finally, a small percentage of the

[8] `https://cloud.google.com/free/docs/map-aws-google-cloud-platform`
`https://cloud.google.com/free/docs/map-azure-google-cloud-platform`
`https://docs.microsoft.com/en-us/azure/architecture/aws-professional/services`

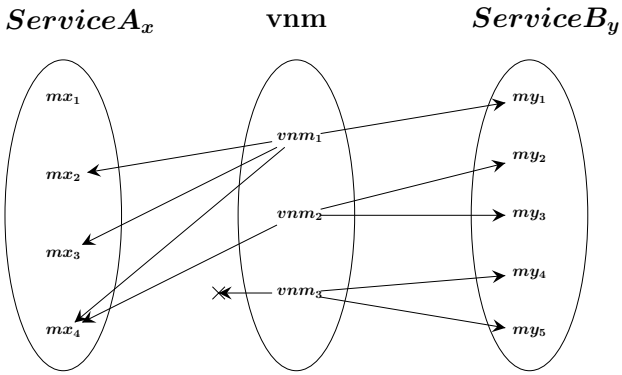


Figure 5.4: In the figure, $ServiceA_x$ and $ServiceB_y$ are the implementation of an equivalent service in a cloud provider x and y, and the metrics of these services mx_i and my_j respectively. The set of vendor-neutral metrics (vnm) is formed by all those metrics that can be measured in all the equivalent services of the different cloud providers, either directly ($vnm_1 = my_1$) or as a function of several metrics ($vnm_2 = f(my_2, my_3)$). If a vnm can not be measured in one cloud provider, this metric has not been considered (vnm_3).

solutions offered by cloud providers either do not provide a set of metrics for the respective service or implement the delivered metrics in a significantly different manner from other providers. In these cases, it is not possible to establish a relationship between the metrics of the different services, as indicated by the not possible value in the mapping column of Table 5.2.

In this study, a total of 21 Cloud services were analysed, grouped into 6 more generic categories (Compute, Network, Storage, Database, Big Data & Analytics and Application Services), offered by the 3 main cloud providers (Amazon Web Services or AWS, Google Cloud or GC and Microsoft Azure). From each of these services, the metrics exported by each of these providers have been examined, making a total of 625 metrics (AWS: 236, GC: 202 and M. Azure: 187). Finally, a repository with 70 PaaS vendor-neutral metrics for 9 equivalent cloud services is presented.

This section presents some examples of the proposed PaaS vendor-neutral mertrics, as well as the related metrics exported by the equivalent solutions of the different cloud providers and the corresponding mappings. However, the PaaS vendor-neutral metrics repository can be consulted in Appendix A. More specifically, Appendix A.0.1 collects all the metrics exported by

the main cloud providers for all the equivalent services. Appendix A.0.2 is then shown for each equivalent service: (i) the proposed vendor-neutral metrics repository, (ii) the related metrics of each cloud provider for each proposed vendor-neutral metric and (iii) the mapping function that relates the metrics of each cloud provider to each of the proposed vendor-neutral metrics in the repository.

In this section, some examples of these PaaS vendor-neutral metrics and the related metrics exported by the equivalent solutions of the different cloud providers are shown in Tables 5.3, 5.4, 5.5 and 5.6 for the block storage, object storage, PostgreSQL and MySQL database management system services, respectively. These tables show, for each service, the set of vendor-neutral metrics, their units of measurement and the metrics used for their calculation in each of the equivalent solutions of the different cloud providers. These relationships can be translated to the metrics exported by the cloud provider for a given service that are necessary to build the corresponding vendor-neutral metrics. More specifically, both metrics explicitly exported by the provider monitoring systems (e.g. Amazon Cloud Watch), and implicitly exported metrics, such as the geographical region where a service is deployed, the amount of RAM, the number of hard disks, etc, which are normally obtained from the service deployment configuration are used. For example, for the PaaS MySQL service offered by the three providers, Table 5.6 defines a vendor-neutral metric called *memory_utilization*. This metric indicates the percentage of RAM memory being used. Both Google Cloud and Microsoft Azure export this metric directly. In the case of AWS, however, this metric can be calculated using the *FreeableMemory* (explicitly exported) and *TotalMemory* (service configuration-based) metrics.

Service Category	Service	AWS	Google Cloud	Microsoft Azure	Mapping
Compute	IaaS	Elastic Compute Cloud	Compute Engine	Virtual Machines	Direct *
	PaaS Framework	Elastic Beanstalk	App Engine	App Service, Cloud Services	Indirect
	Containers	Elastic Container Service, Elastic Kubernetes Service	Container, Google Kubernetes Engine	Azure Container Service, Azure Service Fabric	Direct
	Serverless Functions	Lambda	Cloud Functions	Functions	Indirect
Network	Load Balancer Application	Elastic Load Balancer	Cloud Load Balancing	Application Gateway	Direct
	Load Balancer Networks	Elastic Load Balancer	Cloud Load Balancing	Azure Load Balancer	Direct
	Dedicated Interconnect	Direct Connect	Cloud Interconnect	ExpressRoute	Direct
	Domains and DNS	Route 53	Google Domains, Cloud DNS	Azure DNS	Not possible
	CDN	CloudFront	Cloud Content Delivery Network	Azure CDN	Not possible
Storage	Object Storage	Simple Storage Service	Cloud Storage	Azure Blob Storage	Direct
	Block Storage	Elastic Block Store	Persistent Disk	Disk Storage	Direct
	Reduced-availability Storage	S3 Standard-Infrequent Access, S3 One Zone-Infrequent Access	Cloud Storage Nearline	Azure Cool Blob Storage	Direct
	File Storage	Elastic File System	Cloud Filestore (beta)	Azure File Storage	Indirect
Database	RDBMS	Relational Database Service	Cloud SQL	SQL Database	Direct
	NoSQL: Key-value	DynamoDB	Cloud Datastore, Cloud Bigtable	Table Storage	Indirect
	NoSQL: Indexed	SimpleDB	Cloud Datastore	Cosmos DB	Not possible
Big Data & Analytics	Batch Data Processing	Elastic MapReduce	Cloud Dataproc	Batch	Direct
	Stream Data Processing	Kinesis	Cloud Dataflow	Stream Analytics	Direct
	Stream Data Ingest	Kinesis	Cloud Pub/Sub	Event Hubs, Service Bus	Indirect
	Analytics	Redshift, Athena	BigQuery	Data Lake Analytics, Data Lake Store	Indirect
Application Services	Messaging	Simple Notification Service, Simple Queueing Service	Cloud Pub/Sub	Service Bus	Indirect

* It is possible, but the preferred option is to use collectd, Nagios or similar.

Table 5.2: Equivalence of main cloud provider services and their mapping relationship.

VNM	Unit	AWS Related metric(s)	GCP Related metric(s)	M. Azure Related metric(s)
disk_read	Bytes/s	VolumeReadBytes	read_bytes_count	Data Disk Read Bytes/Sec OS Disk Read Bytes/Sec
disk_write	Bytes/s	VolumeWriteBytes	write_bytes_count	Data Disk Write Bytes/Sec OS Disk Write Bytes/Sec
queue_length	count	VolumeQueueLength	pending_operations	Data Disk QD OS Disk QD
io_operations	operations/s	VolumeReadOpsVolumeWriteOps	operation_countoperation_time	Data Disk Read Operations/Sec Data Disk Write Operations/Sec OS Disk Read Operations/Sec OS Disk Write Operations/Sec

Table 5.3: Block Storage Vendor-Neutral Metrics.

VNM	Unit	AWS Related metric(s)	GCP Related metric(s)	M. Azure Related metric(s)
objects	count	NumberOfObjects	storage/object_count	BlobCount
total_requests	count	AllRequests	api/request_count	Transactions
data_downloaded	Bytes	BytesDownloaded	network/sent_bytes_count	Egress
data_uploaded	Bytes	BytesUploaded	network/received_bytes_count	Ingress
storage_used	Bytes	BucketSizeBytes	storage/total_bytes	BlobCapacity
get_requests	count	GetRequests	api/request_count	Transactions
put_requests	count	PutRequests	api/request_count	Transactions
delete_requests	count	DeleteRequests	api/request_count	Transactions
head_requests	count	HeadRequests	api/request_count	Transactions
post_requests	count	PostRequests	api/request_count	Transactions
4xx_errors	count	4xxErrors	api/request_count	Transactions
5xx_errors	count	5xxErrors	api/request_count	Transactions

Table 5.4: Object Storage Vendor-Neutral Metrics.

VNM	Unit	AWS Related metric(s)	GCP Related metric(s)	M. Azure Related metric(s)
CPU_utilization	%	CPUUtilization	database/cpu/utilization	cpu_percent
memory_usage	Bytes	FreeableMemory (Bytes) TotalMemory (GiB)	database/memory/usage	memory_percentmemory_quota (GB)
memory_utilization	%	FreeableMemory (Bytes) TotalMemory (GiB)	database/memory/utilization	memory_percent
disk_usage	Bytes	FreeStorageSpace (Bytes) TotalDisk (GiB)	database/disk/bytes_used	storage_used
disk_utilization	%	FreeStorageSpace (Bytes) TotalDisk (GiB)	database/disk/utilization	storage_percent
disk_write_ops	Count/s	WriteIOPS	database/disk/write_ops_count	Data Disk Write Operations/Sec OS Disk Write Bytes/Sec
disk_read_ops	Count/s	ReadIOPS	database/disk/read_ops_count	Data Disk Read Operations/Sec OS Disk Read Bytes/Sec
network_received	Bytes/s	NetworkReceiveThroughput	database/network/received_bytes_count	network_bytes_ingress
network_transmitted	Bytes/s	NetworkTransmitThroughput	database/network/sent_bytes_count	network_bytes_egress
connections	Count	DatabaseConnections	database/network/connections	connection_successful

Table 5.5: PostgreSQL Vendor-Neutral Metrics.

VNM	Unit	AWS Related metric(s)	GCP Related metric(s)	M. Azure Related metric(s)
CPU_utilization	%	CPUUtilization	database/cpu/utilization	cpu_percent
memory_usage	Bytes	FreeableMemory (Bytes) TotalMemory (GiB)	database/memory/usage	memory_percentmemory_quota
memory_utilization	%	FreeableMemory (Bytes) TotalMemory (GiB)	database/memory/utilization	memory_percent
disk_usage	Bytes	FreeStorageSpace (Bytes) TotalDisk (GiB)	database/disk/bytes_used	storage_used
disk_utilization	%	FreeStorageSpace (Bytes) TotalDisk (GiB)	database/disk/utilization	storage_percent
disk_write_ops	Count/s	WriteIOPS	database/disk/write_ops_count	Data Disk Write Operations/Sec OS Disk Write Bytes/Sec
disk_read_ops	Count/s	ReadIOPS	database/disk/read_ops_count	Data Disk Read Operations/Sec OS Disk Read Bytes/Sec
network_received	Bytes/s	NetworkReceiveThroughput	database/network/received_bytes_count	network_bytes_ingress
network_transmitted	Bytes/s	NetworkTransmitThroughput	database/network/sent_bytes_count	network_bytes_egress
replica_lag	seconds	ReplicaLag	database/mysql/replication/seconds_behind_\master	seconds_behind_master
connections	Count	DatabaseConnections	database/network/connections	connection_successful

Table 5.6: MySQL Vendor-Neutral Metrics.

This set of vendor-neutral metrics is only a partial solution to RQ4, since they should also be able to be monitored for all cloud providers. To this end, it is necessary to formally define how these provider-independent metrics can be calculated based on the metrics exported by each of the cloud providers. In other words, for every vendor-neutral metric defined in the repository, there is a mapping function that relates this metric to one or more metrics exported by each of the supported cloud providers. These functions establish a static mapping between the vendor-neutral set of metrics in the repository and the set of metrics exported by each cloud provider. In this case, however, this static mapping is a valid solution, as these relationships are not conditioned by highly variable external factors. This is ensured by the contract that represents the API used by each cloud provider to export its metrics and on which not only multi-cloud solutions (which could lead to a conflict of interest), but also a large number of monitoring solutions used by its own customers depend.

Continuing with the previous example, the vendor-neutral metric *memory_usage* of the MySQL service has a direct mapping function with the *database/memory/utilization* and *memory_percent* metrics of the equivalent Google Cloud and Microsoft Azure services, respectively, since they measure the same magnitude, with the same unit and the same sampling period. However, in the case of AWS, this vendor-neutral metric can be calculated by composing two metrics as indicated by the mapping function shown in:

$$\begin{aligned} &memory_utilization = \\ &= \left(1 - \left(\frac{FreeableMemory}{2^{30} \times TotalMemory}\right)\right) \times 100 \end{aligned} \tag{5.1}$$

Another example that illustrates this type of mappings is the case of the vendor-neutral metric *io_operations* of the block storage service, which measures the number of input/output operations per second performed. As shown in Table 5.7, this metric is calculated in the case of AWS by adding the *VolumeReadOps* and *VolumeWriteOps* metrics. As these metrics are collected every minute, however, it is necessary to divide by 60 to calculate the operations per second. In the case of Google Cloud, this is calculated by dividing the number of accumulated input/output operations by the accumulated input/output operations time. Finally, this metric is calculated in Microsoft Azure as the sum of all the input/output operation metrics for both data and OS, since the vendor-neutral metric unit measures them all.

Metric	Unit	AWS	Google Cloud	Microsoft Azure
io_operations	operations/s	$= \frac{VolumeReadOps + VolumeWriteOps}{60}$	$= \frac{operation_count}{operation_time}$	$= Data\ Disk\ ReadOperations/Sec$ $+ DataDiskWriteOperations/Sec$ $+ OSDiskReadOperations/Sec$ $+ OSDiskWriteOperations/Sec$

Table 5.7: Mapping functions that relate the *io_operations* vendor-neutral metric to the corresponding metrics of each of the main cloud providers.

Metric	Unit	AWS	Google Cloud	Microsoft Azure
total_requests	count	$= AllRequests$	$= window(sum, api/request_count)$	$= Transactions$

Table 5.8: Mapping functions that relate the *total_requests* vendor-neutral metric to the respective metrics of each of the main cloud providers.

However, these mappings should be thought of as resembling the process of an ETL (extract, transform and load) rather than merely in terms of mathematical functions. For example, if the metric sampling period is different for two cloud providers or if a metric is exported as an increment by one provider and as an accumulated value by another (Table 5.8), it is necessary to use complex event processing (CEP) solutions to establish a time-order window to calculate this type of values. As shown in Table 5.8, for example, the vendor-neutral metric *total_requests* represents the cumulative value of requests, which matches the metrics directly exported by the equivalent AWS and Microsoft Azure services. However, the Google Cloud solution exports this metric as the differential increment with respect to the previous value. Therefore, it is necessary to calculate the accumulated value of this metric.

In addition, the same metric of a solution may contain necessary information for several vendor-neutral metrics, as shown in Table 5.4, where the same metric (i.e. *api/request_count* and *Transactions* for Google Cloud and Microsoft Azure, respectively) has been mapped to different vendor-neutral metrics. This is due to the fact that the metric outputs contain the necessary information to calculate all the vendor-neutral metrics, that is, the number of requests made for each request type.

For the collection of these vendor-neutral metrics, the multi-cloud broker exports a common API, which must implement all the cloud adapters of the different cloud providers. More specifically, each of these adapters will implement the necessary mapping functions to obtain the vendor-neutral metrics defined in the repository, thus reporting the values for the same vendor-neutral metrics of their services.

As a result of all this diversity and the many specificities, it is not easy to establish a mapping between metrics. To solve this complex problem, it is required the knowledge of experts in each of the solutions provided by the different cloud providers in order to define the most complex mappings (indirect mappings) and discover synergies between equivalent solutions. However, the direct mappings that have been established here are broad enough to demonstrate the viability and advantages of this solution. In fact, the PaaS vendor-neutral metrics repository for each service contains at least one metric for all measurable magnitudes common to all providers.

5.3.3 Context metrics

Besides IaaS and PaaS metrics, there is another set of metrics that are common and orthogonal to these services, such as cost or the region where a service is deployed. They have been grouped under the name of context metrics in the broker repository. These metrics are essential for establishing the mapping between SLOs and specific metric conditions, since many of the SLOs that are most important to cloud consumers are related to economic aspects or legislation about the countries where customer data are stored and processed.

Table 5.9 shows the current set of context metrics offered by the repository. Accordingly, there are three metrics on the location of services, i.e. *regions*, *countries* and *continents*, which represent the set of regions, countries and continents where there is a deployed service. In addition, there are three other metrics that reflect the cardinality of these sets, representing the total number of different regions, countries and continents where services are currently deployed. Finally, the *total_cost* metric reflects the total cost of all the services, measured in dollars per month.

To calculate these metrics, each of the adapters for the different cloud providers export the same metrics, albeit limited to their cloud scope. As a result, the metrics exported by the repository are calculated by aggregating the same partial metrics for each of the cloud providers shown in the Description column of Table 5.9. For example, the total cost is the sum of all the total costs exported by the adapters for the different clouds. Similarly, the set of regions is the union of all the sets of regions exported by the adapters for the different clouds. Finally, each adapter has to establish a mapping between the different regions, countries and continents where it is located, as shown in Table 5.10. In addition, the different regions defined by each cloud provider have been merged into a set of unified regions (vendor-neutral regions). Table 5.11 shows the relationship between the vendor-neutral regions and the regions defined by each cloud provider.

Metric	Unit	Description
regions	-	Set of regions where services are located. $\bigcup_{i=0}^{i=m} regions_i$
countries	-	Set of different countries where services are located. $\bigcup_{i=0}^{i=m} countries_i$
continents	-	Set of different continents where services are located. $\bigcup_{i=0}^{i=m} continents_i$
number_regions	count	card(regions)
number_countries	count	card(countries)
number_continents	count	card(continents)
total_cost	$/month	Total cost of all the services. $\sum_{i=0}^{i=m} total_cost_i$

Table 5.9: Context vendor-neutral metrics.

AWS Region	Context Metric	Value
us-east-1	Region	N. Virginia
	Country	US
	Continent	North America
us-east-2	Region	Ohio
	Country	US
	Continent	North America

Table 5.10: Mapping example of two regions defined by AWS and their respective values in vendor-neutral metrics.

vendor-neutral region	AWS	GCP	M. Azure
US_EAST	us-east-1 us-east-2	us-east4 us-east1 us-central1	East US East US 2 Central US West US 2
US_WEST	us-west-1 us-west-2	us-west1 us-west2	West US West US 2 West Central US West Central US
CANADA	ca-central-1	northamerica-northeast1	Canada Central Canada East
SOUTH_AMERICA	sa-east-1	southamerica-east1	Brazil South
EU_CENTRAL	eu-central-1	europe-west3 europe-west6	Germany Central Germany Northeast
EU_WEST	eu-west-1 eu-west-2 eu-west-3	europe-west1 europe-west2 europe-west4	West Europe France Central France South
EU_NORTH	eu-north-1	europe-north1	North Europe UK South UK West
AP_NORTH	ap-northeast-1 ap-northeast-2 ap-northeast-3	asia-northeast1 asia-east1 asia-east2	Japan East Japan West Korea Central Korea South East Asia China East China North China East 2 China North 2
AP_SOUTH	ap-southeast-1 ap-southeast-2 ap-south-1	asia-southeast1 australia-southeast1 asia-south1	Southeast Asia Australia Central Australia Central 2 Australia East Australia Southeast West India Central India South India

Table 5.11: Relationship between the vendor-neutral regions and the regions defined by each cloud provider.

5.4 Evaluation

This section shows how the above solutions to research questions RQ3 and RQ4 work together, and effectively combine and complement each other. To be precise, the evaluation was conducted on two real and typical use cases of multi-cloud environments in different application domains. One use case addressed the IaaS and the other, the PaaS level both within a multi-cloud environment composed of the three leading cloud providers nowadays, i.e. Amazon Web Services or AWS, Google Cloud and Microsoft Azure. This assures a comprehensive validation of the proposed solution. In each use case, the starting point is a SLA representative of the typical needs of cloud consumers in each of the above contexts. Next, it is explained the process whereby each SLO that is part of the SLA can be transformed into conditions expressed as vendor-neutral metrics that can be measured and monitored across all the cloud providers.

5.4.1 Multi-cloud publish/subscribe middleware

This first use case proposes the deployment of a publish/subscribe middleware to support an IoT environment that demands high-performance, like smart cities. This is a frequent use case nowadays, since the features offered by the cloud can be exploited by these middlewares to achieve the high performance required by this type of new technological paradigms. For this reason, E-SilboPS was selected, since it is a context-aware content-based publish-subscribe (CBPS) middleware, specially designed to be elastic in cloud environments [53, 44]. To this end, it is composed of four layers of operators (Connection Point, Access Point, Matcher and Exit Point) that can scale independently using the cloud resources provided by the IaaS.

The SLA shown below reflects the requirements of the services that are described in this use case in such contexts. SLA is composed of three SLOs: the first establishes the minimum performance of the middleware, measured as notifications/s throughput, the second is related to the maximum total cost of the service per month, and the third imposes a restriction on the geo-location of IaaS resources to ensure that data processing is as close as possible to the place where data are going to be created and consumed (i.e. smart city in Spain).

$$SLO_1 : throughput \geq 30\ k\ notifications/s$$
$$SLO_2 : total_cost < 200\ \$/month$$
$$SLO_3 : region = EU_Central$$

First, the SLOs that constitute the specified SLA must be translated into objectives expressed through the vendor-neutral set of metrics from the repository (RQ4). As explained above, the proposed solution for this purpose is a knowledge-based system, part of whose knowledge base is shown in Figure 5.5. It can be seen that the knowledge base is composed of production rules that formally model expert knowledge. Rules R1 to R6 reflect the expert knowledge about the resource-level performance of the four publish/-subscribe middlewares, as illustrated in Figure 5.2. The above study empirically demonstrated that content-based systems or CBPS (i.e. E-SilboPS and BeeTree) were CPU bound, while topic-based systems or TBPS were I/O bound (i.e. ActiveMQ and RabbitMQ). Therefore, these production rules map a throughput SLO in terms of the conditions with respect to maximum CPU or memory usage based on the application domain, i.e. CBPS or TBPS, respectively (N1 & N3).

Note the flexibility of this solution, since all that would be required to support a new publish/subscribe middleware would be to introduce an additional condition to rules R1 or R2, as appropriate. Despite the benefits of the broad coverage offered by this flexibility, it is still possible to express the most specific expert knowledge by restricting the antecedent of the production rule, as illustrated by rule R7 (N2). This rule can be used to introduce concrete E-SilboPS knowledge, since the latency between the different instances of the Matcher operator must be minimal in order to reach high performance (in terms of throughput measured as notifications/s)[53].

On the other hand, production rules R8, R9 and R10 also reflect the expert knowledge referring to the maximum latency between the different cloud provider data centers depending on their geo-location[9]. In this way, the lowest latencies are achieved between data centers in the same region, and they increase depending on whether they are in the same country or the same continent. Therefore, IaaS resources supporting E-SilboPS must be in

[9] `https://docs.google.com/spreadsheets/u/1/d/1lCUjdT-JNoATftGshtUIPQIl0CLb2Z8DCL-k8UAMtec/pubhtml`
`https://github.com/mda590/cloudping.co`

the same region to ensure minimal latency and achieve the throughput SLO. Finally, rules R11, R12 and R13 do not describe any expert knowledge. They are necessary, however, to add geo-location constraints to vendor-neutral metrics. In the case of the cost metric, the SLO is already expressed as a condition in terms of a vendor-neutral metric, and, therefore, no translation is required.

Table 5.12 gives a step-by-step overview of how, starting from the SLA, the knowledge-based system is able to map the different SLOs into specific vendor-neutral metric objectives. Each step specifies the facts that have been obtained and the set of production rules that can be activated (conflict set) as their antecedents have been fulfilled. In this case, the inference engine evaluates the rules by forward chaining, applying a conflict resolution strategy that prioritizes the execution matching the most recent facts.

R1: **IF** $App = E - SilboPS \vee App = BeTree$
THEN $AT = CBPS$
R2: **IF** $App = RabbitMQ \vee App = ActiveMQ$
THEN $AT = TBPS$
R3: **IF** $AT = CBPS \wedge throughput \geq 10\ k\ notifications/s$
THEN $CPU_usage \leq 80\ \%$
R4: **IF** $AT = CBPS \wedge throughput < 10\ k\ notifications/s$
THEN $CPU_usage \leq 90\ \%$
R5: **IF** $AT = TBPS \wedge throughput \geq 1\ k\ notifications/s$
THEN $memory_usage \leq 40\ \%$
R6: **IF** $AT = TBPS \wedge throughput < 1\ k\ notifications/s$
THEN $memory_usage \leq 70\ \%$
R7: **IF** $App = E\text{-}SilboPS \wedge$
$\wedge\ throughput \geq 10\ k\ notifications/s$
THEN $latency < 30\ ms$
R8: **IF** $Latency < 30\ ms$
THEN $number_regions = 1$
R9: **IF** $30\ ms \leq latency < 50\ ms$
THEN $number_countries = 1$
R10: **IF** $50\ ms \leq latency < 100\ ms$
THEN $number_countinents = 1$
R11: **IF** $continent = X$
THEN$\{X\} \cup continents$
R12: **IF** $country = X$
THEN$\{X\} \cup countries$
R13: **IF** $region = X$
THEN$\{X\} \cup regions$

Figure 5.5: Production rules that are part of the knowledge base for the multi-cloud publish/subcribe middleware use case.

Table 5.12: Inference process for the translation of SLOs into conditions expressed as vendor-neutral metrics for the multi-cloud publish/subscribe middleware use case.

	Facts	**Conflict Set**
Step 0	$App = E\text{-}SilboPS$ $throughput \geq 30\ k\ notifications/s$ $total_cost < 200\ \$/month$ $region = EU_Central$	$\{\mathbf{R1}, R7, R13\}$
Step 1	$App = E\text{-}SilboPS$ $throughput \geq 30\ k\ notifications/s$ $total_cost < 200\ \$/month$ $region = EU_Central$ $AT = CBPS$	$\{\mathbf{R3}, R7, R13\}$
Step 2	$App = E\text{-}SilboPS$ $throughput \geq 30\ k\ notifications/s$ $total_cost < 200\ \$/month$ $region = EU_Central$ $AT = CBPS$ $CPU_usage \leq 80\ \%$	$\{\mathbf{R7}, R13\}$
Step 3	$App = E\text{-}SilboPS$ $throughput \geq 30\ k\ notifications/s$ $total_cost < 200\ \$/month$ $region = EU_Central$ $AT = CBPS$ $CPU_usage \leq 80\ \%$ $latency < 30\ ms$	$\{\mathbf{R8}, R13\}$
Step 4	$App = E\text{-}SilboPS$ $throughput \geq 30\ k\ notifications/s$ $total_cost < 200\ \$/month$ $region = EU_Central$ $AT = CBPS$ $CPU_usage \leq 80\ \%$ $latency < 30\ ms$ $number_regions = 1$	$\{\mathbf{R13}\}$

Step 5	$App = E\text{-}SilboPS$ $throughput \geq 30\ k\ notifications/s$ $total_cost < 200\ \$/month$ $region = EU_Central$ $AT = CBPS$ $CPU_usage \leq 80\ \%$ $latency < 30\ ms$ $number_regions = 1$ $regions = \{EU_Central\}$	$\{\}$

At the end of the process, the set of conditions expressed exclusively by vendor-neutral metrics is output in Step 5 when the conflict set is empty, as shown below. The IaaS resources must be deployed in a single region, namely EU_Central, the total cost of all the resources deployed in the different clouds must not be greater than $200 per month, and CPU use must not exceed 80% of computational resources. Finally, through the above mapping functions, each cloud adapter will monitor and report the necessary resources as vendor-neutral metrics to the multi-cloud broker. In this case, the total cost will be calculated by adding the partial cost reported by each provider. It is important to ensure that no cloud adapter deploys resources outside the EU_Central region at any time and that CPU usage, measured via collectd, does not exceed 80% in any instance.

$$C_1 : total_cost < 200\ \$/month$$
$$C_2 : CPU_usage \leq 80\ \%$$
$$C_3 : number_regions = 1$$
$$C_4 : regions = \{EU_Central\}$$

5.4.2 Multi-cloud RDBMS

The objective of this second use case is to offer a classical OLTP (on-line transactional processing) relational database service (MySQL) as part of a typical three-tier architecture. The requirements of this specific use case, reflected in this SLA:

$$SLO_1 : law = GDPR$$
$$SLO_2 : response_time < 80\ ms \wedge max.\ connections = 60$$
$$SLO_3 : availability = high$$
$$SLO_4 : total_cost \leq 300\ \$/month$$

- The offered service must comply with the European GDPR (General Data Protection Regulation) approved in 2016 by the European Commission [3]. Therefore, data must be stored and processed within Europe (SLO_1).
- The average response time of requests to the database must not exceed 80 milliseconds to ensure a good quality of service. Furthermore, as this database is part of a three-tier architecture, the maximum number of connections is limited to a maximum of 60 simultaneous connections (SLO_2).
- As it is an OLTP service, data availability is essential to ensure business continuity, and therefore availability must be high (SLO_3).
- The total cost of the solution must not exceed $300 per month ($SLO_4$).

Some of the production rules that populate the knowledge base of this use case are shown in Figure 5.6. Rules R1 to R5 model how the GDPR regulation restriction affects service deployment, restricting it to European data centers, albeit in three different regions. In the case of an OLAP (on-line analytical processing) application, the number of locations is reduced since high availability is not so critical. Additionally, if there were no regulatory restrictions on data sovereignty, the high availability criterion would be more restrictive since replicas would have to be deployed on more than one continent (N1, N2 & N3).

Rules R6 and R7 describe that the lag between replicas must be limited in order to achieve a specified response time, and these replicas must be deployed in the same region (R8) if a very low lag between replicas is required.

For their part, production rules R9 to R11 formally model the expert knowledge on the computational resource-level behavior of a MySQL OLTP application discovered experimentally (N1 & N3). OLTP tasks are characterized by short-duration transactions with low I/O and CPU usage. Thus, in order to ensure a response time of 80 ms for this type of OLTP database application, the maximum CPU and disk usage thresholds decrease as the maximum number of connections increases, as a consequence of a possibly much more sudden increase in workload due to the greater number of potential requests at any given time. Similarly, due to the more intensive use of CPU and I/O operations that is characteristic of OLAP applications, the production rules would be similar, albeit establishing more restrictive thresholds for CPU usage and the number of I/O operations per second, since these metrics are the KPIs of this type of applications (N2).

R1: **IF** $law = GDPR$ **THEN** $Europe \in continents \wedge number_continents = 1 \wedge$
$\wedge\ regions = \{EU_NORTH, EU_WEST, EU_CENTRAL\}$

R2: **IF** $availability = high \wedge App = MySQL_OLTP \wedge law = GDPR$
THEN $number_regions = 3$

R3: **IF** $availability = high \wedge App = MySQL_OLAP \wedge law = GDPR$
THEN $number_regions = 2$

R4: **IF** $availability = high \wedge App = MySQL_OLTP \wedge \neg\ law = GDPR$
THEN $number_continents = 3$

R5: **IF** $availability = high \wedge App = MySQL_OLAP \wedge \neg\ law = GDPR$
THEN $number_continents = 2$

R6: **IF** $response_time \leq 50\ ms$ **THEN** $replica_lag \leq 1\ s$

R7: **IF** $response_time \leq 100\ ms$ **THEN** $replica_lag \leq 2\ s$

R8: **IF** $replica_lag \leq 1\ s$ **THEN** $number_regions = 1$

R9: **IF** $max.\ connections \leq 50 \wedge App = MySQL_OLTP \wedge response_time \leq 100\ ms$
THEN $CPU_utilization \leq 90\ \% \wedge disk_utilization \leq 90\ \%$

R10: **IF** $50 < max.\ connections \leq 100 \wedge App = MySQL_OLTP \wedge response_time \leq 100\ ms$
THEN $CPU_utilization \leq 80\ \% \wedge disk_utilization \leq 80\ \%$

R11: **IF** $max.\ connections \geq 100 \wedge App = MySQL_OLTP \wedge response_time \leq 100\ ms$
THEN $CPU_utilization \leq 80\ \% \wedge disk_utilization \leq 80\ \%$

Figure 5.6: Production rules that form the knowledge base for the MySQL OLTP use case.

As in the previous use case, Table 5.13 shows the functioning of the knowledge-based system inference engine, showing the facts and the conflicting set of production rules at each step.

Table 5.13: Inference process for the translation of SLOs in conditions expressed as vendor-neutral metrics for the MySQL OLTP use case.

	Facts	Conflict set
Step 0	$App = MySQL_OLTP$ $response_time < 80\ mss$ $max.\ connections = 60$ $total_cost < 300\ \$/month$ $law = GDPR$	$\{\mathbf{R1}, R2, R7, R10\}$
Step 1	$App = MySQL_OLTP$ $response_time < 80\ mss$ $max.\ connections = 60$ $total_cost < 300\ \$/month$ $law = GDPR$ $number_continents = 1$ $Europe \in continents$ $regions = \{EU_NORTH,$ $EU_WEST, EU_CENTRAL\}$	$\{\mathbf{R2}, R7, R10\}$
Step 2	$App = MySQL_OLTP$ $response_time < 80\ mss$ $max.\ connections = 60$ $total_cost < 300\ \$/month$ $law = GDPR$ $number_continents = 1$ $Europe \in continents$ $regions = \{EU_NORTH,$ $EU_WEST, EU_CENTRAL\}$ $number_regions = 3$	$\{\mathbf{R7}, R10\}$

Step 3	$App = MySQL_OLTP$ $response_time < 80\ mss$ $max.\ connections = 60$ $total_cost < 300\ \$/month$ $law = GDPR$ $number_continents = 1$ $Europe \in continents$ $regions = \{EU_NORTH,$ $EU_WEST, EU_CENTRAL\}$ $number_regions = 3$ $replica_lag \leq 2\ s$	{**R10**}
Step 4	$App = MySQL_OLTP$ $response_time < 80\ mss$ $max.\ connections = 60$ $total_cost < 300\ \$/month$ $law = GDPR$ $number_continents = 1$ $Europe \in continents$ $regions = \{EU_NORTH,$ $EU_WEST, EU_CENTRAL\}$ $number_regions = 3$ $replica_lag \leq 2\ s$ $CPU_utilization \leq 80\ \%$ $disk_utilization \leq 80\ \%$	{}

At the end of this the process in Step 4, i.e. when the conflict set is empty and no further production rules can be activated, the set of conditions expressed as vendor-neutral metrics is output:

$$C_1 : number_continents = 1$$
$$C_2 : Europe \in continents$$
$$C_3 : number_regions = 3$$
$$C_4 : replica_lag \leq 2\ s$$
$$C_5 : CPU_utilization \leq 80\ \%$$
$$C_6 : disk_utilization \leq 80\ \%$$
$$C_7 : total_cost < 300\ \$/month$$

Their fulfillment also ensures compliance with the SLOs. Accordingly, both the regulatory restriction set out in the GDPR law and the high availability requirement mean that all MySQL PaaS resources having to be deployed on the European continent, albeit in three different regions. On the other hand, the lag between the different replicas should not be longer than two seconds, and CPU and disk utilization should not exceed 80 %. Note that the vendor-neutral metrics used to express these conditions are either context or PaaS RDBMS (specifically, MySQL) service metrics.

To ensure compliance with this set of conditions, it is essential to be able to measure and monitor the vendor-neutral metrics subject to the specified conditions in each of the clouds. To this end, as explained in the proposed solution to RQ3, it has to be defined the mapping functions that relate these abstract, provider-independent metrics to the metrics exported by each cloud provider. Table 5.14 shows the mapping functions of the vendor-neutral metrics involved in the RDBMS service. The unit and the function whereby each vendor-neutral metric can be calculated is specified based on the metrics exported by each cloud provider (see Table 5.6). Each of the three cloud providers have equivalent metrics for the *CPU_utilization* and *replica_lag* vendor-neutral metrics, that is, they measure the same magnitude using the same unit and in the same sampling period. The same applies to the *disk_utilization* metric for the Google Cloud and Microsoft Azure providers, where the *database/disk/utilization* and *storage_percent* metrics, respectively, monitor the same magnitude defined in the *disk_utilization* vendor-neutral metric. However, in the case of Amazon Web Services, the mapping function is somewhat more complex, as it must be calculated using the *FreeStorageSpace* and *TotalDisk* metrics. As they are measured in different units, they need to be converted to the same unit before calculating the percentage of disk space used as the complementary value of the percentage of free disk space. On the other hand, the mapping of the context vendor-neutral metrics involved (i.e. *total_cost*, *continents*, *number_continents* and *number_countries*) can be mapped directly as the cumulative value of each of the above metrics collected by each cloud adapter in each cloud, as shown in Section 5.3.3.

Metric	Unit	AWS	Google Cloud	Microsoft Azure
CPU_utilization	%	$= CPUUtilization$	$= database/cpu/utilization$	$= cpu_percent$
disk_utilization	%	$= \left(1 - \frac{FreeStorageSpace}{2^{30} \times TotalDisk}\right) \times 100$	$= database/disk/utilization$	$= storage_percent$
replica_lag	seconds	$= ReplicaLag$	$= database/mysql/$ $replication/seconds_behind_nmaster$	$= seconds_behind_master$

Table 5.14: Mapping functions that relate MySQL OLTP use case vendor-neutral metrics to the corresponding metrics of each of the main cloud providers.

Through these use cases, the viability and operation of the solutions proposed to research questions RQ3 and RQ4 has been evaluated, checking that they met the stated requirements. For the sake of clarity and understanding, only the part of the knowledge bases that relevant to each use case has been shown. This can be expanded and complicated as required by the expert knowledge. Note also that the vendor-neutral metric conditions output as a result are then evaluated by a system that decides and takes appropriate actions to ensure that they are met. For example, the system may decide to scale out the service in order to distribute the load and decrease the overall CPU usage to ensure compliance with the CPU use conditions, when a resource is close to the defined threshold value. Currently there are a wide variety of such systems implementing purely reactive and even proactive approaches [31]. However, as they are completely decoupled from the solution proposed in this work, different approaches can be tested in different contexts.

5.5 Discussion of results

In this chapter we have presented and evaluated a solution capable of automatically translating the SLAs into specific SLOs by means of a knowledge-based system that allows the capture of information and reasoning from experts in different application contexts (as demonstrated in the evaluation carried out), to ensure compliance with the SLAs defined by the cloud-consumer, thus removing this burden from the latter. This allows a positive response to the third research question posed in this work (RQ3). Furthermore, the answer to this third research question complements the solution provided in the previous chapter to the second research question (RQ2), and both constitute a solution to the Problem2 addressed by this thesis.

In the last part of this chapter, a set of metrics at different levels (IaaS, PaaS and Context) have been presented, which are independent of the cloud provider (vendor-neutral) and can be monitored at the main cloud providers by means of a set of mapping functions (cross-provider) that have also been presented, explained and evaluated in the chapter. This solution allows the monitoring of cloud applications deployed in different cloud providers at different levels, making it possible to exploit the advantages of the multi-cloud and avoid vendor lock-in. For this reason, this solution provides a satisfactory response to the last of the research questions (RQ4) identified in this thesis, and therefore provides a solution to the last of the problems addressed in this work (Problem3).

Chapter 6

Conclusions

Every new beginning comes from some other beginning's end.
— Seneca

This work has focused on the validation of the hypothesis presented at the beginning (Hypothesis), which raises the possibility of providing a solution to the three problems that have motivated this study. To carry out this validation, four research questions related to the problems were established (RQ1 → Problem1; RQ2 and RQ3 → Problemm2, RQ4 → Problem3). As demonstrated throughout this document, it has been possible to respond satisfactorily to each and every one of these research questions, thus providing a solution to the problems posed, and validating the hypothesis that has led this study.

For the sake of clarity, the main conclusions of this work have been summarized below, detailing the extent to which each of the research questions posed has been answered.

On the one hand, in order to answer the first two research questions, this work has presented FLAS (Forecasted Load Auto-Scaling) a generic distributed system auto-scaler focused on SLAs by combining the advantages of both auto-scaling approaches, proactive and reactive (RQ1). Furthermore, it is capable of automatically identifying the KPIs to be monitored and the resource involved which is the bottleneck, and therefore needs to be scaled, so that the distributed system is not saturated and the SLA defined by the cloud-consumer is not violated (RQ2).

The main contributions of FLAS are (i) the design and definition of the architecture of a generic framework for distributed systems auto-scaling through the combination of proactive and reactive techniques, and (ii) a solution for the auto-scaling of a content-based publish-subscribe distributed system (E-SilboPS). The

problems that have been addressed by implementing the architectural framework for the case study have been (i) to create a model to predict the trend of relevant SLA parameters (i.e. response time or throughput), (ii) to create a model capable of estimating, agnostically with respect to the type of application, SLA parameters based on resource usage metrics (RQ2), (iii) to create a model to predict the scaling time and (iv) a mainly proactive auto-scaling algorithm that also contemplates a reactive scaling as a contingency to possible prediction failures. In addition, the modular and decoupled design of the FLAS architecture allows it to adapt to different distributed systems by modifying few configuration parameters and to replace the predictive models of the different modules of its architecture while respecting their definition.

Experimental results show how FLAS is able to meet the performance requirements set by the SLA even in the worst possible cases. More specifically, it has been demonstrated how FLAS enables a content-based publish-subscribe system to meet the maximum response time imposed by the SLA for over 99% of the time by taking the necessary scaling decisions at any given time under a wide variety of workloads by exploiting the advantages of both reactive and proactive approaches. In addition, it has also been demonstrated how its scaling algorithm is able to determine those cases in which triggering an scaling action is not efficient, since the overhead involved in the scaling action could lead to a violation of the SLA greater than that incurred in the case of not scaling. To the best of our knowledge, this would be the first auto-scaler for content-based publish-subscribe systems.

Furthermore, it has been empirically demonstrated how FLAS efficiently coordinates both the reactive and predictive auto-scaling approaches that compose it to offer better or equal SLA compliance than each of them separately. Compared to other auto-scalers, FLAS achieves the same positive results as other auto-scalers in compliance with SLAs but with a much lower over-provisioning, reducing the economic cost and the energy consumption.

On the other hand, In this work it has been reported an automatically and transparently to the cloud-consumer solution capable of translating SLAs into specific objectives described as conditions applicable to a set of metrics that can be measured on multiple-cloud (multi-cloud) providers for both IaaS and PaaS services.

In this way, it has been possible to respond satisfactorily to the other two research questions with a solution capable of exploiting the synergies between the two solutions. In response to the third research question, we present a solution based on a knowledge-based system capable of automatically translating SLAs into specific objectives, the enforcement of which ensures compliance with these SLAs. In response to the fourth research question, we presented a solution that allows the specificities of the different cloud providers to be abstracted by means of a large vendor-neutral metrics repository to monitor not only IaaS and PaaS services but

also useful context information, such as service costs and geo-location. In order to establish this repository of vendor-neutral cross-provider measurable metrics, a study has been carried out covering a total of 21 cloud services were analysed, grouped into 6 more generic categories (Compute, Network, Storage, Database, Big Data & Analytics and Application Services), offered by the 3 main Cloud providers (Amazon Web Services or AWS, Google Cloud or GC and Microsoft Azure). From each of these services, the metrics exported by each of these providers have been examined, making a total of 625 metrics (AWS: 236, GC: 202 and M. Azure: 187). Finally, a repository with 70 PaaS vendor-neutral metrics for 9 equivalent cloud services is presented in this work. This metrics repository is monitored in the different providers of a multi-cloud environment by means of mapping functions that statically relate these vendor-neutral metrics to metrics exported by each cloud provider.

The complementarity of the two solutions to these last two research questions results in a system that takes the onus of manually translating its SLAs into lower-level objectives (which tends to be an error-prone process) off the cloud consumer. By expressing these objectives through vendor-neutral cross-provider measurable metrics, all the benefits of multi-cloud can be exploited. This was experimentally verified by evaluating the system based on two use cases in a multi-cloud environment composed of the main cloud providers. All these evaluations have proved the viability and correct functioning of the proposed solution, automatically translating different types of SLOs (such as regulatory restrictions with respect to data sovereignty, performance or cost requirements) taking into account the application domain for both IaaS and PaaS services, respectively.

For all these reasons, it is concluded that: **It is possible to automatically scale cloud services by means of an auto-scaler system that allows (i) compliance and adherence to an SLA specified by the cloud-consumer, (ii) automatic detection of the resource acting as a bottleneck, and (iii) monitoring of the metrics needed to make the necessary scaling decisions can be performed in multi-cloud environments**.

Chapter 7

Future Work

In every end, there is also a beginning.
— Libba Bray, A Great and Terrible Beauty .

This chapter includes the open questions that have not been addressed, as they are beyond the scope of this thesis, as well as the new opportunities discovered during the development of this work.

7.1 About auto-scalers and FLAS

This work addresses all dimensions of scaling, however, the current implementation of FLAS duplicates or divides by two the resources to be added or removed as a first approach to the dimension of how much to scale. We are currently working on a module that allows predicting the minimum configuration required for the performance predictions obtained. However, this is a module whose configuration is quite specific to the application and therefore not as generic for all distributed systems as the rest of the FLAS architecture that we have presented in this work. For example, the E-SilboPS with which we have evaluated FLAS in this work allows to deploy or remove several instances of an operator in a single scaling operation, but other systems do not allow this option and must sequence successive scaling operations.

In addition, we are currently working to improve workload trend prediction in order to predict non-stationary workloads.

Currently, the prediction of the time of a scaling action (T'_{sa}) is determined by the load, more specifically, in the case of E-SilboPS we have seen that it is based on the ratio of notifications/s and the load of processed subscriptions. We believe that it would be interesting to add as a predictor to this model the prediction of the load in future moments, to determine more precisely the time of the scaling action and avoid possible uncontrolled peaks of response time when the scaling algorithm of the application involves an overhead (dynamic distribution of the state of the operators).

Finally, other improvements that are being studied are, on the one hand, the inclusion of the current configuration of the distributed system as an additional predictor of the predictive models to test whether the configuration of a distributed system at a given time can condition future predictions. On the other hand, now that the relationships between low-level and high-level metrics have been studied by means of a statistical predictive model, another predictive model could be developed with other methods (i.e. Artificial Neural Networks) that take into account these discovered relationships.

7.2 About Vendor-Neutral Cross-Provider Measurable Metrics

Although a lot has been published in the literature on the formalization of SLAs, there is currently no de facto standard for the formalization of these agreements. Due to the importance of SLAs as the root of all the solutions proposed in this work, it would be valuable to implement and adapt some of these formalization models to the requirements of this particular research, thus defining an interface for integration with other cloud solutions. In addition, the formalization, using ontologies and/or feature models, of the interconnection interface of the two solutions presented in this work could be very enriching and decouple the two solutions.

Another interesting feature would be to provide the system with a conflict detection and resolution system emulating policy-based solutions [18]. This detection and correction must be carried out at two levels. On the one hand, it is necessary to detect possible conflicts at the SLO level, that is, that no two or more SLOs are contradictory (e.g. $SLO1 = Throughput > 30k\ Notifications/s \wedge SLO2 = Throughput < 30k\ Notifications/s$). The detection of this first case is quite simple, and its correction requires the cloud consumer to redefine any conflicting SLOs. On the other hand, possible conflicts between the objectives of the vendor-neutral metrics output as a result of this second phase must also be detected and corrected. However, they may be harder to detect and resolve. The conflict resolution policy may range widely from the choice of the most restrictive objectives to more complex solutions, such as the prioritization of SLOs.

Another functionality that will add great value to the solution would be to implement a feedback loop to the knowledge-based system. In this way, the system would acquire knowledge not only from experts but also from the results of system use. The system would refine the production rules based on the outcomes of past experiences, thus becoming a cognitive system.

Finally, some of the services offered by cloud providers are quite complex and require not only theoretical knowledge of the solutions, but also a deeper knowledge that only professional experience can provide. For this reason, we believe that the future collaboration of different cloud service experts can contribute not only to transferring part of their expertise to the knowledge-based system, but also to collaborative work on the discovery of new relationships between metrics (indirect mappings) in order to create a larger repository of vendor-neutral metrics. On the other hand, several works on cloud computing propose the measurement of new metrics (e.g. energy consumption) that could extend the vendor-neutral metrics repository, increasing the benefits and possibilities of the solution proposed here.

Bibliography

[1] Scryer: Netflix's Predictive Auto Scaling Engine — by Netflix Technology Blog — Netflix TechBlog, . URL https://netflixtechblog.com/scryer-netflixs-predictive-auto-scaling-engine-a3f8fc922270.

[2] Scryer: Netflix's Predictive Auto Scaling Engine — Part 2 — by Netflix Technology Blog — Netflix TechBlog, . URL https://netflixtechblog.com/scryer-netflixs-predictive-auto-scaling-engine-part-2-bb9c4f9b9385.

[3] Whitepaper on German Data Protection. (January), 2017.

[4] Yahya Al-Dhuraibi, Fawaz Paraiso, Nabil Djarallah, and Philippe Merle. Elasticity in Cloud Computing: State of the Art and Research Challenges. *IEEE Transactions on Services Computing*, 11(2):430–447, mar 2018. ISSN 1939-1374. doi: 10.1109/TSC.2017.2711009. URL https://ieeexplore.ieee.org/document/7937885/.

[5] Bourair Alattar and Norita Md Norwawi. A personalized search engine based on correlation clustering method. *Journal of Theoretical and Applied Information Technology*, 93(2):345–352, 2016. ISSN 18173195.

[6] David E Bakken. Middleware Introduction. *Washington State University*, 2002.

[7] Raphaël Barazzutti, Pascal Felber, Christof Fetzer, Emanuel Onica, Jean-françois Pineau, Marcelo Pasin, Etienne Rivière, and Stefan Weigert. StreamHub. In *Proceedings of the 7th ACM international conference on Distributed event-based systems - DEBS '13*, page 63, New York, New York, USA, 2013. ACM Press. ISBN 9781450317580. doi: 10.1145/2488222.2488260. URL http://dl.acm.org/citation.cfm?doid=2488222.2488260.

[8] Raphael Barazzutti, Thomas Heinze, Andre Martin, Emanuel Onica, Pascal Felber, Christof Fetzer, Zbigniew Jerzak, Marcelo Pasin, and Etienne Riviere. Elastic Scaling of a High-Throughput Content-Based Publish/Subscribe Engine. In *2014 IEEE 34th International Conference on Distributed Computing Systems*, pages 567–576. IEEE, jun 2014. ISBN 978-1-4799-5169-7. doi: 10.1109/ICDCS.2014.64. URL http://ieeexplore.ieee.org/document/6888932/.

[9] Bimal K. Bose. Expert System, Fuzzy Logic, and Neural Network Applications in Power Electronics and Motion Control. *Proceedings of the IEEE*, 82(8): 1303–1323, 1994. ISSN 15582256. doi: 10.1109/5.301690.

[10] Antonio Carzaniga and Alexander L Wolf. Forwarding in a content-based network. In *Proceedings of the 2003 conference on Applications, technologies, architectures, and protocols for computer communications - SIGCOMM '03*, page 163, New York, New York, USA, 2003. ACM Press. ISBN 1581137354. doi: 10.1145/863973.863975. URL `http://portal.acm.org/citation.cfm?doid=863955.863975`.

[11] Antonio Carzaniga, David S Rosenblum, and Alexander L Wolf. Design and evaluation of a wide-area event notification service. *ACM Transactions on Computer Systems*, 19(3):332–383, aug 2001. ISSN 0734-2071. doi: 10.1145/380749.380767. URL `https://dl.acm.org/doi/10.1145/380749.380767`.

[12] Emiliano Casalicchio. A study on performance measures for auto-scaling CPU-intensive containerized applications. *Cluster Computing*, 22(3):995–1006, 2019. ISSN 15737543. doi: 10.1007/s10586-018-02890-1. URL `https://doi.org/10.1007/s10586-018-02890-1`.

[13] Emiliano Casalicchio and Luca Silvestri. Autonomic Management of Cloud-Based Systems: The Service Provider Perspective. In Erol Gelenbe and Ricardo Lent, editors, *Computer and Information Sciences III*, pages 39–47, London, 2013. Springer London. ISBN 978-1-4471-4594-3. doi: 10.1007/978-1-4471-4594-3_5. URL `http://link.springer.com/10.1007/978-1-4471-4594-3{_}5`.

[14] Raul Castro Fernandez, Matteo Migliavacca, Evangelia Kalyvianaki, and Peter Pietzuch. Integrating scale out and fault tolerance in stream processing using operator state management. In *Proceedings of the 2013 international conference on Management of data - SIGMOD '13*, page 725, New York, New York, USA, 2013. ACM Press. ISBN 9781450320375. doi: 10.1145/2463676.2465282. URL `http://dl.acm.org/citation.cfm?doid=2463676.2465282`.

[15] K. Mani Chandy and Leslie Lamport. Distributed snapshots: determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985. ISSN 07342071. doi: 10.1145/214451.214456. URL `http://portal.acm.org/citation.cfm?doid=214451.214456`.

[16] Yuan Chen, Subu Iyer, Xue Liu, Dejan Milojicic, and Akhil Sahai. Translating Service Level Objectives to lower level policies for multi-tier services. *Cluster Computing*, 11(3):299–311, sep 2008. ISSN 1386-7857. doi: 10.1007/s10586-008-0059-6. URL `http://link.springer.com/10.1007/s10586-008-0059-6`.

[17] M. Debusmann and A. Keller. Sla-driven management of distributed systems using the common information model. In *IFIP/IEEE Eighth International Symposium on Integrated Network Management, 2003.*, pages 563–576, 2003. doi: 10.1109/INM.2003.1194211.

[18] N Dunlop, J Indulska, and K Raymond. Methods for Conflict Resolution in {Policy-Based} Management Systems. *7th Int. Enterprise Distributed Object Comp. Conf. ({EDOC})*, pages 98–111, 2003. URL `http://dx.doi.org/10.1109/EDOC.2003.1233841`.

[19] Vincent C. Emeakaroha, Ivona Brandic, Michael Maurer, and Schahram Dustdar. Low level Metrics to High level SLAs - LoM2HiS framework: Bridging the gap between monitored metrics and SLA parameters in cloud environments. In *2010 International Conference on High Performance Computing & Simulation*, pages 48–54. IEEE, jun 2010. ISBN 978-1-4244-6827-0. doi: 10.1109/HPCS.2010.5547150. URL `http://ieeexplore.ieee.org/document/5547150/`.

[20] Vincent C Emeakaroha, Michael Maurer, Ivona Brandic, and Schahram Dustdar. FoSII - Foundations of Self-Governing ICT Infrastructures. *Ercim News*, Number 83(October 2010):40–41, 2010.

[21] Patrick TH. Eugster, Pascal A. Felber, Rachid Guerraoui, and Anne-Marie Kermarrec. The many faces of publish/subscribe. *ACM Computing Surveys*, 35(2):114–131, jun 2003. ISSN 03600300. doi: 10.1145/857076.857078. URL `http://portal.acm.org/citation.cfm?doid=857076.857078`.

[22] Edward A Feigenbaum. Expert systems in the 1980s. *State of the art report on machine intelligence. Maidenhead: Pergamon-Infotech*, 1981.

[23] Nicolas Ferry, Alessandro Rossini, Franck Chauvel, Brice Morin, and Arnor Solberg. Towards Model-Driven Provisioning, Deployment, Monitoring, and Adaptation of Multi-cloud Systems. In *2013 IEEE Sixth International Conference on Cloud Computing*, pages 887–894. IEEE, jun 2013. ISBN 978-0-7695-5028-2. doi: 10.1109/CLOUD.2013.133. URL `http://ieeexplore.ieee.org/lpdocs/epic03/wrapper.htm?arnumber=6740239`.

[24] Nirnay Ghosh and Soumya K. Ghosh. An approach to identify and monitor SLA parameters for storage-as-a-service cloud delivery model. In *2012 IEEE Globecom Workshops*, number vi, pages 724–729. IEEE, dec 2012. ISBN 978-1-4673-4941-3. doi: 10.1109/GLOCOMW.2012.6477664. URL `http://ieeexplore.ieee.org/document/6477664/`.

[25] Jayavardhana Gubbi, Rajkumar Buyya, Slaven Marusic, and Marimuthu Palaniswami. Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7):1645–1660, 2013. ISSN 0167739X. doi: 10.1016/j.future.2013.01.010. URL `http://dx.doi.org/10.1016/j.future.2013.01.010`.

[26] Nikolas Roman Herbst, Samuel Kounev, and Ralf Reussner. Elasticity in Cloud Computing : What It Is , and What It Is Not. *Presented as part of the 10th International Conference on Autonomic Computing*, pages 23–27, 2013. URL `http://sdqweb.ipd.kit.edu/publications/pdfs/HeKoRe2013-ICAC-Elasticity.pdf`.

[27] George Kousiouris, Dimosthenis Kyriazis, Spyridon Gogouvitis, Andreas Menychtas, Kleopatra Konstanteli, and Theodora Varvarigou. Translation of application-level terms to resource-level attributes across the Cloud stack layers. In *2011 IEEE Symposium on Computers and Communications (ISCC)*, pages 153–160. IEEE, jun 2011. ISBN 978-1-4577-0680-6. doi: 10.1109/ISCC.2011.5984009. URL `http://ieeexplore.ieee.org/document/5984009/`.

[28] Edward D Lazowska, John Zahorjan, G Scott Graham, and Kenneth C Sevcik. *Quantitative System Performance: Computer System Analysis Using Queueing Network Models*. Prentice-Hall, Inc., USA, 1984. ISBN 0137469756.

[29] Federico Lombardi, Andrea Muti, Leonardo Aniello, Roberto Baldoni, Silvia Bonomi, and Leonardo Querzoni. PASCAL: An architecture for proactive auto-scaling of distributed services. *Future Generation Computer Systems*, 98:342–361, 2019. ISSN 0167739X. doi: 10.1016/j.future.2019.03.003. URL `https://doi.org/10.1016/j.future.2019.03.003`.

[30] Tania Lorido-Botrán, José Miguel-Alonso, and Jose Antonio Lozano. Comparison of Auto-scaling Techniques for Cloud Environments. *Actas de las XXIV Jornadas de Paralelismo*, (January), 2013.

[31] Tania Lorido-Botran, Jose Miguel-Alonso, and Jose A. Lozano. A Review of Auto-scaling Techniques for Elastic Applications in Cloud Environments. *Journal of Grid Computing*, 12(4):559–592, dec 2014. ISSN 1570-7873. doi: 10.1007/s10723-014-9314-7. URL `http://link.springer.com/10.1007/s10723-014-9314-7`.

[32] Adil Maarouf, Abderrahim Marzouk, and Abdelkrim Haqiq. A review of SLA specification languages in the cloud computing. In *2015 10th International Conference on Intelligent Systems: Theories and Applications (SITA)*, pages 1–6. IEEE, oct 2015. ISBN 978-1-5090-0220-7. doi: 10.1109/SITA.2015.7358406. URL `http://ieeexplore.ieee.org/document/7358406/`.

[33] NIST. NIST Cloud Computing Standards Roadmap. Technical report, National Institute of Standards and Technology, Gaithersburg, MD, jul 2013. URL `https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.500-291r2.pdf`.

[34] NIST/SEMATECH. *e-Handbook of Statistical Methods*. URL `https://doi.org/10.18434/M32189`.

[35] Adrian Paschke and Elisabeth Schnappinger-Gerull. A Categorization Scheme for SLA Metrics. *Service Oriented Electronic Commerce*, pages 25–40, 2006. ISSN 18632351. URL http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.93.1800&rep=rep1&type=pdf.

[36] Pankesh Patel, Ajith Ranabahu, and Amit Seth. Service Level Agreement in Cloud Computing. *Corescholar.Libraries.Wright.Edu*, 2010. URL http://corescholar.libraries.wright.edu/knoesis/78.

[37] Dana Petcu. Multi-Cloud. In *Proceedings of the 2013 international workshop on Multi-cloud applications and federated clouds - MultiCloud '13*, page 1, New York, New York, USA, 2013. ACM Press. ISBN 9781450320504. doi: 10.1145/2462326.2462328. URL http://dl.acm.org/citation.cfm?doid=2462326.2462328.

[38] Dana Petcu, Ciprian Craciun, Marian Neagul, Silviu Panica, Beniamino Di Martino, Salvatore Venticinque, Massimiliano Rak, and Rocco Aversa. Architecturing a Sky Computing Platform. In Michel Cezon and Yaron Wolfsthal, editors, *Towards a Service-Based Internet. ServiceWave 2010 Workshops*, pages 1–13, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg. ISBN 978-3-642-22760-8.

[39] Chenhao Qu, Rodrigo N. Calheiros, and Rajkumar Buyya. Auto-Scaling Web Applications in Clouds. *ACM Computing Surveys*, 51(4):1–33, sep 2018. ISSN 0360-0300. doi: 10.1145/3148149. URL https://dl.acm.org/doi/10.1145/3148149.

[40] Clément Quinton, Nicolas Haderer, Romain Rouvoy, and Laurence Duchien. Towards multi-cloud configurations using feature models and ontologies. *Proceedings of the 2013 international workshop on Multi-cloud applications and federated clouds - MultiCloud '13*, page 21, 2013. ISSN 1999-4893. doi: 10.1145/2462326.2462332. URL http://dl.acm.org/citation.cfm?doid=2462326.2462332.

[41] Clément Quinton, Daniel Romero, and Laurence Duchien. SALOON: a platform for selecting and configuring cloud environments. *Software: Practice and Experience*, 46(1):55–78, jan 2016. ISSN 00380644. doi: 10.1002/spe.2311. URL http://doi.wiley.com/10.1002/spe.2311.

[42] E.G. Radhika and G. Sudha Sadasivam. A review on prediction based autoscaling techniques for heterogeneous applications in cloud environment. *Materials Today: Proceedings*, 2021. ISSN 2214-7853. doi: https://doi.org/10.1016/j.matpr.2020.11.789. URL https://www.sciencedirect.com/science/article/pii/S2214785320394657.

[43] Victor Ramperez. Improving elasticity in publish/subscribe systems for large-scale IoT based services. Master's thesis, Universidad Politecnica de Madrid, 2017. URL http://oa.upm.es/51464/.

[44] Víctor Rampérez, Javier Soriano, and David Lizcano. A Multidomain Standards-Based Fog Computing Architecture for Smart Cities. *Wireless Communications and Mobile Computing*, 2018:1–14, sep 2018. ISSN 1530-8669. doi: 10.1155/2018/4019858. URL `https://www.hindawi.com/journals/wcmc/2018/4019858/`.

[45] Víctor Rampérez, Javier Soriano, David Lizcano, and Juan A. Lara. An innovative approach to improve elasticity and performance of message brokers for green smart cities. In *Proceedings of the Fourth International Conference on Engineering & MIS 2018*, ICEMIS '18, New York, NY, USA, 2018. Association for Computing Machinery. ISBN 9781450363921. doi: 10.1145/3234698.3234732. URL `https://doi.org/10.1145/3234698.3234732`.

[46] Víctor Rampérez, Javier Soriano, David Lizcano, and Juan A. Lara. Flas: A combination of proactive and reactive auto-scaling architecture for distributed services. *Future Generation Computer Systems*, 118:56–72, 2021. ISSN 0167-739X. doi: https://doi.org/10.1016/j.future.2020.12.025. URL `https://www.sciencedirect.com/science/article/pii/S0167739X20330879`.

[47] Luis Rodero-Merino, Luis M. Vaquero, Victor Gil, Fermín Galán, Javier Fontán, Rubén S. Montero, and Ignacio M. Llorente. From infrastructure delivery to service management in clouds. *Future Generation Computer Systems*, 26(8):1226–1240, oct 2010. ISSN 0167739X. doi: 10.1016/j.future.2010.02.013. URL `http://dx.doi.org/10.1016/j.future.2010.02.013`.

[48] Mohammad Sadoghi and Hans-Arno Jacobsen. BE-tree. In *Proceedings of the 2011 international conference on Management of data - SIGMOD '11*, page 637, New York, New York, USA, 2011. ACM Press. ISBN 9781450306614. doi: 10.1145/1989323.1989390. URL `http://portal.acm.org/citation.cfm?doid=1989323.1989390`.

[49] L. Toka, G. Dobreff, B. Fodor, and B. Sonkoly. Machine learning-based scaling management for kubernetes edge clusters. *IEEE Transactions on Network and Service Management*, pages 1–1, 2021. doi: 10.1109/TNSM.2021.3052837.

[50] Mattias Ulbrich, Ulrich Geilmann, Aboubakr Achraf, El Ghazi, and Mana Taghdiri. Karlsruhe Reports in Informatics 2011,37. Technical report, 2011.

[51] Sergio Vavassori. *A Novel Approach to Context-Aware Content-Based Middleware*. PhD thesis, Universidad Politécnica de Madrid, 2016. URL `http://oa.upm.es/39615/`.

[52] Sergio Vavassori, Javier Soriano, David Lizcano, and Miguel Jiménez. Explicit Context Matching in Content-Based Publish/Subscribe Systems. *Sensors*, 13(3):2945–2966, mar 2013. ISSN 1424-8220. doi: 10.3390/s130302945. URL `http://www.mdpi.com/1424-8220/13/3/2945`.

[53] Sergio Vavassori, Javier Soriano, and Rafael Fernández. Enabling Large-Scale IoT-Based Services through Elastic Publish/Subscribe. *Sensors*, 17(9):2148, sep 2017. ISSN 1424-8220. doi: 10.3390/s17092148. URL `http://www.mdpi.com/1424-8220/17/9/2148`.

[54] W. Roy Schulte Yefim Natis, Massimo Pezzini, Keith Guttridge. The 5 Steps Toward Pervasive Event-Driven Architecture. Technical Report June, Gartner, 2019.

[55] Albert Yu, PK Agarwal, and Jun Yang. Generating wide-area content-based publish/subscribe workloads. *Network Meets Database (NetDB)*, 2009.

[56] Dimitris Zeginis, Francesco D'Andria, Stefano Bocconi, Jesus Gorronogoitia Cruz, Oriol Collell Martin, Panagiotis Gouvas, Giannis Ledakis, and Konstantinos A. Tarabanis. A user-centric multi-PaaS application management solution for hybrid multi-Cloud scenarios. *Scalable Computing: Practice and Experience*, 14(1):17–32, apr 2013. ISSN 1895-1767. doi: 10.12694/scpe.v14i1.824. URL `http://www.scpe.org/index.php/scpe/article/view/824`.

[57] Yanlong Zhai and Wu Xu. Efficient Bottleneck Detection in Stream Process System Using Fuzzy Logic Model. In *2017 25th Euromicro International Conference on Parallel, Distributed and Network-based Processing (PDP)*, pages 438–445. IEEE, 2017. ISBN 978-1-5090-6058-0. doi: 10.1109/PDP.2017.71. URL `http://ieeexplore.ieee.org/document/7912685/`.

[58] S. Zhang, T. Wu, M. Pan, C. Zhang, and Y. Yu. A-sarsa: A predictive container auto-scaling algorithm based on reinforcement learning. In *2020 IEEE International Conference on Web Services (ICWS)*, pages 489–497, 2020. doi: 10.1109/ICWS49710.2020.00072.

[59] Zhenhuan Gong, Xiaohui Gu, and John Wilkes. PRESS: PRedictive Elastic ReSource Scaling for cloud systems. In *2010 International Conference on Network and Service Management*, pages 9–16. IEEE, oct 2010. ISBN 978-1-4244-8910-7. doi: 10.1109/CNSM.2010.5691343. URL `http://ieeexplore.ieee.org/document/5691343/`.

Appendix A

Vendor-Neutral Cross-Provider Measurable Metrics

In God we trust; all others must bring data.
— William Edwards Deming

On the one hand, the first section of this chapter (A.0.1) presents all the metrics exported by the three main cloud providers (Amazon Web Services or AWS, Google Cloud Provider or GC and Microsoft Azure or M. Azure) for each of the services studied. On the other hand, the second section (A.0.2) presents the complete repository of the proposed vendor-neutral cross-provider metrics, as well as the mapping functions for each of the cloud providers and the rest of the data used in this study.

A.0.1 Metrics exported for each service by the main cloud providers

In the following tables the rows of those metrics that have been used to define the vendor-neutral metrics repository have been highlighted.

Table A.1: RDS (Relational Database Service)

Metric	Unit	Sample period	Description
BinLogDiskUsage	Bytes	1 min	The amount of disk space occupied by binary logs on the master. Applies to MySQL read
BurstBalance	%	1 min	The percent of General Purpose SSD (gp2) burst-bucket I/O credits available.
CPUUtilization	%	1 min	The percentage of CPU utilization.
CPUCreditUsage	Credits (vCPU-minutes)	1 min	[T2 instances] The number of CPU credits spent by the instance for CPU utilization. One CPU credit equals one vCPU running at 100% utilization for one minute or an equivalent combination of vCPUs, utilization, and time (for example, one vCPU running at 50% utilization for two minutes or two vCPUs running at 25% utilization for two minutes). CPU credit metrics are available at a five-minute frequency only. If you specify a period greater than five minutes, use the Sum statistic instead of the Average statistic.
CPUCreditBalance	Credits (vCPU-minutes)	1 min	[T2 instances] The number of earned CPU credits that an instance has accrued since it was launched or started. For T2 Standard, the CPUCreditBalance also includes the number of launch credits that have been accrued. Credits are accrued in the credit balance after they are earned, and removed from the credit balance when they are spent. The credit balance has a maximum limit, determined by the instance size. Once the limit is reached, any new credits that are earned are discarded. For T2 Standard, launch credits do not count towards the limit. The credits in the CPUCreditBalance are available for the instance to spend to burst beyond its baseline CPU utilization. When an instance is running, credits in the CPUCreditBalance do not expire. When the instance stops, the CPUCreditBalance does not persist, and all accrued credits are lost. CPU credit metrics are available at a five-minute frequency only.
DatabaseConnections	Count	1 min	The number of database connections in use.
DiskQueueDepth	Count	1 min	The number of outstanding IOs (read/write requests) waiting to access the disk.
FailedSQLServerAgentJobsCount	Count/Minute	1 min	The number of failed SQL Server Agent jobs during the last minute.
FreeableMemory	Bytes	1 min	The amount of available random access memory.
FreeStorageSpace	Bytes	1 min	The amount of available storage space.

MaximumUsedTransactionIDs	Count	1 min	The maximum transaction ID that has been used. Applies to PostgreSQL.
NetworkReceiveThroughput	Bytes/second	1 min	The incoming (Receive) network traffic on the DB instance, including both customer database traffic and Amazon RDS traffic used for monitoring and replication.
NetworkTransmitThroughput	Bytes/second	1 min	The outgoing (Transmit) network traffic on the DB instance, including both customer database traffic and Amazon RDS traffic used for monitoring and replication.
OldestReplicationSlotLag	Megabytes	1 min	The lagging size of the replica lagging the most in terms of WAL data received. Applies to PostgreSQL.
ReadIOPS	Count/Second	1 min	The average number of disk read I/O operations per second.
ReadLatency	Seconds	1 min	The average amount of time taken per disk I/O operation.
ReadThroughput	Bytes/Second	1 min	The average number of bytes read from disk per second.
ReplicaLag	Seconds	1 min	The amount of time a Read Replica DB instance lags behind the source DB instance. Applies to MySQL, MariaDB, and PostgreSQL Read Replicas.
ReplicationSlotDiskUsage	Megabytes	1 min	The disk space used by replication slot files. Applies to PostgreSQL.
SwapUsage	Bytes	1 min	The amount of swap space used on the DB instance. This metric is not available for SQL Server.
TransactionLogsDiskUsage	Megabytes	1 min	The disk space used by transaction logs. Applies to PostgreSQL.
TransactionLogsGeneration	Megabytes/second	1 min	The size of transaction logs generated per second. Applies to PostgreSQL.
WriteIOPS	Count/Second	1 min	The average number of disk write I/O operations per second.
WriteLatency	Seconds	1 min	The average amount of time taken per disk I/O operation.
WriteThroughput	Bytes/Second	1 min	The average number of bytes written to disk per second.

Table A.2: EBS (Elastic Block Service)

Metric	**Unit**	**Sample period**	**Description**

VolumeReadBytes	Bytes	1 min	Provides information on the read operations in a specified period of time. The Sum statistic reports the total number of bytes transferred during the period. The Average statistic reports the average size of each read operation during the period, except on volumes attached to a Nitro-based instance, where the average represents the average over the specified period. The SampleCount statistic reports the total number of read operations during the period, except on volumes attached to a Nitro-based instance, where the sample count represents the number of data points used in the statistical calculation. For Xen instances, data is reported only when there is read activity on the volume. The Minimum and Maximum statistics on this metric are supported only by volumes attached to Nitro-based instances. Units: Bytes
VolumeWriteBytes	Bytes	1 min	Provides information on the write operations in a specified period of time. The Sum statistic reports the total number of bytes transferred during the period. The Average statistic reports the average size of each write operation during the period, except on volumes attached to a Nitro-based instance, where the average represents the average over the specified period. The SampleCount statistic reports the total number of write operations during the period, except on volumes attached to a Nitro-based instance, where the sample count represents the number of data points used in the statistical calculation. For Xen instances, data is reported only when there is write activity on the volume. The Minimum and Maximum statistics on this metric are supported only by volumes attached to Nitro-based instances. Units: Bytes
VolumeReadOps	Count	1 min	The total number of read operations in a specified period of time. To calculate the average read operations per second (read IOPS) for the period, divide the total read operations in the period by the number of seconds in that period. The Minimum and Maximum statistics on this metric are supported only by volumes attached to Nitro-based instances. Units: Count
VolumeWriteOps	Count	1 min	The total number of write operations in a specified period of time. To calculate the average write operations per second (write IOPS) for the period, divide the total write operations in the period by the number of seconds in that period. The Minimum and Maximum statistics on this metric are supported only by volumes attached to Nitro-based instances. Units: Count

VolumeTotalReadTime	Seconds	1 min	The total number of seconds spent by all read operations that completed in a specified period of time. If multiple requests are submitted at the same time, this total could be greater than the length of the period. For example, for a period of 5 minutes (300 seconds): if 700 operations completed during that period, and each operation took 1 second, the value would be 700 seconds. For Xen instances, data is reported only when there is read activity on the volume. The Average statistic on this metric is not relevant for volumes attached to Nitro-based instances. The Minimum and Maximum statistics on this metric are supported only by volumes attached to Nitro-based instances. Units: Seconds
VolumeTotalWriteTime	Seconds	1 min	The total number of seconds spent by all write operations that completed in a specified period of time. If multiple requests are submitted at the same time, this total could be greater than the length of the period. For example, for a period of 5 minutes (300 seconds): if 700 operations completed during that period, and each operation took 1 second, the value would be 700 seconds. For Xen instances, data is reported only when there is write activity on the volume. The Average statistic on this metric is not relevant for volumes attached to Nitro-based instances. The Minimum and Maximum statistics on this metric are supported only by volumes attached to Nitro-based instances. Units: Seconds
VolumeIdleTime	Seconds	1 min	The total number of seconds in a specified period of time when no read or write operations were submitted. The Average statistic on this metric is not relevant for volumes attached to Nitro-based instances. The Minimum and Maximum statistics on this metric are supported only by volumes attached to Nitro-based instances. Units: Seconds
VolumeQueueLength	Count	1 min	The number of read and write operation requests waiting to be completed in a specified period of time. The Sum statistic on this metric is not relevant for volumes attached to Nitro-based instances. The Minimum and Maximum statistics on this metric are supported only by volumes attached to Nitro-based instances. Units: Count

VolumeThroughputPercentage	%	1 min	Used with Provisioned IOPS SSD volumes only. The percentage of I/O operations per second (IOPS) delivered of the total IOPS provisioned for an Amazon EBS volume. Provisioned IOPS SSD volumes deliver within 10 percent of the provisioned IOPS performance 99.9 percent of the time over a given year. During a write, if there are no other pending I/O requests in a minute, the metric value will be 100 percent. Also, a volume's I/O performance may become degraded temporarily due to an action you have taken (for example, creating a snapshot of a volume during peak usage, running the volume on a non-EBS-optimized instance, or accessing data on the volume for the first time). Units: Percent
VolumeConsumedReadWriteOps	Count	1 min	Used with Provisioned IOPS SSD volumes only. The total amount of read and write operations (normalized to 256K capacity units) consumed in a specified period of time. I/O operations that are smaller than 256K each count as 1 consumed IOPS. I/O operations that are larger than 256K are counted in 256K capacity units. For example, a 1024K I/O would count as 4 consumed IOPS. Units: Count
BurstBalance	%	1 min	Used with General Purpose SSD (gp2), Throughput Optimized HDD (st1), and Cold HDD (sc1) volumes only. Provides information about the percentage of I/O credits (for gp2) or throughput credits (for st1 and sc1) remaining in the burst bucket. Data is reported to CloudWatch only when the volume is active. If the volume is not attached, no data is reported. The Sum statistic on this metric is not relevant for volumes attached to Nitro-based instances. Units: Percent

Table A.3: ApplicationELB (Elastic Load Balancer)

Metric	Unit	Sample period	Description
ActiveConnectionCount	Count	1 min	The total number of concurrent TCP connections active from clients to the load balancer and from the load balancer to targets. Reporting criteria: There is a nonzero value Statistics: The most useful statistic is Sum. Dimensions LoadBalancer

ClientTLSNegotiationErrorCount	Count	1 min	The number of TLS connections initiated by the client that did not establish a session with the load balancer. Possible causes include a mismatch of ciphers or protocols. Reporting criteria: There is a nonzero value Statistics: The most useful statistic is Sum. Dimensions LoadBalancer AvailabilityZone, LoadBalancer
ConsumedLCUs	Count	1 min	The number of load balancer capacity units (LCU) used by your load balancer. You pay for the number of LCUs that you use per hour. For more information, see Elastic Load Balancing Pricing. Reporting criteria: Always reported Statistics: All Dimensions LoadBalancer
HTTP_Fixed_Response_Count	Count	1 min	The number of fixed-response actions that were successful. Reporting criteria: There is a nonzero value Statistics: The only meaningful statistic is Sum. Dimensions LoadBalancer
HTTP_Redirect_Count	Count	1 min	The number of redirect actions that were successful. Reporting criteria: There is a nonzero value Statistics: The only meaningful statistic is Sum. Dimensions LoadBalancer
HTTP_Redirect_Url_Limit_Exceeded_-Count	Count	1 min	The number of redirect actions that couldn't be completed because the URL in the response location header is larger than 8K. Reporting criteria: There is a nonzero value Statistics: The only meaningful statistic is Sum. Dimensions LoadBalancer
HTTPCode_ELB_3XX_Count	Count	1 min	The number of HTTP 3XX redirection codes that originate from the load balancer. Reporting criteria: There is a nonzero value Statistics: The only meaningful statistic is Sum. Dimensions LoadBalancer
HTTPCode_ELB_4XX_Count	Count	1 min	The number of HTTP 4XX client error codes that originate from the load balancer. Client errors are generated when requests are malformed or incomplete. These requests have not been received by the target. This count does not include any response codes generated by the targets. Reporting criteria: There is a nonzero value Statistics: The most useful statistic is Sum. Note that Minimum, Maximum, and Average all return 1. Dimensions LoadBalancer AvailabilityZone, LoadBalancer
HTTPCode_ELB_5XX_Count	Count	1 min	The number of HTTP 5XX server error codes that originate from the load balancer. This count does not include any response codes generated by the targets. Reporting criteria: There is a nonzero value Statistics: The most useful statistic is Sum. Note that Minimum, Maximum, and Average all return 1. Dimensions LoadBalancer AvailabilityZone, LoadBalancer

IPv6ProcessedBytes	Bytes	1 min	The total number of bytes processed by the load balancer over IPv6. Reporting criteria: There is a nonzero value Statistics: The most useful statistic is Sum. Dimensions LoadBalancer
IPv6RequestCount	Count	1 min	The number of IPv6 requests received by the load balancer. Reporting criteria: There is a nonzero value Statistics: The most useful statistic is Sum. Note that Minimum, Maximum, and Average all return 1. Dimensions LoadBalancer AvailabilityZone, LoadBalancer TargetGroup, LoadBalancer TargetGroup, AvailabilityZone, LoadBalancer
NewConnectionCount	Count	1 min	The total number of new TCP connections established from clients to the load balancer and from the load balancer to targets. Reporting criteria: There is a nonzero value Statistics: The most useful statistic is Sum. Dimensions LoadBalancer
ProcessedBytes	Bytes	1 min	The total number of bytes processed by the load balancer over IPv4 and IPv6. Reporting criteria: There is a nonzero value Statistics: The most useful statistic is Sum. Dimensions LoadBalancer
RejectedConnectionCount	Count	1 min	The number of connections that were rejected because the load balancer had reached its maximum number of connections. Reporting criteria: There is a nonzero value Statistics: The most useful statistic is Sum. Dimensions LoadBalancer AvailabilityZone, LoadBalancer
RequestCount	Count	1 min	The number of requests processed over IPv4 and IPv6. This count includes only the requests with a response generated by a target of the load balancer. Reporting criteria: Always reported Statistics: The most useful statistic is Sum. Dimensions LoadBalancer AvailabilityZone, LoadBalancer TargetGroup, LoadBalancer TargetGroup, AvailabilityZone, LoadBalancer
RuleEvaluations	Count	1 min	The number of rules processed by the load balancer given a request rate averaged over an hour. Reporting criteria: There is a nonzero value Statistics: The most useful statistic is Sum. Dimensions LoadBalancer
StandardProcessedBytes	Bytes	1 min	The total number of bytes processed by the load balancer over IPv4 and IPv6 for instances and IP addresses registered as targets. Reporting criteria: There is a nonzero value Statistics: The most useful statistic is Sum. Dimensions LoadBalancer

HealthyHostCount	Count	1 min	The number of targets that are considered healthy. Reporting criteria: Reported if health checks are enabled Statistics: The most useful statistics are Average, Minimum, and Maximum. Dimensions TargetGroup, LoadBalancer TargetGroup, AvailabilityZone, LoadBalancer
HTTPCode_Target_2XX_Count, HTTPCode_Target_3XX_Count, HTTPCode_Target_4XX_Count, HTTPCode_Target_5XX_Count	Count	1 min	The number of HTTP response codes generated by the targets. This does not include any response codes generated by the load balancer. Reporting criteria: There is a nonzero value Statistics: The most useful statistic is Sum. Note that Minimum, Maximum, and Average all return 1. Dimensions LoadBalancer AvailabilityZone, LoadBalancer TargetGroup, LoadBalancer TargetGroup, AvailabilityZone, LoadBalancer
NonStickyRequestCount	Count	1 min	The number of requests where the load balancer chose a new target because it couldn't use an existing sticky session. For example, the request was the first request from a new client and no stickiness cookie was presented, a stickiness cookie was presented but it did not specify a target that was registered with this target group, the stickiness cookie was malformed or expired, or an internal error prevented the load balancer from reading the stickiness cookie. Reporting criteria: Stickiness is enabled on the target group. Statistics: The only meaningful statistic is Sum.
RequestCountPerTarget	Count	1 min	The average number of requests received by each target in a target group. You must specify the target group using the TargetGroup dimension. This metric does not apply if the target is a Lambda function. Reporting criteria: Always reported Statistics: The only valid statistic is Sum. Note that this represents the average not the sum. Dimensions TargetGroup TargetGroup, LoadBalancer
TargetConnectionErrorCount	Count	1 min	The number of connections that were not successfully established between the load balancer and target. This metric does not apply if the target is a Lambda function. Reporting criteria: There is a nonzero value Statistics: The most useful statistic is Sum. Dimensions LoadBalancer AvailabilityZone, LoadBalancer TargetGroup, LoadBalancer TargetGroup, AvailabilityZone, LoadBalancer

TargetResponseTime	Seconds	1 min	The time elapsed, in seconds, after the request leaves the load balancer until a response from the target is received. This is equivalent to the target_processing_time field in the access logs. Reporting criteria: There is a nonzero value Statistics: The most useful statistics are Average and pNN.NN (percentiles). Dimensions LoadBalancer AvailabilityZone, LoadBalancer TargetGroup, LoadBalancer TargetGroup, AvailabilityZone, LoadBalancer
TargetTLSNegotiationErrorCount	Count	1 min	The number of TLS connections initiated by the load balancer that did not establish a session with the target. Possible causes include a mismatch of ciphers or protocols. This metric does not apply if the target is a Lambda function. Reporting criteria: There is a nonzero value Statistics: The most useful statistic is Sum. Dimensions LoadBalancer AvailabilityZone, LoadBalancer TargetGroup, LoadBalancer TargetGroup, AvailabilityZone, LoadBalancer
UnHealthyHostCount	Count	1 min	The number of targets that are considered unhealthy. Reporting criteria: Reported if health checks are enabled Statistics: The most useful statistics are Average, Minimum, and Maximum. Dimensions TargetGroup, LoadBalancer TargetGroup, AvailabilityZone, LoadBalancer

Table A.4: NetworkELB (Elastic Load Balancer)

Metric	Unit	Sample period	Description
ActiveFlowCount	Count	1 min	The total number of concurrent flows (or connections) from clients to targets. This metric includes connections in the SYN_SENT and ESTABLISHED states. TCP connections are not terminated at the load balancer, so a client opening a TCP connection to a target counts as a single flow. Statistics: The most useful statistics are Average, Maximum, and Minimum.
ActiveFlowCount_TLS	Count	1 min	The total number of concurrent TLS flows (or connections) from clients to targets. This metric includes only connections in the ESTABLISHED states. Statistics: The most useful statistics are Average, Maximum, and Minimum.
ClientTLSNegotiationErrorCount	Count	1 min	The total number of TLS handshakes that failed during negotiation between a client and a TLS listener. Statistics: The most useful statistic is Sum.

ConsumedLCUs	Count	1 min	The number of load balancer capacity units (LCU) used by your load balancer. You pay for the number of LCUs that you use per hour. For more information, see Elastic Load Balancing Pricing.
HealthyHostCount	Count	1 min	The number of targets that are considered healthy. Statistics: The most useful statistics are Maximum and Minimum.
NewFlowCount	Count	1 min	The total number of new flows (or connections) established from clients to targets in the time period. Statistics: The most useful statistic is Sum.
NewFlowCount_TLS	Count	1 min	The total number of new TLS flows (or connections) established from clients to targets in the time period. Statistics: The most useful statistic is Sum.
ProcessedBytes	Bytes	1 min	The total number of bytes processed by the load balancer, including TCP/IP headers. Statistics: The most useful statistic is Sum.
ProcessedBytes_TLS	Bytes	1 min	The total number of bytes processed by TLS listeners. Statistics: The most useful statistic is Sum.
TargetTLSNegotiationErrorCount	Count	1 min	The total number of TLS handshakes that failed during negotiation between a TLS listener and a target. Statistics: The most useful statistic is Sum.
TCP_Client_Reset_Count	Count	1 min	The total number of reset (RST) packets sent from a client to a target. These resets are generated by the client and forwarded by the load balancer. Statistics: The most useful statistic is Sum.
TCP_ELB_Reset_Count	Count	1 min	The total number of reset (RST) packets generated by the load balancer. Statistics: The most useful statistic is Sum.
TCP_Target_Reset_Count	Count	1 min	The total number of reset (RST) packets sent from a target to a client. These resets are generated by the target and forwarded by the load balancer. Statistics: The most useful statistic is Sum.
UnHealthyHostCount	Count	1 min	The number of targets that are considered unhealthy. Statistics: The most useful statistics are Maximum and Minimum.

Table A.5: S3 (Simple Storage Service)

Metric	Unit	Sample period	Description
BucketSizeBytes	Bytes	1 min	The amount of data in bytes stored in a bucket in the STANDARD storage class, INTELLIGENT_TIERING storage class, Standard - Infrequent Access (STANDARD_IA) storage class, OneZone - Infrequent Access (ONEZONE_IA), Reduced Redundancy Storage (RRS) class, or Glacier (GLACIER) storage class. Valid storage type filters: StandardStorage, IntelligentTieringStorage, GlacierS3ObjectOverhead, StandardIAStorage, StandardIAObjectOverhead, StandardIAStorage, StandardIAObjectOverhead, OneZoneIAStorage, OneZoneIAObjectOverhead, ReducedRedundancyStorage, GlacierStorage, and GlacierObjectOverhead (see the StorageType dimension) Units: Bytes Valid statistics: Average
NumberOfObjects	Count	1 min	The total number of objects (includes all objects including all their versions) stored in a bucket for all storage classes Valid storage type filters: AllStorageTypes (see the StorageType dimension) Units: Count Valid statistics: Average
AllRequests	Count	1 min	The total number of HTTP requests made to an Amazon S3 bucket, regardless of type. If you're using a metrics configuration with a filter, then this metric only returns the HTTP requests made to the objects in the bucket that meet the filter's requirements. Units: Count Valid statistics: Sum
GetRequests	Count	1 min	The number of HTTP GET requests made for objects in an Amazon S3 bucket. This doesn't include list operations. Units: Count Valid statistics: Sum Note Paginated list-oriented requests, like List Multipart Uploads, List Parts, Get Bucket Object versions, and others, are not included in this metric.
PutRequests	Count	1 min	The number of HTTP PUT requests made for objects in an Amazon S3 bucket. Units: Count Valid statistics: Sum
DeleteRequests	Count	1 min	The number of HTTP DELETE requests made for objects in an Amazon S3 bucket. This also includes Delete Multiple Objects requests. This metric shows the number of requests, not the number of objects deleted. Units: Count Valid statistics: Sum
HeadRequests	Count	1 min	The number of HTTP HEAD requests made to an Amazon S3 bucket. Units: Count Valid statistics: Sum

PostRequests	Count	1 min	The number of HTTP POST requests made to an Amazon S3 bucket. Units: Count Valid statistics: Sum Note Delete Multiple Objects and SELECT Object Content requests are not included in this metric.
SelectRequests	Count	1 min	The number of Amazon S3 SELECT Object Content requests made for objects in an Amazon S3 bucket. Units: Count Valid statistics: Sum
SelectScannedBytes	Bytes	1 min	The number of bytes of data scanned with Amazon S3 SELECT Object Content requests in an Amazon S3 bucket. Units: Bytes Valid statistics: Average (bytes per request), Sum (bytes per period), Sample Count, Min, Max
SelectReturnedBytes	Bytes	1 min	The number of bytes of data returned with Amazon S3 SELECT Object Content requests in an Amazon S3 bucket. Units: Bytes Valid statistics: Average (bytes per request), Sum (bytes per period), Sample Count, Min, Max
ListRequests	Count	1 min	The number of HTTP requests that list the contents of a bucket. Units: Count Valid statistics: Sum
BytesDownloaded	Bytes	1 min	The number bytes downloaded for requests made to an Amazon S3 bucket, where the response includes a body. Units: Bytes Valid statistics: Average (bytes per request), Sum (bytes per period), Sample Count, Min, Max
BytesUploaded	Bytes	1 min	The number bytes uploaded that contain a request body, made to an Amazon S3 bucket. Units: Bytes Valid statistics: Average (bytes per request), Sum (bytes per period), Sample Count, Min, Max
4xxErrors	Count	1 min	The number of HTTP 4xx client error status code requests made to an Amazon S3 bucket with a value of either 0 or 1. The average statistic shows the error rate, and the sum statistic shows the count of that type of error, during each period. Units: Count Valid statistics: Average (reports per request), Sum (reports per period), Min, Max, Sample Count
5xxErrors	Count	1 min	The number of HTTP 5xx server error status code requests made to an Amazon S3 bucket with a value of either 0 or 1. The average statistic shows the error rate, and the sum statistic shows the count of that type of error, during each period. Units: Counts Valid statistics: Average (reports per request), Sum (reports per period), Min, Max, Sample Count
FirstByteLatency	Milliseconds	1 min	The per-request time from the complete request being received by an Amazon S3 bucket to when the response starts to be returned. Units: Milliseconds Valid statistics: Average, Sum, Min, Max, Sample Count

TotalRequestLatency	Milliseconds	1 min	The elapsed per-request time from the first byte received to the last byte sent to an Amazon S3 bucket. This includes the time taken to receive the request body and send the response body, which is not included in FirstByteLatency. Units: Milliseconds Valid statistics: Average, Sum, Min, Max, Sample Count

Table A.6: EFS (Elastic File System)

Metric	**Unit**	**Sample period**	**Description**
BurstCreditBalance	Bytes	1 min	The number of burst credits that a file system has. Burst credits allow a file system to burst to throughput levels above a file system's baseline level for periods of time. For more information, see Throughput scaling in Amazon EFS. The Minimum statistic is the smallest burst credit balance for any minute during the period. The Maximum statistic is the largest burst credit balance for any minute during the period. The Average statistic is the average burst credit balance during the period. Units: Bytes Valid statistics: Minimum, Maximum, Average
ClientConnections	Count	1 min	The number of client connections to a file system. When using a standard client, there is one connection per mounted Amazon EC2 instance. Note To calculate the average ClientConnections for periods greater than one minute, divide the Sum statistic by the number of minutes in the period. Units: Count of client connections Valid statistics: Sum
DataReadIOBytes	Bytes	1 min	The number of bytes for each file system read operation. The Sum statistic is the total number of bytes associated with read operations. The Minimum statistic is the size of the smallest read operation during the period. The Maximum statistic is the size of the largest read operation during the period. The Average statistic is the average size of read operations during the period. The SampleCount statistic provides a count of read operations. Units: Bytes for Minimum, Maximum, Average, and Sum. Count for SampleCount. Valid statistics: Minimum, Maximum, Average, Sum, SampleCount

DataWriteIOBytes	Bytes	1 min	The number of bytes for each file write operation. The Sum statistic is the total number of bytes associated with write operations. The Minimum statistic is the size of the smallest write operation during the period. The Maximum statistic is the size of the largest write operation during the period. The Average statistic is the average size of write operations during the period. The SampleCount statistic provides a count of write operations. Units: Bytes are the units for the Minimum, Maximum, Average, and Sum statistics. Count for SampleCount. Valid statistics: Minimum, Maximum, Average, Sum, SampleCount
MetadataIOBytes	Bytes	1 min	The number of bytes for each metadata operation. The Sum statistic is the total number of bytes associated with metadata operations. The Minimum statistic is the size of the smallest metadata operation during the period. The Maximum statistic is the size of the largest metadata operation during the period. The Average statistic is the size of the average metadata operation during the period. The SampleCount statistic provides a count of metadata operations. Units: Bytes are the units for the Minimum, Maximum, Average, and Sum statistics. Count for SampleCount. Valid statistics: Minimum, Maximum, Average, Sum, SampleCount
PercentIOLimit	%	1 min	Shows how close a file system is to reaching the I/O limit of the General Purpose performance mode. If this metric is at 100% more often than not, consider moving your application to a file system using the Max I/O performance mode. Note This metric is only submitted for file systems using the General Purpose performance mode. Units: Percent
PermittedThroughput	Bytes/s	1 min	The maximum amount of throughput a file system is allowed. For file systems in the Provisioned Throughput mode, this value is the same as the provisioned throughput. For file systems in the Bursting Throughput mode, this value is a function of the file system size and BurstCreditBalance. For more information, see Amazon EFS Performance. The Minimum statistic is the smallest throughput permitted for any minute during the period. The Maximum statistic is the highest throughput permitted for any minute during the period. The Average statistic is the average throughput permitted during the period. Units: Bytes per second Valid statistics: Minimum, Maximum, Average

TotalIOBytes	Bytes	1 min	The number of bytes for each file system operation, including data read, data write, and metadata operations. The Sum statistic is the total number of bytes associated with all file system operations. The Minimum statistic is the size of the smallest operation during the period. The Maximum statistic is the size of the largest operation during the period. The Average statistic is the average size of an operation during the period. The SampleCount statistic provides a count of all operations. Note To calculate the average operations per second for a period, divide the SampleCount statistic by the number of seconds in the period. To calculate the average throughput (Bytes per second) for a period, divide the Sum statistic by the number of seconds in the period. Units: Bytes for Minimum, Maximum, Average, and Sum statistics. Count for SampleCount. Valid statistics: Minimum, Maximum, Average, Sum, SampleCount

Table A.7: Lambda

Metric	**Unit**	**Sample period**	**Description**
Invocations	Count	1 min	Measures the number of times a function is invoked in response to an event or invocation API call. This replaces the deprecated RequestCount metric. This includes successful and failed invocations, but does not include throttled attempts. This equals the billed requests for the function. Note that AWS Lambda only sends these metrics to CloudWatch if they have a nonzero value.
Errors	Count	1 min	Measures the number of invocations that failed due to errors in the function (response code 4XX). This replaces the deprecated ErrorCount metric. Failed invocations may trigger a retry attempt that succeeds. This includes:Handled exceptions (for example, context.fail(error)) Unhandled exceptions causing the code to exit Out of memory exceptions Timeouts Permissions errorsThis does not include invocations that fail due to invocation rates exceeding default concurrent limits (error code 429) or failures due to internal service errors (error code 500).

DeadLetterErrors	Count	1 min	Incremented when Lambda is unable to write the failed event payload to your configured Dead Letter Queues. This could be due to the following:Permissions errors Throttles from downstream services Misconfigured resources Timeouts
Duration	Milliseconds	1 min	Measures the elapsed wall clock time from when the function code starts executing as a result of an invocation to when it stops executing. The maximum data point value possible is the function timeout configuration. The billed duration will be rounded up to the nearest 100 millisecond. Note that AWS Lambda only sends these metrics to CloudWatch if they have a nonzero value.
Throttles	Count	1 min	Measures the number of Lambda function invocation attempts that were throttled due to invocation rates exceeding the customer's concurrent limits (error code 429). Failed invocations may trigger a retry attempt that succeeds.
IteratorAge	Milliseconds	1 min	Emitted for stream-based invocations only (functions triggered by an Amazon DynamoDB stream or Kinesis stream). Measures the age of the last record for each batch of records processed. Age is the difference between the time Lambda received the batch, and the time the last record in the batch was written to the stream.
ConcurrentExecutions	Count	1 min	Emitted as an aggregate metric for all functions in the account, and for functions that have a custom concurrency limit specified. Not applicable for versions or aliases. Measures the sum of concurrent executions for a given function at a given point in time. Must be viewed as an average metric if aggregated across a time period.
UnreservedConcurrentExecutions	Count	1 min	Emitted as an aggregate metric for all functions in the account only. Not applicable for functions, versions, or aliases. Represents the sum of the concurrency of the functions that do not have a custom concurrency limit specified. Must be viewed as an average metric if aggregated across a time period.

Table A.8: ECS (Elastic Container Service

Metric	**Unit**	**Sample period**	**Description**

CPUReservation	Percent	1 min	The percentage of CPU units that are reserved by running tasks in the cluster.Cluster CPU reservation (this metric can only be filtered by ClusterName) is measured as the total CPU units that are reserved by Amazon ECS tasks on the cluster, divided by the total CPU units that were registered for all of the container instances in the cluster. This metric is only used for tasks using the EC2 launch type.Valid dimensions: ClusterName.Valid statistics: Average, Minimum, Maximum, Sum, Sample Count. The most useful statistic is Average.
CPUUtilization	Percent	1 min	The percentage of CPU units that are used in the cluster or service.Cluster CPU utilization (metrics that are filtered by ClusterName without ServiceName) is measured as the total CPU units in use by Amazon ECS tasks on the cluster, divided by the total CPU units that were registered for all of the container instances in the cluster. Cluster CPU utilization metrics are only used for tasks using the EC2 launch type.Service CPU utilization (metrics that are filtered by ClusterName and ServiceName) is measured as the total CPU units in use by the tasks that belong to the service, divided by the total number of CPU units that are reserved for the tasks that belong to the service. Service CPU utilization metrics are used for tasks using both the Fargate and the EC2 launch type.Valid dimensions: ClusterName, ServiceName.Valid statistics: Average, Minimum, Maximum, Sum, Sample Count. The most useful statistic is Average.
MemoryReservation	Percent	1 min	The percentage of memory that is reserved by running tasks in the cluster.Cluster memory reservation (this metric can only be filtered by ClusterName) is measured as the total memory that is reserved by Amazon ECS tasks on the cluster, divided by the total amount of memory that was registered for all of the container instances in the cluster. This metric is only used for tasks using the EC2 launch type.Valid dimensions: ClusterName.Valid statistics: Average, Minimum, Maximum, Sum, Sample Count. The most useful statistic is Average.

MemoryUtilization	Percent	1 min	The percentage of memory that is used in the cluster or service.Cluster memory utilization (metrics that are filtered by ClusterName without ServiceName) is measured as the total memory in use by Amazon ECS tasks on the cluster, divided by the total amount of memory that was registered for all of the container instances in the cluster. Cluster memory utilization metrics are only used for tasks using the EC2 launch type.Service memory utilization (metrics that are filtered by ClusterName and ServiceName) is measured as the total memory in use by the tasks that belong to the service, divided by the total memory that is reserved for the tasks that belong to the service. Service memory utilization metrics are used for tasks using both the Fargate and EC2 launch types.Valid dimensions: ClusterName, ServiceName.Valid statistics: Average, Minimum, Maximum, Sum, Sample Count. The most useful statistic is Average.
GPUReservation	Percent	1 min	The percentage of total available GPUs that are reserved by running tasks in the cluster.Cluster GPU reservation is measured as the number of GPUs reserved by Amazon ECS tasks on the cluster, divided by the total number of GPUs that was available on all of the GPU-enabled container instances in the cluster.Valid dimensions: ClusterName.Valid statistics: Average, Minimum, Maximum, Sum, Sample Count. The most useful statistic is Average.
Cluster CPU reservation	Percent	1 min	(Total CPU units reserved by tasks in cluster) x 100Cluster CPU reservation = (Total CPU units registered by container instances in cluster)
Cluster memory reservation	Percent	1 min	(Total MiB of memory reserved by tasks in cluster x 100)Cluster memory reservation = (Total MiB of memory registered by container instances in cluster)
Cluster GPU reservation	Percent	1 min	(Total GPUs reserved by tasks in cluster x 100)Cluster GPU reservation = (Total GPUs registered by container instances in cluster)
Cluster CPU utilization	Percent	1 min	(Total CPU units used by tasks in cluster) x 100Cluster CPU utilization = (Total CPU units registered by container instances in cluster)
Cluster memory utilization	Percent	1 min	(Total MiB of memory used by tasks in cluster x 100)Cluster memory utilization = (Total MiB of memory registered by container instances in cluster)
Service CPU utilization	Percent	1 min	(Total CPU units used by tasks in service) x 100Service CPU utilization = (Total CPU units specified in task definition) x (number of tasks in service)

Service memory utilization	Percent	1 min	(Total MiB of memory used by tasks in service) x 100Service memory utilization = (Total MiB of memory specified in task definition) x (number of tasks in service)

Table A.9: DX (Direct Connect)

Metric	Unit	Sample period	Description
ConnectionState	Boolean	1 min	The state of the connection. 0 indicates DOWN and 1 indicates UP.
ConnectionBpsEgress	Bits per second	1 min	The bitrate for outbound data from the AWS side of the connection. The number reported is the aggregate over the specified time period (5 minutes by default, 1 minute minimum).
ConnectionBpsIngress	Bits per second	1 min	The bitrate for inbound data to the AWS side of the connection. The number reported is the aggregate over the specified time period (5 minutes by default, 1 minute minimum).
ConnectionPpsEgress	Packets per second	1 min	The packet rate for outbound data from the AWS side of the connection. The number reported is the aggregate over the specified time period (5 minutes by default, 1 minute minimum).
ConnectionPpsIngress	Packets per second	1 min	The packet rate for inbound data to the AWS side of the connection. The number reported is the aggregate over the specified time period (5 minutes by default, 1 minute minimum).
ConnectionCRCErrorCount	Integer	1 min	The number of times cyclic redundancy check (CRC) errors are observed for the data received at the connection.
ConnectionLightLevelTx	dBm	1 min	Indicates the health of the fiber connection for egress (outbound) traffic from the AWS side of the connection. This metric is available for connections with 10-Gbps port speeds only.
ConnectionLightLevelRx	dBm	1 min	Indicates the health of the fiber connection for ingress (inbound) traffic to the AWS side of the connection. This metric is available for connections with 10-Gbps port speeds only.

Table A.10: EMR (Elastic Map Reduce)

Metric	Unit	Sample period	Description
Cluster Status			
IsIdle	Boolean	5 min	Indicates that a cluster is no longer performing work, but is still alive and accruing charges. It is set to 1 if no tasks are running and no jobs are running, and set to 0 otherwise. This value is checked at five-minute intervals and a value of 1 indicates only that the cluster was idle when checked, not that it was idle for the entire five minutes. To avoid false positives, you should raise an alarm when this value has been 1 for more than one consecutive 5-minute check. For example, you might raise an alarm on this value if it has been 1 for thirty minutes or longer. Use case: Monitor cluster performance
ContainerAllocated	Count	5 min	The number of resource containers allocated by the ResourceManager. Use case: Monitor cluster progress
ContainerReserved	Count	5 min	The number of containers reserved. Use case: Monitor cluster progress
ContainerPending	Count	5 min	The number of containers in the queue that have not yet been allocated. Use case: Monitor cluster progress
ContainerPendingRatio	Count	5 min	The ratio of pending containers to containers allocated (ContainerPendingRatio = ContainerPending / ContainerAllocated). If ContainerAllocated = 0, then ContainerPendingRatio = ContainerPending. The value of ContainerPendingRatio represents a number, not a percentage. This value is useful for scaling cluster resources based on container allocation behavior.
AppsCompleted	Count	5 min	The number of applications submitted to YARN that have completed. Use case: Monitor cluster progress
AppsFailed	Count	5 min	The number of applications submitted to YARN that have failed to complete. Use case: Monitor cluster progress, Monitor cluster health
AppsKilled	Count	5 min	The number of applications submitted to YARN that have been killed. Use case: Monitor cluster progress, Monitor cluster health
AppsPending	Count	5 min	The number of applications submitted to YARN that are in a pending state. Use case: Monitor cluster progress
AppsRunning	Count	5 min	The number of applications submitted to YARN that are running. Use case: Monitor cluster progress

AppsSubmitted	Count	5 min	The number of applications submitted to YARN. Use case: Monitor cluster progress
Node Status			
CoreNodesRunning	Count	5 min	The number of core nodes working. Data points for this metric are reported only when a corresponding instance group exists. Use case: Monitor cluster health
CoreNodesPending	Count	5 min	The number of core nodes waiting to be assigned. All of the core nodes requested may not be immediately available; this metric reports the pending requests. Data points for this metric are reported only when a corresponding instance group exists. Use case: Monitor cluster health
LiveDataNodes	Percent	5 min	The percentage of data nodes that are receiving work from Hadoop. Use case: Monitor cluster health
MRTotalNodes	Count	5 min	The number of nodes presently available to MapReduce jobs. Equivalent to YARN metric mapred.resourcemanager.TotalNodes. Use ase: Monitor cluster progress
MRActiveNodes	Count	5 min	The number of nodes presently running MapReduce tasks or jobs. Equivalent to YARN metric mapred.resourcemanager.NoOfActiveNodes. Use case: Monitor cluster progress
MRLostNodes	Count	5 min	The number of nodes allocated to MapReduce that have been marked in a LOST state. Equivalent to YARN metric mapred.resourcemanager.NoOfLostNodes. Use case: Monitor cluster health, Monitor cluster progress
MRUnhealthyNodes	Count	5 min	The number of nodes available to MapReduce jobs marked in an UNHEALTHY state. Equivalent to YARN metric mapred.resourcemanager.NoOfUnhealthyNodes. Use case: Monitor cluster progress
MRDecommissionedNodes	Count	5 min	The number of nodes allocated to MapReduce applications that have been marked in a DECOMMISSIONED state. Equivalent to YARN metric mapred.resourcemanager.NoOfDecommissionedNodes. Use ase: Monitor cluster health, Monitor cluster progress
MRRebootedNodes	Count	5 min	The number of nodes available to MapReduce that have been rebooted and marked in a REBOOTED state. Equivalent to YARN metric mapred.resourcemanager.NoOfRebootedNodes. Use case: Monitor cluster health, Monitor cluster progress
IO			

S3BytesWritten	Count	5 min	The number of bytes written to Amazon S3. Use case: Analyze cluster performance, Monitor cluster progress
S3BytesRead	Count	5 min	The number of bytes read from Amazon S3. Use case: Analyze cluster performance, Monitor cluster progress
HDFSUtilization	Percent	5 min	The percentage of HDFS storage currently used. Use case: Analyze cluster performance
HDFSBytesRead	Count	5 min	The number of bytes read from HDFS. This metric aggregates MapReduce jobs only, and does not apply for other workloads on EMR. Use case: Analyze cluster performance, Monitor cluster progress
HDFSBytesWritten	Count	5 min	The number of bytes written to HDFS. This metric aggregates MapReduce jobs only, and does not apply for other workloads on EMR. Use case: Analyze cluster performance, Monitor cluster progress
MissingBlocks	Count	5 min	The number of blocks in which HDFS has no replicas. These might be corrupt blocks. Use case: Monitor cluster health
CorruptBlocks	Count	5 min	The number of blocks that HDFS reports as corrupted. Use case: Monitor cluster health
TotalLoad	Count	5 min	The total number of concurrent data transfers. Use case: Monitor cluster health
MemoryTotalMB	Count	5 min	The total amount of memory in the cluster. Use case: Monitor cluster progress
MemoryReservedMB	Count	5 min	The amount of memory reserved. Use case: Monitor cluster progress
MemoryAvailableMB	Count	5 min	The amount of memory available to be allocated. Use case: Monitor cluster progress
YARNMemoryAvailablePercentage		5 min	The percentage of remaining memory available to YARN (YARNMemoryAvailablePercentage = MemoryAvailableMB / MemoryTotalMB). This value is useful for scaling cluster resources based on YARN memory usage.
MemoryAllocatedMB	Count	5 min	The amount of memory allocated to the cluster. Use case: Monitor cluster progress
PendingDeletionBlocks	Count	5 min	The number of blocks marked for deletion. Use case: Monitor cluster progress, Monitor cluster health
UnderReplicatedBlocks	Count	5 min	The number of blocks that need to be replicated one or more times. Use case: Monitor cluster progress, Monitor cluster health

DfsPendingReplicationBlocks	Count	5 min	The status of block replication: blocks being replicated, age of replication requests, and unsuccessful replication requests. Use case: Monitor cluster progress, Monitor cluster health
CapacityRemainingGB	Count	5 min	The amount of remaining HDFS disk capacity. Use case: Monitor cluster progress, Monitor cluster health
HBase			
HbaseBackupFailed	Count	5 min	Whether the last backup failed. This is set to 0 by default and updated to 1 if the previous backup attempt failed. This metric is only reported for HBase clusters. Use case: Monitor HBase backups
MostRecentBackupDuration	Minutes	5 min	The amount of time it took the previous backup to complete. This metric is set regardless of whether the last completed backup succeeded or failed. While the backup is ongoing, this metric returns the number of minutes after the backup started. This metric is only reported for HBase clusters. Use case: Monitor HBase Backups
TimeSinceLastSuccessfulBackup	Minutes	5 min	The number of elapsed minutes after the last successful HBase backup started on your cluster. This metric is only reported for HBase clusters. Use case: Monitor HBase backups

Table A.11: KinesisAnalytic

Metric	Unit	Sample period	Description
IncomingBytes	Bytes	1 min	The number of bytes successfully put to the shard over the specified time period. This metric includes bytes from PutRecord and PutRecords operations. Minimum, Maximum, and Average statistics represent the bytes in a single put operation for the shard in the specified time period. Stream-level metric name: IncomingBytes Dimensions: StreamName, ShardId Statistics: Minimum, Maximum, Average, Sum, Samples
IncomingRecords	Count	1 min	The number of records successfully put to the shard over the specified time period. This metric includes record counts from PutRecord and PutRecords operations. Minimum, Maximum, and Average statistics represent the records in a single put operation for the shard in the specified time period. Stream-level metric name: IncomingRecords Dimensions: StreamName, ShardId Statistics: Minimum, Maximum, Average, Sum, Samples

IteratorAgeMilliseconds	Milliseconds	1 min	The age of the last record in all GetRecords calls made against a shard, measured over the specified time period. Age is the difference between the current time and when the last record of the GetRecords call was written to the stream. The Minimum and Maximum statistics can be used to track the progress of Kinesis consumer applications. A value of 0 (zero) indicates that the records being read are completely caught up with the stream. Stream-level metric name: GetRecords.IteratorAgeMilliseconds Dimensions: StreamName, ShardId Statistics: Minimum, Maximum, Average, Samples
OutgoingBytes	Bytes	1 min	The number of bytes retrieved from the shard, measured over the specified time period. Minimum, Maximum, and Average statistics represent the bytes returned in a single GetRecords operation or published in a single SubscribeToShard event for the shard in the specified time period. Stream-level metric name: GetRecords.Bytes Dimensions: StreamName, ShardId Statistics: Minimum, Maximum, Average, Sum, Samples
OutgoingRecords	Count	1 min	The number of records retrieved from the shard, measured over the specified time period. Minimum, Maximum, and Average statistics represent the records returned in a single GetRecords operation or published in a single SubscribeToShard event for the shard in the specified time period. Stream-level metric name: GetRecords.Records Dimensions: StreamName, ShardId Statistics: Minimum, Maximum, Average, Sum, Samples
ReadProvisionedThroughputExceeded	Count	1 min	The number of GetRecords calls throttled for the shard over the specified time period. This exception count covers all dimensions of the following limits: 5 reads per shard per second or 2 MB per second per shard. The most commonly used statistic for this metric is Average. When the Minimum statistic has a value of 1, all records were throttled for the shard during the specified time period. When the Maximum statistic has a value of 0 (zero), no records were throttled for the shard during the specified time period. Stream-level metric name: ReadProvisionedThroughputExceeded Dimensions: StreamName, ShardId Statistics: Minimum, Maximum, Average, Sum, Samples

WriteProvisionedThroughputExceeded	Count	1 min	The number of records rejected due to throttling for the shard over the specified time period. This metric includes throttling from PutRecord and PutRecords operations and covers all dimensions of the following limits: 1,000 records per second per shard or 1 MB per second per shard. The most commonly used statistic for this metric is Average. When the Minimum statistic has a non-zero value, records were being throttled for the shard during the specified time period. When the Maximum statistic has a value of 0 (zero), no records were being throttled for the shard during the specified time period. Stream-level metric name: WriteProvisionedThroughputExceeded Dimensions: StreamName, ShardId Statistics: Minimum, Maximum, Average, Sum, Samples
GetRecords.Bytes	Bytes	1 min	The number of bytes retrieved from the Kinesis stream, measured over the specified time period. Minimum, Maximum, and Average statistics represent the bytes in a single GetRecords operation for the stream in the specified time period. Shard-level metric name: OutgoingBytes Dimensions: StreamName Statistics: Minimum, Maximum, Average, Sum, Samples
GetRecords.IteratorAge	Milliseconds	1 min	This metric is deprecated. Use GetRecords.IteratorAgeMilliseconds.
GetRecords.IteratorAgeMilliseconds	Milliseconds	1 min	The age of the last record in all GetRecords calls made against an Kinesis stream, measured over the specified time period. Age is the difference between the current time and when the last record of the GetRecords call was written to the stream. The Minimum and Maximum statistics can be used to track the progress of Kinesis consumer applications. A value of zero indicates that the records being read are completely caught up with the stream. Shard-level metric name: IteratorAgeMilliseconds Dimensions: StreamName Statistics: Minimum, Maximum, Average, Samples
GetRecords.Latency	Milliseconds	1 min	The time taken per GetRecords operation, measured over the specified time period. Dimensions: StreamName Statistics: Minimum, Maximum, Average
GetRecords.Records	Count	1 min	The number of records retrieved from the shard, measured over the specified time period. Minimum, Maximum, and Average statistics represent the records in a single GetRecords operation for the stream in the specified time period. Shard-level metric name: OutgoingRecords Dimensions: StreamName Statistics: Minimum, Maximum, Average, Sum, Samples
GetRecords.Success	Count	1 min	The number of successful GetRecords operations per stream, measured over the specified time period. Dimensions: StreamName Statistics: Average, Sum, Samples

IncomingBytes	Bytes	1 min	The number of bytes successfully put to the Kinesis stream over the specified time period. This metric includes bytes from PutRecord and PutRecords operations. Minimum, Maximum, and Average statistics represent the bytes in a single put operation for the stream in the specified time period. Shard-level metric name: IncomingBytes Dimensions: StreamName Statistics: Minimum, Maximum, Average, Sum, Samples
IncomingRecords	Count	1 min	The number of records successfully put to the Kinesis stream over the specified time period. This metric includes record counts from PutRecord and PutRecords operations. Minimum, Maximum, and Average statistics represent the records in a single put operation for the stream in the specified time period. Shard-level metric name: IncomingRecords Dimensions: StreamName Statistics: Minimum, Maximum, Average, Sum, Samples
PutRecord.Bytes	Bytes	1 min	The number of bytes put to the Kinesis stream using the PutRecord operation over the specified time period. Dimensions: StreamName Statistics: Minimum, Maximum, Average, Sum, Samples
PutRecord.Latency	Milliseconds	1 min	The time taken per PutRecord operation, measured over the specified time period. Dimensions: StreamName Statistics: Minimum, Maximum, Average
PutRecord.Success	Count	1 min	The number of successful PutRecord operations per Kinesis stream, measured over the specified time period. Average reflects the percentage of successful writes to a stream. Dimensions: StreamName Statistics: Average, Sum, Samples
PutRecords.Bytes	Bytes	1 min	The number of bytes put to the Kinesis stream using the PutRecords operation over the specified time period. Dimensions: StreamName Statistics: Minimum, Maximum, Average, Sum, Samples
PutRecords.Latency	Milliseconds	1 min	The time taken per PutRecords operation, measured over the specified time period. Dimensions: StreamName Statistics: Minimum, Maximum, Average
PutRecords.Records	Count	1 min	The number of successful records in a PutRecords operation per Kinesis stream, measured over the specified time period. Dimensions: StreamName Statistics: Minimum, Maximum, Average, Sum, Samples
PutRecords.Success	Count	1 min	The number of PutRecords operations where at least one record succeeded, per Kinesis stream, measured over the specified time period. Dimensions: StreamName Statistics: Average, Sum, Samples

ReadProvisionedThroughputExceeded	Count	1 min	The number of GetRecords calls throttled for the stream over the specified time period. The most commonly used statistic for this metric is Average. When the Minimum statistic has a value of 1, all records were throttled for the stream during the specified time period. When the Maximum statistic has a value of 0 (zero), no records were throttled for the stream during the specified time period. Shard-level metric name: ReadProvisionedThroughputExceeded Dimensions: StreamName Statistics: Minimum, Maximum, Average, Sum, Samples
SubscribeToShard.RateExceeded	Boolean	1 min	This metric is emitted when a new subscription attempt fails because there already is an active subscription by the same consumer or if you exceed the number of calls per second allowed for this operation.Dimensions: StreamName, ConsumerName
SubscribeToShard.Success	Boolean	1 min	This metric records whether the SubscribeToShard subscription was successfully established. The subscription only lives for at most 5 minutes. Therefore, this metric gets emitted at least once every 5 minutes. Dimensions: StreamName, ConsumerName
SubscribeToShardEvent.Bytes	Bytes	1 min	The number of bytes received from the shard, measured over the specified time period. Minimum, Maximum, and Average statistics represent the bytes published in a single event for the specified time period. Shard-level metric name: OutgoingBytes Dimensions: StreamName, ConsumerName Statistics: Minimum, Maximum, Average, Sum, Samples
SubscribeToShardEvent.MillisBehindLatest	Milliseconds	1 min	The difference between the current time and when the last record of the SubscribeToShard event was written to the stream. Dimensions: StreamName, ConsumerName Statistics: Minimum, Maximum, Average, Samples
SubscribeToShardEvent.Records	Count	1 min	The number of records received from the shard, measured over the specified time period. Minimum, Maximum, and Average statistics represent the records in a single event for the specified time period. Shard-level metric name: OutgoingRecords Dimensions: StreamName, ConsumerName Statistics: Minimum, Maximum, Average, Sum, Samples
SubscribeToShardEvent.Success	Count	1 min	This metric is emitted every time an event is published successfully. It is only emitted when there's an active subscription. Dimensions: StreamName, ConsumerName Statistics: Minimum, Maximum, Average, Sum, Samples

WriteProvisionedThroughputExceeded	Count	1 min	The number of records rejected due to throttling for the stream over the specified time period. This metric includes throttling from PutRecord and PutRecords operations. The most commonly used statistic for this metric is Average. When the Minimum statistic has a non-zero value, records were being throttled for the stream during the specified time period. When the Maximum statistic has a value of 0 (zero), no records were being throttled for the stream during the specified time period. Shard-level metric name: WriteProvisionedThroughputExceeded Dimensions: StreamName Statistics: Minimum, Maximum, Average, Sum, Samples
downtime	Milliseconds	1 min	For jobs currently in a failing/recovering situation, the time elapsed during this outage.
lastCheckpointDuration	Milliseconds	1 min	The time it took to complete the last checkpoint
lastCheckpointSize	Bytes	1 min	The total size of the last checkpoint
numRecordsIn	Count	1 min	The total number of records this operator or task has received.
numRecordsInPerSecond	Bytes/Second	1 min	The total number of records this operator or task has received per second.
numRecordsOut	Count	1 min	The total number of records this operator or task has emitted.
numRecordsOutPerSecond	Bytes/Second	1 min	The total number of records this operator or task has emitted per second.
numLateRecordsDropped	Count	1 min	The number of records this operator or task has dropped due to arriving late.
currentInputWatermark	Milliseconds	1 min	The last watermark this operator/tasks has received
Bytes	Bytes	1 min	The number of bytes read (per input stream) or written (per output stream). Levels: Per input stream and per output stream
KPUs	Count	1 min	The number of Kinesis Processing Units that are used to run your stream processing application. The average number of KPUs used each hour determines the billing for your application. Levels: Application-level
MillisBehindLatest	Milliseconds	1 min	Indicates how far behind from the current time an application is reading from the streaming source. Levels: Application-level
Records	Count	1 min	The number of records read (per input stream) or written (per output stream). Levels: Per input stream and per output stream
Success	Count	1 min	1 for each successful delivery attempt to the destination configured for your application; 0 for every failed delivery attempt. The average value of this metric indicates how many successful deliveries are performed. Levels: Per destination.

InputProcessing.Duration	*No Information*	1 min	The time taken for each AWS Lambda function invocation performed by Kinesis Data Analytics. Levels: Per input stream
InputProcessing.OkRecords	Count	1 min	The number of records returned by a Lambda function that were marked with Ok status. Levels: Per input stream
InputProcessing.OkBytes	*No Information*	1 min	The sum of bytes of the records returned by a Lambda function that were marked with Ok status. Levels: Per input stream
InputProcessing.DroppedRecords	Count	1 min	The number of records returned by a Lambda function that were marked with Dropped status. Levels: Per input stream
InputProcessing.ProcessingFailedRecord	Count	1 min	The number of records returned by a Lambda function that were marked with ProcessingFailed status. Levels: Per input stream
InputProcessing.Success	Count	1 min	The number of successful Lambda invocations by Kinesis Data Analytics. Levels: Per input stream
LambdaDelivery.OkRecords	Count	1 min	The number of records returned by a Lambda function that were marked with Ok status. Levels: Per Lambda destination
LambdaDelivery.DeliveryFailedRecords	Count	1 min	The number of records returned by a Lambda function that were marked with DeliveryFailed status. Levels: Per Lambda destination
LambdaDelivery.Duration	*No Information*	1 min	The time taken for each Lambda function invocation performed by Kinesis Data Analytics. Levels: Per Lambda destination

Table A.12: Regions

Code	Name
us-east-1	US East (N. Virginia)
us-east-2	US East (Ohio)
us-west-1	US West (N. California)
us-west-2	US West (Oregon)
ca-central-1	Canada (Central)

eu-central-1	EU (Frankfurt)
eu-west-1	EU (Ireland)
eu-west-2	EU (London)
eu-west-3	EU (Paris)
eu-north-1	EU (Stockholm)
ap-northeast-1	Asia Pacific (Tokyo)
ap-northeast-2	Asia Pacific (Seoul)
ap-northeast-3	Asia Pacific (Osaka-Local)
ap-southeast-1	Asia Pacific (Singapore)
ap-southeast-2	Asia Pacific (Sydney)
ap-south-1	Asia Pacific (Mumbai)
sa-east-1	South America (São Paulo)

Table A.13: Cloud SQL

Metric	Unit	Sample period	Description
database/auto_failover_request_count	DELTA, INT64, 1	1 minute	Delta of number of instance auto-failover requests. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.
database/available_for_failover	GAUGE, INT64, 1	1 minute	This is >0 if the failover operation is available on the instance. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds.
database/cpu/reserved_cores	GAUGE, DOUBLE, 1	1 minute	Number of cores reserved for the database. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.
database/cpu/usage_time	DELTA, DOUBLE, s	1 minute	Cumulative CPU usage time in seconds. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.
database/cpu/utilization	GAUGE, DOUBLE, 1	1 minute	The fraction of the reserved CPU that is currently in use. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.
database/disk/bytes_used	GAUGE, INT64, By	1 minute	Data utilization in bytes. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.
database/disk/quota	GAUGE, INT64, By	1 minute	Maximum data disk size in bytes. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.
database/disk/read_ops_count	DELTA, INT64, 1	1 minute	Delta count of data disk read IO operations. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.
database/disk/utilization	GAUGE, DOUBLE, 1	1 minute	The fraction of the disk quota that is currently in use. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.
database/disk/write_ops_count	DELTA, INT64, 1	1 minute	Delta count of disk write IO operations. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.
database/memory/quota	GAUGE, INT64, By	1 minute	Maximum RAM size in bytes. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.
database/memory/usage	GAUGE, INT64, By	1 minute	RAM usage in bytes. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.

database/memory/utilization	GAUGE, DOUBLE, 1	1 minute	The fraction of the memory quota that is currently in use. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.
database/mysql/innodb_buffer_pool_-pages_\dirty	GAUGE, DOUBLE, 1	1 minute	Number of unflushed pages in the InnoDB buffer pool. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds.
database/mysql/innodb_buffer_pool_-pages_\free	GAUGE, INT64, 1	1 minute	Number of unused pages in the InnoDB buffer pool. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds.
database/mysql/innodb_buffer_pool_-pages_\total	GAUGE, INT64, 1	1 minute	Total number of pages in the InnoDB buffer pool. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds.
database/mysql/innodb_data_fsyncs	DELTA, INT64, 1	1 minute	Delta count of InnoDB fsync() calls. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds.
database/mysql/innodb_os_log_fsyncs	DELTA, INT64, 1	1 minute	Delta count of InnoDB fsync() calls to the log file. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds.
database/mysql/innodb_pages_read	DELTA, INT64, 1	1 minute	Delta count of InnoDB pages read. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds.
database/mysql/innodb_pages_written	DELTA, INT64, 1	1 minute	Delta count of InnoDB pages written. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds.
database/mysql/queries	DELTA, INT64, 1	1 minute	Delta count of statements executed by the server. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds.
database/mysql/questions	DELTA, INT64, 1	1 minute	Delta count of statements executed by the server sent by the client. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds.
database/mysql/received_bytes_count	DELTA, INT64, By	1 minute	Delta count of bytes received by MySQL process. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds.
database/mysql/replication/seconds_-behind_\master	GAUGE, INT64, s	1 minute	Number of seconds the read replica is behind its master (approximation). Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds
database/mysql/replication/slave_io_-\running	GAUGE, STRING,	1 minute	Indicates whether the I/O thread for reading the master's binary log is running. Possible values are Yes, No and Connecting. Sampled every 60 seconds. After sampling, data is not visible for up to 60 seconds.

database/mysql/replication/slave_-sql_\running	GAUGE, STRING,	1 minute	Indicates whether the SQL thread for executing events in the relay log is running. Sampled every 60 seconds. After sampling, data is not visible for up to 60 seconds.
database/mysql/sent_bytes_count	DELTA, INT64, By	1 minute	Delta count of bytes sent by MySQL process. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds.
database/network/connections	GAUGE, INT64, 1	1 minute	Number of connections to the Cloud SQL MySQL instance. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.
database/network/received_bytes_-count	DELTA, INT64, By	1 minute	Delta count of bytes received through the network. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.
database/network/sent_bytes_count	DELTA, INT64, By	1 minute	Delta count of bytes sent through the network. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.destination: Where the data is being sent.
database/postgresql/num_backends	GAUGE, INT64, 1	1 minute	Number of connections to the Cloud SQL PostgreSQL instance. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds.
database/postgresql/replication/replica_-\byte_lag	GAUGE, INT64, By	1 minute	Replication lag in bytes. Reported from the master per replica. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.replica_name: Name of the replica.replica_lag_type: Type of lag. This can be either replay_location, flush_location, write_location or sent_location.
database/postgresql/transaction_-count	DELTA, INT64, 1	1 minute	Delta count of number of transactions. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds.database: Name of the database.transaction_type: transaction_-type can be commit or rollback.
database/state	GAUGE, STRING,	1 minute	The current serving state of the Cloud SQL instance. This can be one of the following. RUNNABLE: The instance is running, or is ready to run when accessed. SUSPENDED: The instance is not available, for example due to problems with billing. PENDING_CREATE: The instance is being created. MAINTENANCE: The instance is down for maintenance. UNKNOWN_STATE: The state of the instance is unknown. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.
database/up	GAUGE, INT64, 1	1 minute	Indicates if the server is up or not. On-demand instances are spun down if no connections are made for a sufficient amount of time. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.

database/uptime	DELTA, INT64, s	1 minute	Delta count of the time in seconds the instance has been running. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds.

Table A.14: Persistent Disk

Metric	Unit	Sample period	Description
bytes_used	GAUGE, DOUBLE, By	1 minute	Disk bytes used. Linux only. Sampled every 60 seconds. device: Device name. state: Type of usage (free, used, reserved)
io_time	CUMULATIVE, INT64, s	1 minute	Average time an I/O-operation took to complete. Linux only. Sampled every 60 seconds. device: Device name.
merged_operations	CUMULATIVE, INT64, 1	1 minute	Merged Operations Count. Linux only. Sampled every 60 seconds. device: Device name. direction: Direction (read, write)
operation_count	CUMULATIVE, INT64, 1	1 minute	Disk Operations Count. Linux only. Sampled every 60 seconds. device: Device name. direction: Direction (read, write)
operation_time	CUMULATIVE, INT64, s	1 minute	Time Spent In Disk Operations Sampled every 60 seconds. device: Device name. direction: Direction (read, write)
pending_operations	GAUGE, DOUBLE, 1	1 minute	Pending Operations Count. Linux only. Sampled every 60 seconds. device: Device name.
percent_used	GAUGE, DOUBLE, %	1 minute	Disk percent used Sampled every 60 seconds. device: Device name. state: Type of usage (free, used, reserved)
read_bytes_count	CUMULATIVE, INT64, By	1 minute	Disk Bytes Read. Linux only. Sampled every 60 seconds. device: Device name.
weighted_io_time	CUMULATIVE, INT64, s	1 minute	Weighted I/O time. Linux only. Sampled every 60 seconds. device: Device name.
write_bytes_count	CUMULATIVE, INT64, By	1 minute	Disk Bytes Transferred. Linux only. Sampled every 60 seconds. device: Device name.

Table A.15: Load Balancing

Metric	Unit	Sample period	Description
https/backend_latencies	DELTA, DISTRIBUTION, ms	1 minute	A distribution of the latency calculated from when the request was sent by the proxy to the backend until the proxy received from the backend the last byte of response. Sampled every 60 seconds. After sampling, data is not visible for up to 210 seconds. protocol: Protocol used by the client: 'HTTP/1.0', 'HTTP/1.1', 'HTTP/2.0', 'SPDY' or 'UNKNOWN'. response_code: (INT64) HTTP response code. response_code_class: (INT64) HTTP response code class: 200, 300, 400, 500 or 0 for none. proxy_continent: Continent of the HTTP proxy which handled the hit: 'America', 'Europe' or 'Asia'. cache_result: Cache result for serving HTTP request by proxy: 'HIT', 'MISS', 'DISABLED' or 'UNKNOWN'. client_country: Country of the client that issued the HTTP request (e.g. 'United States', 'Germany').
https/backend_request_bytes_count	DELTA, INT64, By	1 minute	The number of bytes sent as requests from HTTP/S load balancer to backends. Sampled every 60 seconds. After sampling, data is not visible for up to 210 seconds. response_code: (INT64) HTTP backend response code. response_code_class: (INT64) HTTP backend response code class: 200, 300, 400, 500 or 0 for none. proxy_continent: Continent of the HTTP proxy which handled the request: 'America', 'Europe' or 'Asia'. cache_result: Cache result for serving the HTTP request by backends (cache hits are abstracted as backend requests): 'HIT', 'MISS', 'DISABLED' or 'UNKNOWN'.
https/backend_request_count	DELTA, INT64, 1	1 minute	The number of requests served by backends of HTTP/S load balancer. Sampled every 60 seconds. After sampling, data is not visible for up to 210 seconds. response_code: (INT64) HTTP backend response code. response_code_class: (INT64) HTTP backend response code class: 200, 300, 400, 500 or 0 for none. proxy_continent: Continent of the HTTP proxy which handled the request: 'America', 'Europe' or 'Asia'. cache_result: Cache result for serving the HTTP request by backends (cache hits are abstracted as backend requests): 'HIT', 'MISS', 'DISABLED' or 'UNKNOWN'.

https/backend_response_bytes_count	DELTA, INT64, By	1 minute	The number of bytes sent as responses from backends (or cache) to HTTP/S load balancer. Sampled every 60 seconds. After sampling, data is not visible for up to 210 seconds. response_code: (INT64) HTTP backend response code. response_code_class: (INT64) HTTP backend response code class: 200, 300, 400, 500 or 0 for none. proxy_continent: Continent of the HTTP proxy which handled the request: 'America', 'Europe' or 'Asia'. cache_result: Cache result for serving the HTTP request by backends (cache hits are abstracted as backend requests): 'HIT', 'MISS', 'DISABLED' or 'UNKNOWN'.
https/frontend_tcp_rtt	DELTA, DISTRIBUTION, ms	1 minute	A distribution of the RTT measured for each connection between client and proxy. Sampled every 60 seconds. After sampling, data is not visible for up to 210 seconds. proxy_continent: Continent of the HTTP proxy which handled the hit: 'America', 'Europe' or 'Asia'. client_country: Country of the client that issued the HTTP request (e.g. 'United States', 'Germany').
https/request_bytes_count	DELTA, INT64, By	1 minute	The number of bytes sent as requests from clients to HTTP/S load balancer. Sampled every 60 seconds. After sampling, data is not visible for up to 210 seconds. protocol: Protocol used by the client: 'HTTP/1.0', 'HTTP/1.1', 'HTTP/2.0', 'SPDY' or 'UNKNOWN'. response_code: (INT64) HTTP response code. response_code_class: (INT64) HTTP response code class: 200, 300, 400, 500 or 0 for none. proxy_continent: Continent of the HTTP proxy which handled the hit: 'America', 'Europe' or 'Asia'. cache_result: Cache result for serving HTTP request by proxy: 'HIT', 'MISS', 'DISABLED' or 'UNKNOWN'. client_country: Country of the client that issued the HTTP request (e.g. 'United States', 'Germany').
https/request_count	DELTA, INT64, 1	1 minute	The number of requests served by HTTP/S load balancer. Sampled every 60 seconds. After sampling, data is not visible for up to 210 seconds. protocol: Protocol used by the client: 'HTTP/1.0', 'HTTP/1.1', 'HTTP/2.0', 'QUIC/HTTP/2.0' or 'UNKNOWN'. response_code: (INT64) HTTP response code. response_code_class: (INT64) HTTP response code class: 200, 300, 400, 500 or 0 for none. proxy_continent: Continent of the HTTP proxy which handled the hit: 'America', 'Europe' or 'Asia'. cache_result: Cache result for serving HTTP request by proxy: 'HIT', 'MISS', 'DISABLED' or 'UNKNOWN'. client_country: Country of the client that issued the HTTP request (e.g. 'United States', 'Germany').

https/response_bytes_count	DELTA, INT64, By	1 minute	The number of bytes sent as responses from HTTP/S load balancer to clients. Sampled every 60 seconds. After sampling, data is not visible for up to 210 seconds. protocol: Protocol used by the client: 'HTTP/1.0', 'HTTP/1.1', 'HTTP/2.0', 'SPDY' or 'UNKNOWN'. response_-code: (INT64) HTTP response code. response_code_class: (INT64) HTTP response code class: 200, 300, 400, 500 or 0 for none. proxy_continent: Continent of the HTTP proxy which handled the hit: 'America', 'Europe' or 'Asia'. cache_result: Cache result for serving HTTP request by proxy: 'HIT', 'MISS', 'DISABLED' or 'UNKNOWN'. client_country: Country of the client that issued the HTTP request (e.g. 'United States', 'Germany').
https/total_latencies	DELTA, DISTRIBUTION, ms	1 minute	A distribution of the latency calculated from when the request was received by the proxy until the proxy got ACK from client on last response byte. Sampled every 60 seconds. After sampling, data is not visible for up to 210 seconds. protocol: Protocol used by the client: 'HTTP/1.0', 'HTTP/1.1', 'HTTP/2.0', 'SPDY' or 'UNKNOWN'. response_code: (INT64) HTTP response code. response_code_class: (INT64) HTTP response code class: 200, 300, 400, 500 or 0 for none. proxy_continent: Continent of the HTTP proxy which handled the hit: 'America', 'Europe' or 'Asia'. cache_result: Cache result for serving HTTP request by proxy: 'HIT', 'MISS', 'DISABLED' or 'UNKNOWN'. client_country: Country of the client that issued the HTTP request (e.g. 'United States', 'Germany').
l3/internal/egress_bytes_count	DELTA, INT64, By	1 minute	The number of bytes sent from ILB backend to client (for TCP flows it's counting bytes on application stream only). Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds. client_network: Network of the client instance in ILB flow. client_-subnetwork: Subnetwork of the client instance in ILB flow. client_zone: Zone of the client instance in ILB flow.
l3/internal/egress_packets_count	DELTA, INT64, 1	1 minute	The number of packets sent from ILB backend to client of the flow. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds. client_network: Network of the client instance in ILB flow. client_subnetwork: Subnetwork of the client instance in ILB flow. client_zone: Zone of the client instance in ILB flow.
l3/internal/ingress_bytes_count	DELTA, INT64, By	1 minute	The number of bytes sent from client to ILB backend (for TCP flows it's counting bytes on application stream only). Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds. client_network: Network of the client instance in ILB flow. client_-subnetwork: Subnetwork of the client instance in ILB flow. client_zone: Zone of the client instance in ILB flow.

l3/internal/ingress_packets_count	DELTA, INT64, 1	1 minute	The number of packets sent from client to ILB backend. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds. client_network: Network of the client instance in ILB flow. client_subnetwork: Subnetwork of the client instance in ILB flow. client_zone: Zone of the client instance in ILB flow.
l3/internal/rtt_latencies	DELTA, DISTRIBUTION, ms	1 minute	A distribution of RTT measured over TCP connections for ILB flows. Sampled every 60 seconds. After sampling, data is not visible for up to 150 seconds. client_network: Network of the client instance in ILB flow. client_subnetwork: Subnetwork of the client instance in ILB flow. client_zone: Zone of the client instance in ILB flow.
tcp_ssl_proxy/closed_connections	DELTA, INT64, 1	1 minute	Number of connections that were terminated over TCP/SSL proxy. Sampled every 60 seconds. After sampling, data is not visible for up to 210 seconds. proxy_continent: Continent of the SSL proxy which handles the connection: 'America', 'Europe' or 'Asia'. client_country: Country of the client that opened the TCP/SSL connection (e.g. 'United States', 'Germany').
tcp_ssl_proxy/egress_bytes_count	DELTA, INT64, By	1 minute	Number of bytes sent from VM to client using proxy. Sampled every 60 seconds. After sampling, data is not visible for up to 210 seconds. proxy_continent: Continent of the proxy which handles the connection: 'America', 'Europe' or 'Asia'. client_country: Country of the client that opened the TCP/SSL connection (e.g. 'United States', 'Germany').
tcp_ssl_proxy/frontend_tcp_rtt	DELTA, DISTRIBUTION, ms	1 minute	A distribution of the smoothed RTT (in ms) measured by the proxy's TCP stack, each minute application layer bytes pass from proxy to client. Sampled every 60 seconds. After sampling, data is not visible for up to 210 seconds. proxy_continent: Continent of the SSL proxy which handles the connection: 'America', 'Europe' or 'Asia'. client_country: Country of the client that opened the TCP/SSL connection (e.g. 'United States', 'Germany').
tcp_ssl_proxy/ingress_bytes_count	DELTA, INT64, By	1 minute	Number of bytes sent from client to VM using proxy. Sampled every 60 seconds. After sampling, data is not visible for up to 210 seconds. proxy_continent: Continent of the proxy which handles the connection: 'America', 'Europe' or 'Asia'. client_country: Country of the client that opened the TCP/SSL connection (e.g. 'United States', 'Germany').
tcp_ssl_proxy/new_connections	DELTA, INT64, 1	1 minute	Number of connections that were created over TCP/SSL proxy. Sampled every 60 seconds. After sampling, data is not visible for up to 210 seconds. proxy_continent: Continent of the SSL proxy which handles the connection: 'America', 'Europe' or 'Asia'. client_country: Country of the client that opened the TCP/SSL connection (e.g. 'United States', 'Germany').

tcp_ssl_proxy/open_connections	GAUGE, INT64, 1	1 minute	Current number of outstanding connections through the TCP/SSL proxy. Sampled every 60 seconds. After sampling, data is not visible for up to 210 seconds. proxy_continent: Continent of the SSL proxy which handles the connection: 'America', 'Europe' or 'Asia'. client_country: Country of the client that opened the TCP/SSL connection (e.g. 'United States', 'Germany').

Table A.16: Cloud Storage

Metric	Unit	Sample period	Description
api/request_count	DELTA, INT64, 1	1 minute	Delta count of API calls, grouped by the API method name and response code. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. response_code: The response code of the requests. method: The name of the API method called.
authz/acl_based_object_access_count	DELTA, INT64, 1	1 minute	Delta count of requests that result in an object being granted access solely due to object ACLs. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds.
authz/object_specific_acl_mutation_-count	DELTA, INT64, 1	1 minute	Delta count of changes made to object specific ACLs. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds.
network/received_bytes_count	DELTA, INT64, By	1 minute	Delta count of bytes received over the network, grouped by the API method name and response code. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. response_code: The response code of the requests. method: The name of the API method called.
network/sent_bytes_count	DELTA, INT64, By	1 minute	Delta count of bytes sent over the network, grouped by the API method name and response code. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. response_code: The response code of the requests. method: The name of the API method called.
storage/object_count	GAUGE, INT64, 1	5 minutes	Total number of objects per bucket, grouped by storage class. Values are measured once per day. Sampled every 300 seconds. After sampling, data is not visible for up to 600 seconds. storage_class: Storage class of the data.

storage/total_byte_seconds	GAUGE, DOUBLE, By.s	5 minutes	Total daily storage in byte*seconds used by the bucket, grouped by storage class. Values are measured once per day. Sampled every 300 seconds. After sampling, data is not visible for up to 600 seconds. storage_class: Storage class of the data.
storage/total_bytes	GAUGE, DOUBLE, By	5 minutes	Total size of all objects in the bucket, grouped by storage class. Values are measured once per day. Sampled every 300 seconds. After sampling, data is not visible for up to 600 seconds. storage_class: Storage class of the data.

Table A.17: Filestore

Metric	Unit	Sample period	Description
nfs/server/free_bytes	GAUGE, INT64, By	1 minute	Total free disk bytes. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. file_share: The name of the File Share.
nfs/server/free_bytes_percent	GAUGE, DOUBLE, %	1 minute	Total free disk bytes percentage. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. file_share: The name of the File Share.
nfs/server/procedure_call_count	DELTA, INT64, 1	1 minute	NFS server procedure call count. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. name: The name of the NFS Server procedure. version: The version of the NFS protocol.
nfs/server/read_bytes_count	DELTA, INT64, By	1 minute	Bytes read from disk. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. file_share: The name of the File Share.
nfs/server/read_milliseconds_count	DELTA, INT64, ms	1 minute	Count of milliseconds spent on disk read operations. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. file_share: The name of the File Share.
nfs/server/read_ops_count	DELTA, INT64, 1	1 minute	Count of disk read operations. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. file_share: The name of the File Share.
nfs/server/used_bytes	GAUGE, INT64, By	1 minute	Total used disk bytes. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. file_share: The name of the File Share.
nfs/server/used_bytes_percent	GAUGE, DOUBLE, %	1 minute	Total used disk bytes percentage. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. file_share: The name of the File Share.

nfs/server/write_bytes_count	DELTA, INT64, By	1 minute	Bytes written to disk. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. file_share: The name of the File Share.
nfs/server/write_milliseconds_count	DELTA, INT64, ms	1 minute	Count of milliseconds spent on disk write operations. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. file_share: The name of the File Share.
nfs/server/write_ops_count	DELTA, INT64, 1	1 minute	Count of disk write operations. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. file_share: The name of the File Share.

Table A.18: Cloud Functions

Metric	Unit	Sample period	Description
function/active_instances	GAUGE, INT64, 1	1 minute	The number of active function instances. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds.
function/execution_count	DELTA, INT64, 1	1 minute	Count of function executions broken down by status. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds. memory: Memory assigned to function in MB. status: Function execution status e.g. 'ok', 'timeout', 'error'. trigger_type: Function's trigger type
function/execution_times	DELTA, DISTRIBUTION, ns	1 minute	Distribution of functions execution times in nanoseconds. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds. memory: Memory assigned to function in MB. status: Function execution status e.g. 'ok', 'timeout', 'error'. trigger_type: Function's trigger type
function/network_egress	DELTA, INT64, By	1 minute	Outgoing network traffic of function, in bytes Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds. memory: Memory assigned to function in MB. trigger_type: Function's trigger type
function/user_memory_bytes	DELTA, DISTRIBUTION, By	1 minute	Distribution of maximum function's memory usage during execution, in bytes. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds. memory: Memory assigned to function in MB. trigger_type: Function's trigger type

Table A.19: Kubernetes

Metric	Unit	Sample period	Description
container/accelerator/duty_cycle	GAUGE, INT64, %	1 minute	Percent of time over the past sample period (10s) during which the accelerator was actively processing. Sampled every 60 seconds. make: Make of the accelerator (e.g. nvidia) accelerator_id: ID of the accelerator model: Model of the accelerator (e.g. 'Tesla P100')
container/accelerator/memory_total	GAUGE, INT64, By	1 minute	Total accelerator memory in bytes. Sampled every 60 seconds. make: Make of the accelerator (e.g. nvidia) accelerator_id: ID of the accelerator model: Model of the accelerator (e.g. 'Tesla P100')
container/accelerator/memory_used	GAUGE, INT64, By	1 minute	Total accelerator memory allocated in bytes. Sampled every 60 seconds. make: Make of the accelerator (e.g. nvidia) accelerator_id: ID of the accelerator model: Model of the accelerator (e.g. 'Tesla P100')
container/accelerator/request	GAUGE, INT64, {devices}	1 minute	Number of accelerator devices requested by the container. Sampled every 60 seconds. resource_name: Name of the requested accelerator resource.
container/cpu/core_usage_time	CUMULATIVE, DOUBLE, s	1 minute	Cumulative CPU usage on all cores used by the container in seconds. Sampled every 60 seconds.
container/cpu/limit_cores	GAUGE, DOUBLE, {cpu}	1 minute	CPU cores limit of the container. Sampled every 60 seconds.
container/cpu/limit_utilization	GAUGE, DOUBLE, 1	1 minute	The fraction of the CPU limit that is currently in use on the instance. This value cannot exceed 1 as usage cannot exceed the limit. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds.
container/cpu/request_cores	GAUGE, DOUBLE, {cpu}	1 minute	Number of CPU cores requested by the container. Sampled every 60 seconds.
container/cpu/request_utilization	GAUGE, DOUBLE, 1	1 minute	The fraction of the requested CPU that is currently in use on the instance. This value can be greater than 1 as usage can exceed the request. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds.

container/ephemeral_storage/limit_-bytes	GAUGE, INT64, By	1 minute	Local ephemeral storage limit in bytes. Sampled every 60 seconds.
container/ephemeral_storage/re-quest_bytes	GAUGE, INT64, By	1 minute	Local ephemeral storage request in bytes. Sampled every 60 seconds.
container/ephemeral_storage/used_-bytes	GAUGE, INT64, By	1 minute	Local ephemeral storage usage in bytes. Sampled every 60 seconds.
container/memory/limit_bytes	GAUGE, INT64, By	1 minute	Memory limit of the container in bytes. Sampled every 60 seconds.
container/memory/limit_utilization	GAUGE, DOUBLE, 1	1 minute	The fraction of the memory limit that is currently in use on the instance. This value cannot exceed 1 as usage cannot exceed the limit. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. memory_type: Either ‘evictable‘ or ‘non-evictable‘. Evictable memory is memory that can be easily reclaimed by the kernel, while non-evictable memory cannot.
container/memory/request_bytes	GAUGE, INT64, By	1 minute	Memory request of the container in bytes. Sampled every 60 seconds.
container/memory/request_utilization	GAUGE, DOUBLE, 1	1 minute	The fraction of the requested memory that is currently in use on the instance. This value can be greater than 1 as usage can exceed the request. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. memory_type: Either ‘evictable‘ or ‘non-evictable‘. Evictable memory is memory that can be easily reclaimed by the kernel, while non-evictable memory cannot.
container/memory/used_bytes	GAUGE, INT64, By	1 minute	Memory usage in bytes. Sampled every 60 seconds. memory_type: Either ‘evictable‘ or ‘non-evictable‘. Evictable memory is memory that can be easily reclaimed by the kernel, while non-evictable memory cannot.
container/restart_count	CUMULATIVE, INT64, 1	1 minute	Number of times the container has restarted. Sampled every 60 seconds.
container/uptime	GAUGE, DOUBLE, s	1 minute	Time in seconds that the container has been running. Sampled every 60 seconds.

node/cpu/allocatable_cores	GAUGE, DOUBLE, {cpu}	1 minute	Number of allocatable CPU cores on the node. Sampled every 60 seconds.
node/cpu/allocatable_utilization	GAUGE, DOUBLE, 1	1 minute	The fraction of the allocatable CPU that is currently in use on the instance. This value cannot exceed 1 as usage cannot exceed allocatable CPU cores. Sampled every 60 seconds. After sampling, data is not visible for up to 240 seconds.
node/cpu/core_usage_time	CUMULATIVE, DOUBLE, s	1 minute	Cumulative CPU usage on all cores used on the node in seconds. Sampled every 60 seconds.
node/cpu/total_cores	GAUGE, DOUBLE, {cpu}	1 minute	Total number of CPU cores on the node. Sampled every 60 seconds.
node/ephemeral_storage/allocatable_-bytes	GAUGE, INT64, By	1 minute	Local ephemeral storage bytes allocatable on the node. Sampled every 60 seconds.
node/ephemeral_storage/total_bytes	GAUGE, INT64, By	1 minute	Total ephemeral storage bytes on the node. Sampled every 60 seconds.
node/ephemeral_storage/used_bytes	GAUGE, INT64, By	1 minute	Local ephemeral storage bytes used by the node. Sampled every 60 seconds.
node/memory/allocatable_bytes	GAUGE, INT64, By	1 minute	Cumulative memory bytes used by the node. Sampled every 60 seconds.
node/memory/allocatable_utilization	GAUGE, DOUBLE, 1	1 minute	The fraction of the allocatable memory that is currently in use on the instance. This value cannot exceed 1 as usage cannot exceed allocatable memory bytes. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. memory_type: Either ‘evictable‘ or ‘non-evictable‘. Evictable memory is memory that can be easily reclaimed by the kernel, while non-evictable memory cannot.
node/memory/total_bytes	GAUGE, INT64, By	1 minute	Number of bytes of memory allocatable on the node. Sampled every 60 seconds.
node/memory/used_bytes	GAUGE, INT64, By	1 minute	Cumulative memory bytes used by the node. Sampled every 60 seconds. memory_type: Either ‘evictable‘ or ‘non-evictable‘. Evictable memory is memory that can be easily reclaimed by the kernel, while non-evictable memory cannot.

node/network/received_bytes_count	CUMULATIVE, INT64, By	1 minute	Cumulative number of bytes received by the node over the network. Sampled every 60 seconds.
node/network/sent_bytes_count	CUMULATIVE, INT64, By	1 minute	Cumulative number of bytes transmitted by the node over the network. Sampled every 60 seconds.
node_daemon/cpu/core_usage_time	CUMULATIVE, DOUBLE, s	1 minute	Cumulative CPU usage on all cores used by the node level system daemon in seconds. Sampled every 60 seconds. component: Name of the respective system daemon.
node_daemon/memory/used_bytes	GAUGE, INT64, By	1 minute	Memory usage by the system daemon in bytes. Sampled every 60 seconds. component: Name of the respective system daemon. memory_type: Either ‘evictable‘ or ‘non-evictable‘. Evictable memory is memory that can be easily reclaimed by the kernel, while non-evictable memory cannot.
pod/network/received_bytes_count	CUMULATIVE, INT64, By	1 minute	Cumulative number of bytes received by the pod over the network. Sampled every 60 seconds.
pod/network/sent_bytes_count	CUMULATIVE, INT64, By	1 minute	Cumulative number of bytes transmitted by the pod over the network. Sampled every 60 seconds.
pod/volume/total_bytes	GAUGE, INT64, By	1 minute	Total number of disk bytes available to the pod. Sampled every 60 seconds. volume_name: The name of the volume (e.g. ‘/dev/sda1‘).
pod/volume/used_bytes	GAUGE, INT64, By	1 minute	Number of disk bytes used by the pod. Sampled every 60 seconds. volume_name: The name of the volume (e.g. ‘/dev/sda1‘).
pod/volume/utilization	GAUGE, DOUBLE, 1	1 minute	The fraction of the volume that is currently being used by the instance. This value cannot be greater than 1 as usage cannot exceed the total available volume space. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. volume_name: The name of the volume (e.g. ‘/dev/sda1‘).

Table A.20: Interconnect

Metric	Unit	Sample period	Description

network/attachment/capacity	GAUGE, INT64, By/s	1 minute	Network Capacity of the Attachment Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds.
network/attachment/received_bytes_-count	DELTA, INT64, By	1 minute	Number of inbound bytes received. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds.
network/attachment/received_pack-ets_count	DELTA, INT64, {packets}	1 minute	Number of inbound packets received. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds.
network/attachment/sent_bytes_count	DELTA, INT64, By	1 minute	Number of outbound bytes sent. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds.
network/attachment/sent_packets_-count	DELTA, INT64, {packets}	1 minute	Number of outbound packets sent. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds.
network/interconnect/capacity	GAUGE, INT64, By/s	1 minute	Active capacity of the interconnect. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds.
network/interconnect/dropped_pack-ets_count	DELTA, DOUBLE, {packets}	1 minute	Number of outbound packets dropped due to link congestion. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds.
network/interconnect/link/operational	GAUGE, BOOL,	1 minute	Whether the operational status of the circuit is 'up'. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. link_id: Identifier for the individual physical circuit.
network/interconnect/link/rx_power	GAUGE, DOUBLE, {dBm}	1 minute	Light level received over physical circuit. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. link_id: Identifier for the individual physical circuit.
network/interconnect/link/tx_power	GAUGE, DOUBLE, {dBm}	1 minute	Light level transmitted over physical circuit. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. link_id: Identifier for the individual physical circuit.
network/interconnect/operational	GAUGE, BOOL,	1 minute	Whether the operational status of the interconnect is 'up'. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds.

network/interconnect/receive_errors_-count	DELTA, DOUBLE, {errors}	1 minute	Number of errors encountered while receiving packets. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds.
network/interconnect/received_bytes_-count	DELTA, DOUBLE, By	1 minute	Number of inbound bytes received. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds.
network/interconnect/received_unicast_\packets_count	DELTA, DOUBLE, {packets}	1 minute	Number of inbound unicast packets received. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds.
network/interconnect/send_errors_-count	DELTA, DOUBLE, {errors}	1 minute	Number of errors encountered while sending packets. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds.
network/interconnect/sent_bytes_-count	DELTA, DOUBLE, By	1 minute	Number of outbound bytes sent. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds.
network/interconnect/sent_unicast_-packets_\count	DELTA, DOUBLE, {packets}	1 minute	Number of outbound unicast packets sent. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds.

Table A.21: Dataproc

Metric	Unit	Sample period	Description
cluster/hdfs/datanodes	GAUGE, INT64, 1	1 minute	Indicates the number of HDFS DataNodes that are running inside a cluster. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. status: The status of DataNode such as running, decommissioning, decommissioned.
cluster/hdfs/storage_capacity	GAUGE, DOUBLE, GiBy	1 minute	Indicates capacity of HDFS system running on cluster in GB. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. status: The status of HDFS capacity such as used, remaining.
cluster/hdfs/storage_utilization	GAUGE, DOUBLE, 1	1 minute	The percentage of HDFS storage currently used. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds.

cluster/hdfs/unhealthy_blocks	GAUGE, INT64, 1	1 minute	Indicates the number of unhealthy blocks inside the cluster. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. status: The status of unhealthy HDFS block such as missing, under_replication, corrupt, pending_deletion, missing_repl_one.
cluster/job/completion_time	DELTA, DISTRIBUTION, s	1 minute	The time jobs took to complete from the time the user submits a job to the time Dataproc reports it is completed. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. job_type: The type of job such as HADOOP_JOB or SPARK_JOB.
cluster/job/duration	DELTA, DISTRIBUTION, s	1 minute	The time jobs have spent in a given state. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. job_type: The type of job such as HADOOP_JOB or SPARK_JOB. state: The state of the job such as PENDING or RUNNING.
cluster/job/failed_count	DELTA, INT64, 1	1 minute	Indicates the number of jobs that have failed on a cluster. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. error_type: The rpc code of the error such as 404 or 503. job_type: The type of job such as HADOOP_JOB or SPARK_JOB.
cluster/job/running_count	GAUGE, INT64, 1	1 minute	Indicates the number of jobs that are running on a cluster. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. job_type: The type of job such as HADOOP_JOB or SPARK_JOB.
cluster/job/submitted_count	DELTA, INT64, 1	1 minute	Indicates the number of jobs that have been submitted to a cluster. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. job_type: The type of job such as HADOOP_JOB or SPARK_JOB.
cluster/operation/completion_time	DELTA, DISTRIBUTION, s	1 minute	The time operations took to complete from the time the user submits a operation to the time Dataproc reports it is completed. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. operation_type: The type of operation such as CREATE_CLUSTER or DELETE_CLUSTER.
cluster/operation/duration	DELTA, DISTRIBUTION, s	1 minute	The time operations have spent in a given state. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. operation_type: The state of the operation such as PENDING or CREATE_VMS. state: The type of operation such as CREATE_CLUSTER or DELETE_CLUSTER.
cluster/operation/failed_count	DELTA, INT64, 1	1 minute	Indicates the number of operations that have failed on a cluster. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. error_type: The rpc code of the error such as 404 or 503. operation_type: The type of operation such as CREATE_CLUSTER or DELETE_CLUSTER.

cluster/operation/running_count	GAUGE, INT64, 1	1 minute	Indicates the number of operations that are running on a cluster. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. operation_type: The type of operation such as CREATE_CLUSTER or DELETE_CLUSTER.
cluster/operation/submitted_count	DELTA, INT64, 1	1 minute	Indicates the number of operations that have been submitted to a cluster. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. operation_type: The type of operation such as CREATE_CLUSTER or DELETE_CLUSTER.
cluster/yarn/allocated_memory_percentage	GAUGE, DOUBLE, 1	1 minute	The percentage of YARN memory is allocated. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds.
cluster/yarn/apps	GAUGE, INT64, 1	1 minute	Indicates the number of active YARN applications. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. status: The status of YARN application such as running, pending, completed, failed, killed.
cluster/yarn/containers	GAUGE, INT64, 1	1 minute	Indicates the number of YARN containers. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. status: The status of YARN container such as allocated, pending, reserved.
cluster/yarn/memory_size	GAUGE, DOUBLE, GiBy	1 minute	Indicates the YARN memory size in GB. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. status: The status of YARN memory such as available, allocated, reserved.
cluster/yarn/nodemanagers	GAUGE, INT64, 1	1 minute	Indicates the number of YARN NodeManagers running inside cluster. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. status: The status of YARN NodeManager such as active, lost, unhealthy, rebooted, decommissioned.
cluster/yarn/pending_memory_size	GAUGE, DOUBLE, GiBy	1 minute	The current memory request, in GB, that is pending to be fulfilled by the scheduler. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds.
cluster/yarn/virtual_cores	GAUGE, INT64, 1	1 minute	Indicates the number of virtual cores in YARN. Sampled every 60 seconds. After sampling, data is not visible for up to 120 seconds. status: The status of YARN virtual cores such as pending, allocated, available, reserved.

Table A.22: Dataflow

Metric	Unit	Sample period	Description
job/billable_shuffle_data_processed	GAUGE, INT64, By	1 minute	The billable bytes of shuffle data processed by this Dataflow job. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. job_id: The ID of the current run of this pipeline.
job/current_num_vcpus	GAUGE, INT64, 1	1 minute	The number of vCPUs currently being used by this Dataflow job. This is the current number of workers times the number of vCPUs per worker. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. job_id: The ID of the current run of this pipeline.
job/current_shuffle_slots	GAUGE, INT64, 1	1 minute	The current shuffle slots used by this Dataflow job. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. job_id: The ID of the current run of this pipeline.
job/data_watermark_age	GAUGE, INT64, s	1 minute	The age (time since event timestamp) of the most recent item of data that has been fully processed by the pipeline. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. job_id: The ID of the current run of this pipeline.
job/elapsed_time	GAUGE, INT64, s	1 minute	Duration that the current run of this pipeline has been in the Running state so far, in seconds. When a run completes, this stays at the duration of that run until the next run starts. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. job_id: The ID of the current run of this pipeline.
job/element_count	GAUGE, INT64, 1	1 minute	Number of elements added to the pcollection so far. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. job_id: The ID of the current run of this pipeline. pcollection: The pipeline pcollection reporting this element count. PCollections are usually named by appending .out to the producing transform name.
job/estimated_byte_count	GAUGE, INT64, By	1 minute	An estimated number of bytes added to the pcollection so far. Dataflow calculates the average encoded size of elements in a pcollection and mutiplies it by the number of elements. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. job_id: The ID of the current run of this pipeline. pcollection: The pipeline pcollection reporting this estimated byte count. PCollections are usually named by appending .out to the producing transform name.

job/is_failed	GAUGE, INT64, 1	1 minute	Has this job failed. Sampled every 60 seconds. After sampling, data is not visible for up to 60 seconds.
job/status	GAUGE, STRING,	1 minute	Current state of this pipeline (e.g.: RUNNING, DONE, CANCELLED, FAILED, ...). Not reported while the pipeline is not running. Sampled every 60 seconds. After sampling, data is not visible for up to 60 seconds. job_id: The ID of the current run of this pipeline.
job/system_lag	GAUGE, INT64, s	1 minute	The current maximum duration that an item of data has been awaiting processing, in seconds. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. job_id: The ID of the current run of this pipeline.
job/total_memory_usage_time	GAUGE, INT64, GBy.s	1 minute	The total GB seconds of memory allocated to this Dataflow job. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. job_id: The ID of the current run of this pipeline.
job/total_pd_usage_time	GAUGE, INT64, GBy.s	1 minute	The total GB seconds for all persistent disk used by all workers associated with this Dataflow job. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. job_id: The ID of the current run of this pipeline. storage_type: The type of persistent disk: HDD, SSD.
job/total_shuffle_data_processed	GAUGE, INT64, By	1 minute	The total bytes of shuffle data processed by this Dataflow job. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. job_id: The ID of the current run of this pipeline.
job/total_streaming_data_processed	GAUGE, INT64, By	1 minute	The total bytes of streaming data processed by this Dataflow job. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. job_id: The ID of the current run of this pipeline.
job/total_vcpu_time	GAUGE, INT64, s	1 minute	The total vCPU seconds used by this Dataflow job. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. job_id: The ID of the current run of this pipeline.
job/user_counter	GAUGE, DOUBLE, 1	1 minute	A user-defined counter metric. Sampled every 60 seconds. After sampling, data is not visible for up to 180 seconds. metric_name: The name provided by the user for this counter. job_id: The ID of the current run of this pipeline. ptransform: The pipeline PTransform reporting this counter value.

Table A.23: Regions

Code	Location
asia-east1	Changhua County, Taiwan
asia-east2	Hong Kong
asia-northeast1	Tokyo, Japan
asia-south1	Mumbai, India
asia-southeast1	Jurong West, Singapore
australia-southeast1	Sydney, Australia
europe-north1	Hamina, Finland
europe-west1	St. Ghislain, Belgium
europe-west2	London, England, UK
europe-west3	Frankfurt, Germany
europe-west4	Eemshaven, Netherlands
europe-west6	Zürich, Switzerland
northamerica-northeast1	Montréal, Québec, Canada
southamerica-east1	São Paulo, Brazil
us-central1	Council Bluffs, Iowa, USA
us-east1	Moncks Corner, South Carolina, USA
us-east4	Ashburn, Northern Virginia, USA
us-west1	The Dalles, Oregon, USA
us-west2	Los Angeles, California, USA

Table A.24: Microsoft.DBforMySQL/servers

Metric	Metric Display Name	Unit	Aggregation Type	Description	Sample Period	Dimensions
cpu_percent	CPU percent	Percent	Average	CPU percent	1 minute	No Dimensions
memory_percent	Memory percent	Percent	Average	Memory percent	1 minute	No Dimensions
io_consumption_percent	IO percent	Percent	Average	IO percent	1 minute	No Dimensions
storage_percent	Storage percent	Percent	Average	Storage percent	1 minute	No Dimensions
storage_used	Storage used	Bytes	Average	Storage used	1 minute	No Dimensions
storage_limit	Storage limit	Bytes	Average	Storage limit	1 minute	No Dimensions
serverlog_storage_percent	Server Log storage percent	Percent	Average	Server Log storage percent	1 minute	No Dimensions
serverlog_storage_usage	Server Log storage used	Bytes	Average	Server Log storage used	1 minute	No Dimensions
serverlog_storage_limit	Server Log storage limit	Bytes	Average	Server Log storage limit	1 minute	No Dimensions
active_connections	Active Connections	Count	Average	Active Connections	1 minute	No Dimensions
connections_failed	Failed Connections	Count	Total	Failed Connections	1 minute	No Dimensions
seconds_behind_master	Replication lag in seconds	Count	Average	Replication lag in seconds	1 minute	No Dimensions
network_bytes_egress	Network Out	Bytes	Total	Network Out across active connections	1 minute	No Dimensions

network_bytes_ingress	Network In	Bytes	Total	Network In across active connections	1 minute	No Dimensions

Table A.25: Microsoft.DBforPostgreSQL/servers

Metric	**Metric Display Name**	**Unit**	**Aggregation Type**	**Description**	**Sample Period**	**Dimensions**
cpu_percent	CPU percent	Percent	Average	CPU percent	1 minute	No Dimensions
memory_percent	Memory percent	Percent	Average	Memory percent	1 minute	No Dimensions
io_consumption_percent	IO percent	Percent	Average	IO percent	1 minute	No Dimensions
storage_percent	Storage percent	Percent	Average	Storage percent	1 minute	No Dimensions
storage_used	Storage used	Bytes	Average	Storage used	1 minute	No Dimensions
storage_limit	Storage limit	Bytes	Average	Storage limit	1 minute	No Dimensions
serverlog_storage_percent	Server Log storage percent	Percent	Average	Server Log storage percent	1 minute	No Dimensions
serverlog_storage_usage	Server Log storage used	Bytes	Average	Server Log storage used	1 minute	No Dimensions
serverlog_storage_limit	Server Log storage limit	Bytes	Average	Server Log storage limit	1 minute	No Dimensions
active_connections	Active Connections	Count	Average	Active Connections	1 minute	No Dimensions
connections_failed	Failed Connections	Count	Total	Failed Connections	1 minute	No Dimensions

network_bytes_egress	Network Out	Bytes	Total	Network Out across active connections	1 minute	No Dimensions
network_bytes_ingress	Network In	Bytes	Total	Network In across active connections	1 minute	No Dimensions

Table A.26: Microsoft Azure Disk Storage

Metric	**Metric Display Name**	**Unit**	**Aggregation Type**	**Description**	**Sample Period**	**Dimensions**
Data Disk Read Bytes/Sec	Per Disk Read Bytes/sec	CountPerSecond	Average	FALSE	1 minute	slotID
Data Disk Write Bytes/Sec	Per Disk Write Bytes/sec	CountPerSecond	Average	FALSE	1 minute	slotID
Data Disk Read Operations/Sec	Per Disk Read Operations/Sec	CountPerSecond	Average	FALSE	1 minute	slotID
Data Disk Write Operations/Sec	Per Disk Write Operations/Sec	CountPerSecond	Average	FALSE	1 minute	slotID
Data Disk QD	Per Disk QD	Count	Average	FALSE	1 minute	slotID
OS Disk Read Bytes/Sec	OS Per Disk Read Bytes/sec	CountPerSecond	Average	FALSE	1 minute	No Dimensions
OS Disk Write Bytes/Sec	OS Per Disk Write Bytes/sec	CountPerSecond	Average	FALSE	1 minute	No Dimensions

OS Disk Read Operations/Sec	OS Per Disk Read Operations/Sec	CountPerSecond	Average	FALSE	1 minute	No Dimensions
OS Disk Write Operations/Sec	OS Per Disk Write Operations/Sec	CountPerSecond	Average	FALSE	1 minute	No Dimensions
OS Disk QD	OS Per Disk QD	Count	Average	FALSE	1 minute	No Dimensions

Table A.27: Microsoft.Network/applicationGateways

Metric	**Metric Display Name**	**Unit**	**Aggregation Type**	**Description**	**Sample Period**	**Dimensions**
Throughput	Throughput	BytesPerSecond	Total	Number of bytes per second the Application Gateway has served	1 minute	No Dimensions
UnhealthyHostCount	Unhealthy Host Count	Count	Average	Number of unhealthy backend hosts	1 minute	BackendSettingsPool
HealthyHostCount	Healthy Host Count	Count	Average	Number of healthy backend hosts	1 minute	BackendSettingsPool

TotalRequests	Total Requests	Count	Total	Count of successful requests that Application Gateway has served	1 minute	BackendSettingsPool
FailedRequests	Failed Requests	Count	Total	Count of failed requests that Application Gateway has served	1 minute	BackendSettingsPool
ResponseStatus	Response Status	Count	Total	Http response status returned by Application Gateway	1 minute	HttpStatusGroup
CurrentConnections	Current Connections	Count	Total	Count of current connections established with Application Gateway	1 minute	No Dimensions

Table A.28: Microsoft.Network/virtualNetworkGateways

Metric	**Metric Display Name**	**Unit**	**Aggregation Type**	**Description**	**Sample Period**	**Dimensions**

AverageBandwidth	Gateway S2S Bandwidth	BytesPerSecond	Average	Average site-to-site bandwidth of a gateway in bytes per second	1 minute	No Dimensions
P2SBandwidth	Gateway P2S Bandwidth	BytesPerSecond	Average	Average point-to-site bandwidth of a gateway in bytes per second	1 minute	No Dimensions
P2SConnectionCount	P2S Connection Count	Count	Maximum	Point-to-site connection count of a gateway	1 minute	Protocol
TunnelAverageBandwidth	Tunnel Bandwidth	BytesPerSecond	Average	Average bandwidth of a tunnel in bytes per second	1 minute	ConnectionName, RemoteIP
TunnelEgressBytes	Tunnel Egress Bytes	Bytes	Total	Outgoing bytes of a tunnel	1 minute	ConnectionName, RemoteIP
TunnelIngressBytes	Tunnel Ingress Bytes	Bytes	Total	Incoming bytes of a tunnel	1 minute	ConnectionName, RemoteIP
TunnelEgressPackets	Tunnel Egress Packets	Count	Total	Outgoing packet count of a tunnel	1 minute	ConnectionName, RemoteIP
TunnelIngressPackets	Tunnel Ingress Packets	Count	Total	Incoming packet count of a tunnel	1 minute	ConnectionName, RemoteIP

TunnelEgressPacketDropTSMismatch	Tunnel Egress TS Mismatch Packet Drop	Count	Total	Outgoing packet drop count from traffic selector mismatch of a tunnel	1 minute	ConnectionName, RemoteIP
TunnelIngressPacketDropTSMismatch	Tunnel Ingress TS Mismatch Packet Drop	Count	Total	Incoming packet drop count from traffic selector mismatch of a tunnel	1 minute	ConnectionName, RemoteIP

Table A.29: Microsoft.Storage/storageAccounts/blobServices

Metric	**Metric Display Name**	**Unit**	**Aggregation Type**	**Description**	**Sample Period**	**Dimensions**
BlobCapacity	Blob Capacity	Bytes	Total	The amount of storage used by the storage account's Blob service in bytes.	1 minute	BlobType
BlobCount	Blob Count	Count	Total	The number of Blob in the storage account's Blob service.	1 minute	BlobType
ContainerCount	Blob Container Count	Count	Average	The number of containers in the storage account's Blob service.	1 minute	No Dimensions

Transactions	Transactions	Count	Total	The number of requests made to a storage service or the specified API operation. This number includes successful and failed requests, as well as requests which produced errors. Use ResponseType dimension for the number of different type of response.	1 minute	ResponseType, GeoType, ApiName, Authentication
Ingress	Ingress	Bytes	Total	The amount of ingress data, in bytes. This number includes ingress from an external client into Azure Storage as well as ingress within Azure.	1 minute	GeoType, ApiName, Authentication
Egress	Egress	Bytes	Total	The amount of egress data, in bytes. This number includes egress from an external client into Azure Storage as well as egress within Azure. As a result, this number does not reflect billable egress.	1 minute	GeoType, ApiName, Authentication
SuccessServerLatency	Success Server Latency	Milliseconds	Average	The average latency used by Azure Storage to process a successful request, in milliseconds. This value does not include the network latency specified in AverageE2ELatency.	1 minute	GeoType, ApiName, Authentication

SuccessE2ELatency	Success E2E Latency	Milliseconds	Average	The average end-to-end latency of successful requests made to a storage service or the specified API operation, in milliseconds. This value includes the required processing time within Azure Storage to read the request, send the response, and receive acknowledgment of the response.	1 minute	GeoType, ApiName, Authentication
Availability	Availability	Percent	Average	The percentage of availability for the storage service or the specified API operation. Availability is calculated by taking the TotalBillableRequests value and dividing it by the number of applicable requests, including those that produced unexpected errors. All unexpected errors result in reduced availability for the storage service or the specified API operation.	1 minute	GeoType, ApiName, Authentication

Table A.30: Microsoft.Storage/storageAccounts/fileServices

Metric	**Metric Display Name**	**Unit**	**Aggregation Type**	**Description**	**Sample Period**	**Dimensions**
FileCapacity	File Capacity	Bytes	Average	The amount of storage used by the storage account's File service in bytes.	1 minute	No Dimensions

FileCount	File Count	Count	Average	The number of file in the storage account's File service.	1 minute	No Dimensions
FileShareCount	File Share Count	Count	Average	The number of file shares in the storage account's File service.	1 minute	No Dimensions
Transactions	Transactions	Count	Total	The number of requests made to a storage service or the specified API operation. This number includes successful and failed requests, as well as requests which produced errors. Use ResponseType dimension for the number of different type of response.	1 minute	ResponseType, GeoType, ApiName, Authentication
Ingress	Ingress	Bytes	Total	The amount of ingress data, in bytes. This number includes ingress from an external client into Azure Storage as well as ingress within Azure.	1 minute	GeoType, ApiName, Authentication
Egress	Egress	Bytes	Total	The amount of egress data, in bytes. This number includes egress from an external client into Azure Storage as well as egress within Azure. As a result, this number does not reflect billable egress.	1 minute	GeoType, ApiName, Authentication
SuccessServerLatency	Success Server Latency	Milliseconds	Average	The average latency used by Azure Storage to process a successful request, in milliseconds. This value does not include the network latency specified in AverageE2ELatency.	1 minute	GeoType, ApiName, Authentication

SuccessE2ELatency	Success E2E Latency	Milliseconds	Average	The average end-to-end latency of successful requests made to a storage service or the specified API operation, in milliseconds. This value includes the required processing time within Azure Storage to read the request, send the response, and receive acknowledgment of the response.	1 minute	GeoType, ApiName, Authentication
Availability	Availability	Percent	Average	The percentage of availability for the storage service or the specified API operation. Availability is calculated by taking the TotalBillableRequests value and dividing it by the number of applicable requests, including those that produced unexpected errors. All unexpected errors result in reduced availability for the storage service or the specified API operation.	1 minute	GeoType, ApiName, Authentication

Table A.31: Microsoft.Web/sites (functions)

Metric	**Metric Display Name**	**Unit**	**Aggregation Type**	**Description**	**Sample Period**	**Dimensions**
BytesReceived	Data In	Bytes	Total	Data In	1 minute	Instance

BytesSent	Data Out	Bytes	Total	Data Out	1 minute	Instance
Http5xx	Http Server Errors	Count	Total	Http Server Errors	1 minute	Instance
MemoryWorkingSet	Memory working set	Bytes	Average	Memory working set	1 minute	Instance
AverageMemoryWorkingSet	Average memory working set	Bytes	Average	Average memory working set	1 minute	Instance
FunctionExecutionUnits	Function Execution Units	Count	Total	Function Execution Units	1 minute	Instance
FunctionExecutionCount	Function Execution Count	Count	Total	Function Execution Count	1 minute	Instance
PrivateBytes	Private Bytes	Bytes	Average	Private Bytes	1 minute	Instance
IoReadBytesPerSecond	IO Read Bytes Per Second	BytesPerSecond	Total	IO Read Bytes Per Second	1 minute	Instance
IoWriteBytesPerSecond	IO Write Bytes Per Second	BytesPerSecond	Total	IO Write Bytes Per Second	1 minute	Instance
IoOtherBytesPerSecond	IO Other Bytes Per Second	BytesPerSecond	Total	IO Other Bytes Per Second	1 minute	Instance
IoReadOperationsPerSecond	IO Read Operations Per Second	BytesPerSecond	Total	IO Read Operations Per Second	1 minute	Instance
IoWriteOperationsPerSecond	IO Write Operations Per Second	BytesPerSecond	Total	IO Write Operations Per Second	1 minute	Instance

IoOtherOperationsPerSecond	IO Other Operations Per Second	BytesPerSecond	Total	IO Other Operations Per Second	1 minute	Instance
RequestsInApplicationQueue	Requests In Application Queue	Count	Average	Requests In Application Queue	1 minute	Instance
CurrentAssemblies	Current Assemblies	Count	Average	Current Assemblies	1 minute	Instance
TotalAppDomains	Total App Domains	Count	Average	Total App Domains	1 minute	Instance
TotalAppDomainsUnloaded	Total App Domains Unloaded	Count	Average	Total App Domains Unloaded	1 minute	Instance
Gen0Collections	Gen 0 Garbage Collections	Count	Total	Gen 0 Garbage Collections	1 minute	Instance
Gen1Collections	Gen 1 Garbage Collections	Count	Total	Gen 1 Garbage Collections	1 minute	Instance
Gen2Collections	Gen 2 Garbage Collections	Count	Total	Gen 2 Garbage Collections	1 minute	Instance

Table A.32: Microsoft.ContainerInstance/containerGroups and Microsoft.ContainerService/managedClusters

Metric	**Metric Display Name**	**Unit**	**Aggregation Type**	**Description**	**Sample Period**	**Dimensions**
CpuUsage	CPU Usage	Count	Average	CPU usage on all cores in millicores.	1 minute	containerName
MemoryUsage	Memory Usage	Bytes	Average	Total memory usage in byte.	1 minute	containerName

NetworkBytesReceivedPerSecond	Network Bytes Received Per Second	Bytes	Average	The network bytes received per second.	1 minute	No Dimensions
NetworkBytesTransmittedPerSecond	Network Bytes Transmitted Per Second	Bytes	Average	The network bytes transmitted per second.	1 minute	No Dimensions
kube_node_status_allocatable_cpu_-cores	Total number of available cpu cores in a managed cluster	Count	Total	Total number of available cpu cores in a managed cluster	1 minute	No Dimensions
kube_node_status_allocatable_mem-ory_bytes	Total amount of available memory in a managed cluster	Bytes	Total	Total amount of available memory in a managed cluster	1 minute	No Dimensions
kube_pod_status_ready	Number of pods in Ready state	Count	Total	Number of pods in Ready state	1 minute	namespace, pod
kube_node_status_condition	Statuses for various node conditions	Count	Total	Statuses for various node con-ditions	1 minute	condition, sta-tus, node
kube_pod_status_phase	Number of pods by phase	Count	Total	Number of pods by phase	1 minute	phase, names-pace, pod

Table A.33: Microsoft.Network/expressRouteCircuits

Metric	**Metric Dis-play Name**	**Unit**	**Aggregation Type**	**Description**	**Sample Period**	**Dimensions**

BitsInPerSecond	BitsInPerSecond	CountPerSecond	Average	Bits ingressing Azure per second	1 minute	No Dimensions
BitsOutPerSecond	BitsOutPerSecon	CountPerSecond	Average	Bits egressing Azure per second	1 minute	No Dimensions

Table A.34: Microsoft.Network/expressRouteCircuits

Metric	**Metric Display Name**	**Unit**	**Aggregation Type**	**Description**	**Sample Period**	**Dimensions**
CoreCount	Dedicated Core Count	Count	Total	Total number of dedicated cores in the batch account	1 minute	No Dimensions
TotalNodeCount	Dedicated Node Count	Count	Total	Total number of dedicated nodes in the batch account	1 minute	No Dimensions
LowPriorityCoreCount	LowPriority Core Count	Count	Total	Total number of low-priority cores in the batch account	1 minute	No Dimensions
TotalLowPriorityNodeCount	Low-Priority Node Count	Count	Total	Total number of low-priority nodes in the batch account	1 minute	No Dimensions
CreatingNodeCount	Creating Node Count	Count	Total	Number of nodes being created	1 minute	No Dimensions
StartingNodeCount	Starting Node Count	Count	Total	Number of nodes starting	1 minute	No Dimensions
WaitingForStartTaskNodeCount	Waiting For Start Task Node Count	Count	Total	Number of nodes waiting for the Start Task to complete	1 minute	No Dimensions
StartTaskFailedNodeCount	Start Task Failed Node Count	Count	Total	Number of nodes where the Start Task has failed	1 minute	No Dimensions

IdleNodeCount	Idle Node Count	Count	Total	Number of idle nodes	1 minute	No Dimensions
OfflineNodeCount	Offline Node Count	Count	Total	Number of offline nodes	1 minute	No Dimensions
RebootingNodeCount	Rebooting Node Count	Count	Total	Number of rebooting nodes	1 minute	No Dimensions
ReimagingNodeCount	Reimaging Node Count	Count	Total	Number of reimaging nodes	1 minute	No Dimensions
RunningNodeCount	Running Node Count	Count	Total	Number of running nodes	1 minute	No Dimensions
LeavingPoolNodeCount	Leaving Pool Node Count	Count	Total	Number of nodes leaving the Pool	1 minute	No Dimensions
UnusableNodeCount	Unusable Node Count	Count	Total	Number of unusable nodes	1 minute	No Dimensions
PreemptedNodeCount	Preempted Node Count	Count	Total	Number of preempted nodes	1 minute	No Dimensions
TaskStartEvent	Task Start Events	Count	Total	Total number of tasks that have started	1 minute	No Dimensions
TaskCompleteEvent	Task Complete Events	Count	Total	Total number of tasks that have completed	1 minute	No Dimensions
TaskFailEvent	Task Fail Events	Count	Total	Total number of tasks that have completed in a failed state	1 minute	No Dimensions
PoolCreateEvent	Pool Create Events	Count	Total	Total number of pools that have been created	1 minute	No Dimensions
PoolResizeStartEvent	Pool Resize Start Events	Count	Total	Total number of pool resizes that have started	1 minute	No Dimensions

PoolResizeCompleteEvent	Pool Resize Complete Events	Count	Total	Total number of pool resizes that have completed	1 minute	No Dimensions
PoolDeleteStartEvent	Pool Delete Start Events	Count	Total	Total number of pool deletes that have started	1 minute	No Dimensions
PoolDeleteCompleteEvent	Pool Delete Complete Events	Count	Total	Total number of pool deletes that have completed	1 minute	No Dimensions
JobDeleteCompleteEvent	Job Delete Complete Events	Count	Total	Total number of jobs that have been successfully deleted.	1 minute	No Dimensions
JobDeleteStartEvent	Job Delete Start Events	Count	Total	Total number of jobs that have been requested to be deleted.	1 minute	No Dimensions
JobDisableCompleteEvent	Job Disable Complete Events	Count	Total	Total number of jobs that have been successfully disabled.	1 minute	No Dimensions
JobDisableStartEvent	Job Disable Start Events	Count	Total	Total number of jobs that have been requested to be disabled.	1 minute	No Dimensions
JobStartEvent	Job Start Events	Count	Total	Total number of jobs that have been successfully started.	1 minute	No Dimensions
JobTerminateCompleteEvent	Job Terminate Complete Events	Count	Total	Total number of jobs that have been successfully terminated.	1 minute	No Dimensions
JobTerminateStartEvent	Job Terminate Start Events	Count	Total	Total number of jobs that have been requested to be terminated.	1 minute	No Dimensions

Table A.35: Microsoft.StreamAnalytics/streamingjobs

Metric	**Metric Display Name**	**Unit**	**Aggregation Type**	**Description**	**Sample Period**	**Dimensions**
ResourceUtilization	SU % Utilization	Percent	Maximum	SU % Utilization	1 minute	No Dimensions
InputEvents	Input Events	Count	Total	Input Events	1 minute	No Dimensions
InputEventBytes	Input Event Bytes	Bytes	Total	Input Event Bytes	1 minute	No Dimensions
LateInputEvents	Late Input Events	Count	Total	Late Input Events	1 minute	No Dimensions
OutputEvents	Output Events	Count	Total	Output Events	1 minute	No Dimensions
ConversionErrors	Data Conversion Errors	Count	Total	Data Conversion Errors	1 minute	No Dimensions
Errors	Runtime Errors	Count	Total	Runtime Errors	1 minute	No Dimensions
DroppedOrAdjustedEvents	Out of order Events	Count	Total	Out of order Events	1 minute	No Dimensions
AMLCalloutRequests	Function Requests	Count	Total	Function Requests	1 minute	No Dimensions
AMLCalloutFailedRequests	Failed Function Requests	Count	Total	Failed Function Requests	1 minute	No Dimensions
AMLCalloutInputEvents	Function Events	Count	Total	Function Events	1 minute	No Dimensions
DeserializationError	Input Deserialization Errors	Count	Total	Input Deserialization Errors	1 minute	No Dimensions

EarlyInputEvents	Early Input Events	Count	Total	Early Input Events	1 minute	No Dimensions
OutputWatermarkDelaySeconds	Watermark Delay	Seconds	Maximum	Watermark Delay	1 minute	No Dimensions

Table A.36: Regions

Code	Location
Central US	Iowa
East US 2	Virginia
East US	Virginia
North Central US	Illinois
South Central US	Texas
West US 2	Washington
West Central US	Wyoming
West US	California
Canada Central	Toronto
Canada East	Quebec City
Brazil South	Sao Paulo State
North Europe	Ireland
West Europe	Netherlands
France Central	Paris
France South	Marseille
UK South	London
UK West	Cardiff

Germany Central	Frankfurt
Germany Northeast	Magdeburg
East Asia	Hong Kong
Southeast Asia	Singapore
Australia Central	Canberra
Australia Central 2	Canberra
Australia East	New South Wales
Australia Southeast	Victoria
China East	Shanghai
China North	Beijing
China East 2	Shanghai
China North 2	Beijing
Central India	Pune
South India	Chennai
West India	Mumbai
Japan East	Tokyo, Saitama
Japan West	Osaka
Korea Central	Seoul
Korea South	Busan
South Africa North	Johannesburg
South Africa West	Cape Town

A.0.2 Repository of vendor-neutral metrics and their mappings with the metrics of the main cloud providers

Below is the set of vendor-neutral metrics and their corresponding mappings for each of the equivalent services from the major cloud providers.

Table A.37: VNM Regions

vendor-neutral region	AWS	GCP	M. Azure
US_EAST	us-east-1 us-east-2	us-east4 us-east1 us-central1	East US East US 2 Central US West US 2
US_WEST	us-west-1 us-west-2	us-west1 us-west2	West US West US 2 West Central US West Central US
CANADA	ca-central-1	northamerica-northeast1	Canada Central Canada East
SOUTH_AMERICA	sa-east-1	southamerica-east1	Brazil South
EU_CENTRAL	eu-central-1	europe-west3 europe-west6	Germany Central Germany Northeast
EU_WEST	eu-west-1 eu-west-2 eu-west-3	europe-west1 europe-west2 europe-west4	West Europe France Central France South
EU_NORTH	eu-north-1	europe-north1	North Europe UK South UK West
AP_NORTH	ap-northeast-1 ap-northeast-2 ap-northeast-3	asia-northeast1 asia-east1 asia-east2	Japan East Japan West Korea Central Korea South East Asia China East China North China East 2 China North 2

AP_SOUTH	ap-southeast-1 ap-southeast-2 ap-south-1	asia-southeast1 australia-southeast1 asia-south1	Southeast Asia Australia Central Australia Central 2 Australia East Australia Southeast West India Central India South India

Table A.38: VNM Stream Processing Related Metrics

Metric	Unit	AWS	GC	M. Azure
received_events	count	IncomingRecords	job/element_count	InputEvents
received_bytes	Bytes	IncomingBytes	job/estimated_byte_count	InputEventBytes
watermark_age	seconds	currentInputWatermark	job/data_watermark_age	OutputWatermarkDelaySeconds
failed_records	count	InputProcessing.ProcessingFailedRecords	job/is_failed	Errors

Table A.39: VNM Stream Processing Mapping

Metric	Unit	AWS	GC	M. Azure
received_events	count	=IncomingRecords	=job/element_count	=InputEvents
received_bytes	Bytes	=IncomingBytes	=job/estimated_byte_count	=InputEventBytes
watermark_age	seconds	=currentInputWatermark	=job/data_watermark_age	=OutputWatermarkDelaySeconds
failed_records	count	=InputProcessing.ProcessingFailedRecords	=job/is_failed	=Errors

Table A.40: VNM Batch Data Processing Related Metrics

Metric	Unit	AWS	GC	M. Azure
jobs_running	Count	AppsRunning	cluster/job/running_count	JobStartEvent
jobs_completed	Count	AppsCompleted	cluster/job/completion_time	JobTerminateCompleteEvent
jobs_failed	Count	AppsFailed	cluster/job/failed_count	JobDisableCompleteEvent
total_cores	Count	CoreNodesRunning CoreNodesPending	cluster/yarn/virtual_cores	CoreCount

Table A.41: VNM Batch Data Processing Mappings

Metric	Unit	AWS	GC	M. Azure
jobs_running	Count	=AppsRunning	=cluster/job/running_count	=JobStartEvent
jobs_completed	Count	=AppsCompleted	=count(cluster/job/completion_time)	=JobTerminateCompleteEvent
jobs_failed	Count	=AppsFailed	=cluster/job/failed_count"	JobDisableCompleteEvent
total_cores	Count	=CoreNodesRunning + CoreNodesPending	=cluster/yarn/virtual_cores	=CoreCount

Table A.42: VNM Dedicated Interconnect Related Metrics

Metric	Unit	AWS	GC	M. Azure
received_bytes_count	bits/s	ConnectionBpsIngress	network/attachment/received_bytes_count	BitsInPerSecond
sent_bytes_count	bits/s	ConnectionBpsEgress	network/attachment/sent_bytes_count	BitsOutPerSecond

Table A.43: VNM Dedicated Interconnect Processing Mappings

Metric	Unit	AWS	GC	M. Azure
received_bytes_count	bits/s	=ConnectionBpsIngress	=network/attachment/received_bytes_count * 8	=BitsInPerSecond
sent_bytes_count	bits/s	=ConnectionBpsEgress	=network/attachment/sent_bytes_count * 8	=BitsOutPerSecond

Table A.44: VNM Containers Services Related Metrics

Metric	Unit	AWS	GC	M. Azure
CPU_utilization	%	CPUUtilization	container/cpu/request_utilization	CpuUsage

memory_utilization	%	MemoryUtilization	container/memory/request_utilization	MemoryUsage
average_received_bytes	Bytes	(*)	node/network/received_bytes_count container/uptime	NetworkBytesReceivedPerSecond
average_sent_bytes	Bytes	(*)	node/network/sent_bytes_count container/uptime	NetworkBytesTransmittedPerSecond
allocatable_cores	count	(*)	node/cpu/allocatable_cores	kube_node_status_allocatable_cpu_cores
allocatable_memory	Bytes	(*)	node/memory/allocatable_bytes	kube_node_status_allocatable_memory_bytes

(*) The AWS Kubernetes service (Elastic Kubernetes Service or EKS) is relatively recent, and does not export metrics via CloudWatch. However, there are indications that it will export metrics in the future. This is the reason for including partial mappings.

Table A.45: VNM Containers Service Mappings

Metric	Unit	AWS	GC	M. Azure
CPU_utilization	%	=CPUUtilization	=request_utilization	=CpuUsage
memory_utilization	%	=MemUtilization	=container/memory/request_utilization	=MemoryUsage
average_received_bytes	Bytes	(*)	=node/network/received_bytes_count / container/uptime	=NetworkBytesReceivedPerSecond
average_sent_bytes	Bytes	(*)	=node/network/sent_bytes_count / container/uptime	=NetworkBytesTransmittedPerSecond
allocatable_cores	count	(*)	=node/cpu/allocatable_cores	=kube_node_status_allocatable_cpu_cores
allocatable_memory	Bytes	(*)	=node/memory/allocatable_bytes	=kube_node_status_allocatable_memory_bytes

(*) The AWS Kubernetes service (Elastic Kubernetes Service or EKS) is relatively recent, and does not export metrics via CloudWatch. However, there are indications that it will export metrics in the future. This is the reason for including partial mappings.

Table A.46: VNM MySQL Related Metrics

Metric	Unit	AWS	GC	M. Azure

CPU_utilization	%	CPUUtilization	database/cpu/utilization	cpu_percent
memory_usage	Bytes	FreeableMemory (Bytes) TotalMemory (GiB)	database/memory/usage	memory_percent memory_quota (GB)
memory_utilization	%	FreeableMemory (Bytes) TotalMemory (GiB)	database/memory/utilization	memory_percent
disk_usage	Bytes	FreeStorageSpace (Bytes) TotalDisk (GiB)	database/disk/bytes_used	storage_used
disk_utilization	%	FreeStorageSpace (Bytes) TotalDisk (GiB)	database/disk/utilization	storage_percent
disk_write_ops	Count/s	WriteIOPS	database/disk/write_ops_count	Data Disk Write Operations/Sec OS Disk Write Bytes/Sec
disk_read_ops	Count/s	ReadIOPS	database/disk/read_ops_count	Data Disk Read Operations/Sec OS Disk Read Bytes/Sec
network_received	Bytes/s	NetworkReceiveThroughput	database/network/received_bytes_count	network_bytes_ingress
network_transmitted	Bytes/s	NetworkTransmitThroughput	database/network/sent_bytes_count	network_bytes_egress
replica_lag	seconds	ReplicaLag	database/mysql/replication/seconds_behind_\master	seconds_behind_master
connections	Count	DatabaseConnections	database/network/connections	connection_successful

Table A.47: VNM MySQL Mappings (AWS and GC)

Metric	**Unit**	**AWS**	**GC**
CPU_utilization	%	=CPUUtilization	=database/cpu/utilization
memory_usage	Bytes	=2^30*TotalMemory-FreeableMemory	=database/memory/usage
memory_utilization	%	=(1-(FreeableMemory/2^30*TotalMemory))*100	=database/memory/utilization
disk_usage	Bytes	=2^30*TotalDisk-FreeStorageSpace	=database/disk/bytes_used

disk_utilization	%	=(1-(FreeStorageSpace/2^30*TotalDisk))*100	=database/disk/utilization
disk_write_ops	Count/s	=WriteIOPS	=database/disk/write_ops_count/60
disk_read_ops	Count/s	=ReadIOPS	=database/disk/read_ops_count/60
network_received	Bytes/s	=NetworkReceiveThroughput	=60*database/network/received_bytes_count
network_transmitted	Bytes/s	=NetworkTransmitThroughput	=60*database/network/sent_bytes_count
replica_lag	seconds	=ReplicaLag	=database/mysql/replication/seconds_behind_\master
connections	Count	=DatabaseConnections	=database/network/connections

Table A.48: VNM MySQL Mappings (M. Azure)

Metric	Unit	M. Azure
CPU_utilization	%	=cpu_percent
memory_usage	Bytes	=(memory_percent*100)/(memory_quota*10^9)
memory_utilization	%	=memory_percent
disk_usage	Bytes	=storage_used
disk_utilization	%	=storage_percent
disk_write_ops	Count/s	=Data Disk Write Operations/Sec + OS Disk Write Bytes/Sec
disk_read_ops	Count/s	=Data Disk Read Operations/Sec + OS Disk Read Bytes/Sec
network_received	Bytes/s	=60*network_bytes_ingress
network_transmitted	Bytes/s	=60*network_bytes_egress
replica_lag	seconds	=seconds_behind_master
connections	Count	=connection_successul

Table A.49: VNM PostgreSQL Related Metrics

Metric	Unit	AWS	GC	M. Azure
CPU_utilization	%	CPUUtilization	database/cpu/utilization	cpu_percent
memory_usage	Bytes	FreeableMemory (Bytes) TotalMemory (GiB)	database/memory/usage	memory_percent memory_quota (GB)
memory_utilization	%	FreeableMemory (Bytes) TotalMemory (GiB)	database/memory/utilization	memory_percent
disk_usage	Bytes	FreeStorageSpace (Bytes) TotalDisk (GiB)	database/disk/bytes_used	storage_used
disk_utilization	%	FreeStorageSpace (Bytes) TotalDisk (GiB)	database/disk/utilization	storage_percent
disk_write_ops	Count/s	WriteIOPS	database/disk/write_ops_count	Data Disk Write Operations/Sec OS Disk Write Bytes/Sec
disk_read_ops	Count/s	ReadIOPS	database/disk/read_ops_count	Data Disk Read Operations/Sec OS Disk Read Bytes/Sec
network_received	Bytes/s	NetworkReceiveThroughput	database/network/received_bytes_count	network_bytes_ingress
network_transmitted	Bytes/s	NetworkTransmitThroughput	database/network/sent_bytes_count	network_bytes_egress
connections	Count	DatabaseConnections	database/network/connections	connection_successful

Table A.50: VNM PostgreSQL Mappings (AWS and GC)

Metric	Unit	AWS	GC
CPU_utilization	%	=CPUUtilization	=database/cpu/utilization
memory_usage	Bytes	=2^30*TotalMemory-FreeableMemory	=database/memory/usage
memory_utilization	%	=(1-(FreeableMemory/2^30*TotalMemory))*100	=database/memory/utilization

disk_usage	Bytes	=2^30*TotalDisk-FreeStorageSpace	=database/disk/bytes_used
disk_utilization	%	=(1-(FreeStorageSpace/2^30*TotalDisk))*100	=database/disk/utilization
disk_write_ops	Count/s	=WriteIOPS	=database/disk/write_ops_count/60
disk_read_ops	Count/s	=ReadIOPS	=database/disk/read_ops_count/60
network_received	Bytes/s	=NetworkReceiveThroughput	=60*database/network/received_bytes_count
network_transmitted	Bytes/s	=NetworkTransmitThroughput	=60*database/network/sent_bytes_count
connections	Count	=DatabaseConnections	=database/network/connections

Table A.51: VNM PostgreSQL Mappings (M. Azure)

Metric	Unit	M. Azure
CPU_utilization	%	=cpu_percent
memory_usage	Bytes	=(memory_percent*100)/(memory_quota*10^9)
memory_utilization	%	=memory_percent
disk_usage	Bytes	=storage_used
disk_utilization	%	=storage_percent
disk_write_ops	Count/s	=Data Disk Write Operations/Sec + OS Disk Write Bytes/Sec
disk_read_ops	Count/s	=Data Disk Read Operations/Sec + OS Disk Read Bytes/Sec
network_received	Bytes/s	=60*network_bytes_ingress
network_transmitted	Bytes/s	=60*network_bytes_egress
connections	Count	=connection_successul

Table A.52: VNM Block Storage Related Metrics

Metric	Unit	AWS	GC	M. Azure
disk_read	Bytes/s	VolumeReadBytes	read_bytes_count	Data Disk Read Bytes/Sec OS Disk Read Bytes/Sec
disk_write	Bytes/s	VolumeWriteBytes	write_bytes_count	Data Disk Write Bytes/Sec OS Disk Write Bytes/Sec
queue_length	count	VolumeQueueLength	pending_operations	Data Disk QD OS Disk QD
io_operations	operations/s	VolumeReadOps VolumeWriteOps	operation_count operation_time	Data Disk Read Operations/Sec Data Disk Write Operations/Sec OS Disk Read Operations/Sec OS Disk Write Operations/Sec

Table A.53: VNM Block Storage Mappings (AWS and GC)

Metric	Unit	AWS	GC
disk_read	Bytes/s	=avg(VolumeReadBytes)/60	=read_bytes_count/uptime
disk_write	Bytes/s	=avg(VolumeWriteBytes)/60	=write_bytes_count/uptime
queue_length	count	=VolumeQueueLength	=pending_operations
io_operations	operations/s	=(VolumeReadOps + VolumeWriteOps)/60	=operation_count /operation_time

Table A.54: VNM Block Storage Mappings (M. Azure)

Metric	Unit	M. Azure

disk_read	Bytes/s	=Data Disk Read Bytes/Sec + OS Disk Read Bytes/Sec
disk_write	Bytes/s	=Data Disk Write Bytes/Sec + OS Disk Write Bytes/Sec
queue_length	count	=Data Disk QD + OS Disk QD
io_operations	operations/s	=Data Disk Read Operations/Sec + Data Disk Write Operations/Sec + OS Disk Read Operations/Sec + OS Disk Write Operations/Sec

Table A.55: VNM Object Storage Related Metrics

Metric	Unit	AWS	GC	M. Azure
objects	count	NumberOfObjects	storage/object_count	BlobCount
total_requests	count	AllRequests	api/request_count	Transactions
data_downloaded	Bytes	BytesDownloaded	network/sent_bytes_count	Egress
data_uploaded	Bytes	BytesUploaded	network/received_bytes_count	Ingress
storage_used	Bytes	BucketSizeBytes	storage/total_bytes	BlobCapacity
get_requests	count	GetRequests	api/request_count	Transactions
put_requests	count	PutRequests	api/request_count	Transactions
delete_requests	count	DeleteRequests	api/request_count	Transactions
head_requests	count	HeadRequests	api/request_count	Transactions
post_requests	count	PostRequests	api/request_count	Transactions
4xx_errors	count	4xxErrors	api/request_count	Transactions
5xx_errors	count	5xxErrors	api/request_count	Transactions

Table A.56: VNM Object Storage Mappings

Metric	Unit	AWS	GC	M. Azure
objects	count	=window(5,NumberOfObjects)	=storage/object_count	=window(5, BlobCount)
total_requests	count	=AllRequests	=sum(api/request_count)	=Transactions
data_downloaded	Bytes	=BytesDownloaded	=network/sent_bytes_count	=Egress
data_uploaded	Bytes	=BytesUploaded	=network/received_bytes_count	=Ingress
storage_used	Bytes	=BucketSizeBytes	=storage/total_bytes	=BlobCapacity
get_requests	count	=GetRequest	=api/request_count	=Transactions
put_requests	count	=PutRequests	=api/request_count	=Transactions
delete_requests	count	=DeleteRequests	=api/request_count	=Transactions
head_requests	count	=HeadRequests	=api/request_count	=Transactions
post_requests	count	=PostRequests	=api/request_count	=Transactions
4xx_errors	count	=4xxErrors	=api/request_count	=Transactions
5xx_errors	count	=5xxErrors	=api/request_count	=Transactions

Table A.57: VNM Network Load Balancer Related Metrics

Metric	Unit	AWS	GC	M. Azure
active_connections	Count	ActiveFlowCount	tcp_ssl_proxy/open_connections	P2SConnectionCount
processed_bytes	bytes/s	ProcessedBytes	l3/internal/ingress_bytes_count	TunnelIngressBytes

Table A.58: VNM Network Load Balancer Mappings

Metric	Unit	AWS	GC	M. Azure

active_connections	Count	=ActiveFlowCount	=tcp_ssl_proxy/open_connections	=P2SConnectionCount
processed_bytes	bytes/s	=ProcessedBytes/uptime	=sum(l3/internal/ingress_bytes_count)/uptime	=TunnelIngressBytes

Table A.59: VNM Application Load Balancer Related Metrics

Metric	Unit	AWS	GC	M. Azure
active_connections	Count	=ActiveFlowCount	=tcp_ssl_proxy/open_connections	=P2SConnectionCount
processed_bytes	bytes/s	=ProcessedBytes/uptime	=sum(l3/internal/ingress_bytes_count)/uptime	=TunnelIngressBytes

Table A.60: VNM Application Load Balancer Mappings

Metric	Unit	AWS	GC	M. Azure
active_connections	Count	=ActiveFlowCount	=tcp_ssl_proxy/open_connections	=P2SConnectionCount
processed_bytes	bytes/s	=ProcessedBytes/uptime	=sum(l3/internal/ingress_bytes_count)/uptime	=TunnelIngressBytes